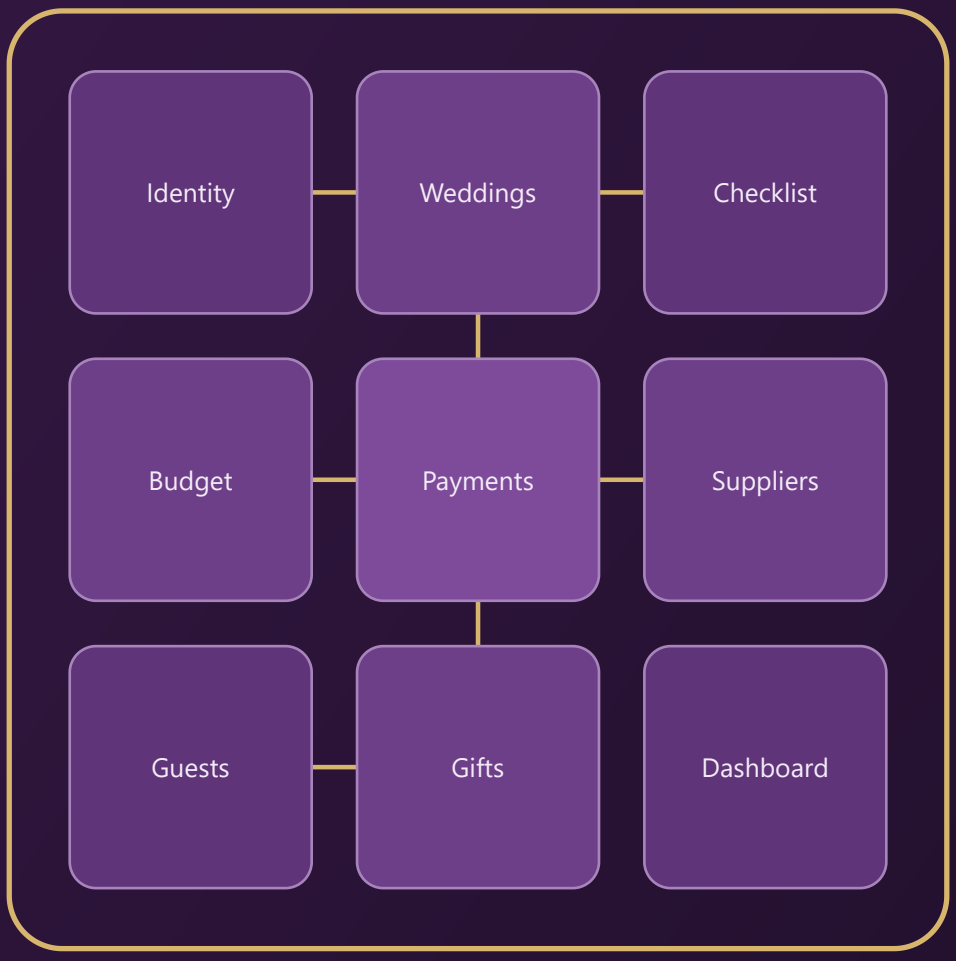
ARQUITETURA DE SOFTWARE

# Monolito **Modular** na Prática

Fundamentos, padrões de mercado e a construção passo a passo do Laçoos, uma API real em .NET 10




Material de estudo baseado no repositório LacoosApi
Novembro de 2026

# Sumário

Como ler este livro 3

PARTE I · ENTENDENDO O PROBLEMA

1 Por que software fica difícil de mudar 6
2 Do monolito aos microsserviços 11
3 O que faz um módulo ser um módulo 17

PARTE II · DESENHANDO AS FRONTEIRAS

4 Encontrando as fronteiras com DDD 25
5 O host: onde os módulos se encontram 31

PARTE III · CONSTRUINDO OS MÓDULOS

6 Anatomia interna de um módulo 39
7 Contratos e comunicação entre módulos 47
8 Dados: cada módulo é dono dos seus 54
9 Consistência e comunicação assíncrona 62
10 O que é compartilhado: kernel e temas transversais 71
11 Telas que precisam de vários módulos 81

PARTE IV · MANTENDO A ARQUITETURA VIVA

12 Testes em um monolito modular 88
13 Arquitetura como código 96
14 Entrega e operação 104

PARTE V · EVOLUÇÃO E MERCADO

15 Evoluindo: quando e como extrair um módulo 113
16 O que o mercado aprendeu 118
17 O Laçoos de ponta a ponta 122

APÊNDICES

Apêndice A: Checklist de revisão 128
Apêndice B: Glossário 130
Apêndice C: Referências 132

# Como ler este livro

Este livro ensina monolito modular de duas formas ao mesmo tempo. Cada capítulo explica um conceito por inteiro, sem pressupor que você já o conheça. Em seguida, aplica esse conceito na construção de um sistema real: o **Laçoos**, uma API para casais e organizadores planejarem um casamento.

A ideia é que você nunca fique só na teoria nem só no código. Você entende por que uma prática existe, vê como o mercado a usa, e depois encontra a mesma prática funcionando em um projeto que compila, roda e tem 236 testes passando.

## Para quem é

Para quem desenvolve back-end e já sabe construir uma API, mas quer aprender a organizar um sistema que vai crescer. Ajuda conhecer C# e ASP.NET Core, porque os exemplos usam .NET 10, Entity Framework Core e PostgreSQL. Os conceitos, porém, valem para qualquer linguagem. O capítulo 16 mostra como Java, Ruby e Go tratam os mesmos problemas.

## Como cada capítulo é organizado

Cada capítulo abre com um mapa das seções e segue três movimentos:

1. **Conceito.** O problema, a ideia, as alternativas que o mercado usa e os custos de cada uma.
2. **Construindo o Laçoos.** A faixa verde marca o passo em que o conceito vira código. Os passos são numerados ao longo de todo o livro.
3. **Resumo e exercícios.** O essencial em poucas linhas e perguntas para você testar o que entendeu.

Quatro tipos de caixa aparecem pelo caminho:

> **Para lembrar**
> A ideia central daquele trecho, para guardar.

> **Analogia**
> Uma comparação com o mundo real para deixar um conceito abstrato mais concreto.

> **Cuidado**
> Um erro comum, ou uma armadilha que custa caro.

> **Visão crítica**
> O que o Laçoos fez de forma imperfeita ou o preço que pagou. Um projeto real tem concessões, e aprender a enxergá-las vale tanto quanto aprender os padrões.

## O código deste livro é real

Os blocos de código com um caminho de arquivo na legenda foram **extraídos automaticamente do repositório** no momento em que este PDF foi gerado. Eles não foram redigitados nem simplificados. Quando um trecho do meio foi omitido, aparece um comentário `// ...`.

Blocos com a legenda amarela, **"Exemplo ilustrativo"**, não existem no repositório. Eles mostram técnicas que o Laçoos não usa, como eventos e Outbox, para você saber como seriam.

## Uma observação honesta sobre a ordem

Os passos deste livro seguem uma **ordem didática**, não o histórico real de construção do projeto. Um sistema de verdade não nasce em uma linha reta. Aqui, os passos foram organizados para que cada conceito apareça no momento em que faz mais sentido.

Você também não vai digitar as quase 10 mil linhas do projeto. O livro constrói o esqueleto completo e aprofunda um módulo, o `Budget`, com visitas ao `Payments`, ao `Dashboard` e a outros quando eles ensinam algo novo. Os módulos restantes repetem a mesma estrutura, e você pode lê-los no repositório.

Se o tema é novo para você, leia na ordem, porque cada passo depende do anterior. Se já conhece a teoria, os capítulos 8, 9 e 13 trazem as decisões mais difíceis, e o capítulo 17 mostra o sistema inteiro em poucas páginas.

## Como rodar o projeto

Você precisa de Docker com Compose. Na raiz do repositório:

Subindo o Laçoos

```
cp .env.example .env
docker compose up --build
```

A API sobe em `http://localhost:8080`, o Swagger UI em `http://localhost:8080/swagger` e a caixa de e-mails de desenvolvimento em `http://localhost:8025`. Para rodar os testes, com o Docker ligado:

Rodando os testes

```
dotnet test --solution LacoosApi.sln
```

PARTE I

# Entendendo o problema

Antes de falar em módulos, precisamos entender o que estamos tentando resolver: por que sistemas ficam difíceis de mudar, o que cada estilo de arquitetura oferece e o que, afinal, faz um módulo ser um módulo.

CAPÍTULO 1

# Por que software fica difícil de mudar

*Arquitetura não é sobre desenhar caixas bonitas. É sobre manter barato o custo da próxima mudança.*


1.1 O sintoma que todo mundo conhece
1.2 Acoplamento e coesão
1.3 A grande bola de lama
1.4 A lei de Conway
1.5 O que a arquitetura compra
1.6 Conhecendo o Laçoos
1.7 Resumo e exercícios


## 1.1 O sintoma que todo mundo conhece

No começo de um projeto, tudo é rápido. Você adiciona uma tela em uma tarde, muda uma regra em minutos, e qualquer pessoa do time entende o sistema inteiro.

Dois anos depois, a mesma equipe leva uma semana para mudar um campo. Corrigir um cálculo de pagamento quebra um relatório de convidados. Ninguém sabe se pode apagar uma classe. Antes de cada entrega, alguém diz "vamos testar tudo de novo, por garantia".

O que aconteceu? O sistema não ficou lento porque tem mais linhas de código. Ficou lento porque **cada mudança passou a exigir que você entenda e toque em muita coisa**. Esse é o problema central da arquitetura de software, e tem nome: complexidade acidental que se acumula por causa de dependências descontroladas.

> **Para lembrar**
>
> O objetivo de uma boa arquitetura é simples de dizer: **uma mudança de negócio deve exigir mudar pouco código, em um lugar previsível, sem medo de quebrar o resto.**

## 1.2 Acoplamento e coesão

Dois conceitos dos anos 1970, formulados por Larry Constantine e Edward Yourdon no *structured design*, continuam sendo a base de quase tudo que vem depois neste livro.

**Coesão** mede o quanto as coisas que estão juntas pertencem de fato ao mesmo assunto. Um módulo de pagamentos que só trata de parcelas, baixas e comprovantes tem alta coesão. Uma classe `Utils` com formatação de CPF, envio de e-mail e cálculo de juros tem baixa coesão.

**Acoplamento** mede o quanto uma parte depende de outra. Se mudar a tabela de orçamento obriga a mudar o código de pagamentos, os dois estão acoplados.

A meta é **alta coesão e baixo acoplamento**: o que muda junto fica junto, e o que muda separado fica separado.

### Nem todo acoplamento é igual

Dizer "baixo acoplamento" é fácil. Na prática, ajuda saber que existem tipos diferentes, alguns muito mais perigosos:

| Tipo | Como aparece | Gravidade |
|---|---|---|
| **De conteúdo** | Um módulo lê ou altera diretamente os dados internos de outro | Muito alta |
| **Por banco de dados** | Dois módulos fazem `JOIN` nas mesmas tabelas | Alta |
| **Temporal** | Uma operação só funciona se outra terminar antes | Média |
| **Semântico** | Dois lugares precisam concordar sobre o que um valor significa | Média e invisível |
| **Por contrato** | Um módulo chama uma interface pública de outro | Baixa: é o tipo que queremos |

O acoplamento nunca chega a zero. Se duas partes nunca conversam, elas não formam um sistema. O trabalho do arquiteto é **escolher qual acoplamento aceitar**: trocar o tipo perigoso pelo tipo controlado.

## 1.3 A grande bola de lama

Em 1997, Brian Foote e Joseph Yoder publicaram um artigo chamado *Big Ball of Mud*. A tese deles foi desconfortável: a arquitetura mais comum do mundo não é nenhuma das que aparecem nos livros, é a **bola de lama**, um sistema sem estrutura perceptível, em que tudo está ligado a tudo.

Ninguém decide criar uma bola de lama. Ela nasce de decisões locais razoáveis: "é só um `JOIN`", "vou reaproveitar essa entidade aqui", "é mais rápido chamar direto". Cada atalho é pequeno. A soma deles é fatal.

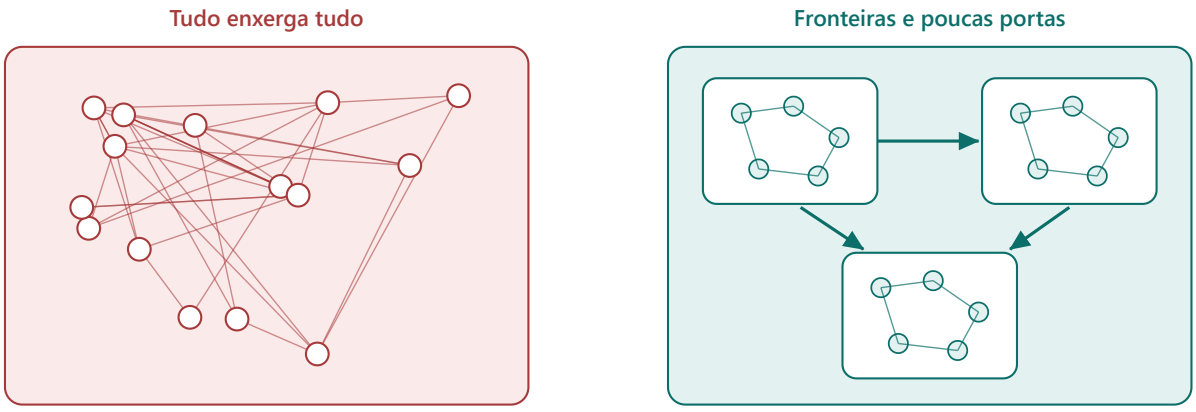



**Figura 1.1.** À esquerda, sem fronteiras, qualquer classe alcança qualquer outra. À direita, as dependências internas continuam, mas entre os grupos só há poucas portas conhecidas.

> **Analogia**
>
> Pense em uma cidade sem bairros, sem ruas planejadas e sem endereços. Cada casa puxou sua própria ligação de água e luz da casa vizinha. Funciona, até o dia em que você precisa trocar um cano. Módulos são os bairros: cada um tem sua vida interna, e o que entra e sai passa por vias conhecidas.

## 1.4 A lei de Conway

Em 1968, Melvin Conway observou algo que parece piada e virou lei: **organizações produzem sistemas que copiam suas estruturas de comunicação**. Se três equipes constroem um compilador, sai um compilador de três passos.

Isso importa aqui por dois motivos.

Primeiro, fronteiras técnicas e fronteiras de pessoas tendem a se alinhar. Se o time de pagamentos e o time de convidados mexem no mesmo arquivo todo dia, o conflito não é só técnico. Bons módulos reduzem a quantidade de coordenação necessária.

Segundo, a lei tem uma versão inversa, a *manobra inversa de Conway*: se você quer uma arquitetura, organize as equipes de forma compatível com ela. O livro *Team Topologies*, de Skelton e Pais, leva essa ideia adiante e usa **carga cognitiva** como critério: um time só consegue manter bem aquilo que cabe na cabeça dele. Fronteiras claras são uma forma de limitar o que cada pessoa precisa saber.

## 1.5 O que a arquitetura compra

Vale ser concreto. Uma boa modularização oferece:

- **Compreensão local.** Você entende e muda uma parte sem carregar o sistema inteiro na cabeça.
- **Mudança com segurança.** Os efeitos de uma alteração ficam contidos.
- **Trabalho em paralelo.** Pessoas diferentes mexem em partes diferentes com poucos conflitos.
- **Testes mais fáceis.** Partes com dependências explícitas são mais simples de isolar.
- **Liberdade futura.** Trocar uma tecnologia ou extrair um serviço fica viável.

Ela também tem custo: mais estrutura, mais código de ligação e mais regras para seguir. Em um script de cem linhas, nada disso compensa. Arquitetura é um investimento que se paga em sistemas com vida longa, com regras de negócio de verdade, ou mantidos por mais de uma pessoa.

CONSTRUINDO O LAÇOOS · PASSO 1

# Conhecendo o Laçoos

Nosso projeto-guia é a API do **Laçoos**, um sistema de administração para quem está planejando um casamento. Já existe um front-end em Angular usando dados falsos. Nosso trabalho é criar o back-end real.

## O que o sistema faz

- **Contas:** cadastro, login, recuperação de senha.
- **Casamentos:** um usuário pode participar de vários casamentos, com papéis diferentes.
- **Equipe:** convites por e-mail para outras pessoas ajudarem.
- **Checklist:** tarefas com prazo, prioridade, categoria e responsável.
- **Orçamento:** categorias, itens, valor planejado e valor contratado.
- **Convidados:** grupos, confirmação de presença e acompanhantes.
- **Fornecedores:** cadastro, serviços e contratos com arquivo anexo.
- **Pagamentos:** parcelas, baixas parciais, estornos e comprovantes.
- **Presentes:** lista, reservas e recebimentos.
- **Dashboard:** uma tela com o resumo de tudo.

## Por que ele é um bom exemplo

O Laçoos parece simples, mas tem o tipo de problema que faz sistemas virarem bola de lama. Veja como os assuntos se cruzam:

- Um **pagamento** pode estar ligado a um **fornecedor** e a um **item de orçamento**.
- Um **fornecedor** não pode ser excluído se tiver **pagamentos** ou **itens de orçamento**.
- Excluir um **convidado** não pode apagar o histórico de quem deu um **presente**.
- O **dashboard** precisa de dados de seis áreas diferentes.
- Tudo pertence a um **casamento**, e nenhum casamento pode ver os dados de outro.

Se você modelar isso da forma mais rápida, com uma única camada de entidades ligadas por propriedades de navegação, em poucos meses qualquer mudança vai tocar em tudo. Esse é o problema que vamos resolver.

## As restrições do projeto

Toda arquitetura responde a restrições. As do Laçoos são:

| Restrição | Consequência |
|---|---|
| Equipe pequena | Operar muitos serviços custaria mais do que ajudaria |
| Deve subir com `docker compose up --build` | Implantação simples, poucos processos |
| Dados de casamentos diferentes não podem se misturar | Isolamento precisa ser garantido por padrão |

| Restrição | Consequência |
|---|---|
| Regras financeiras exigem precisão | Transações e valores decimais exatos |
| O produto vai crescer | Fronteiras precisam existir desde o início |
| Hospedagem barata | Um processo e um banco, em vez de vários |

A combinação pede uma pergunta: que estilo de arquitetura entrega fronteiras fortes sem o custo de um sistema distribuído? É o assunto do próximo capítulo.

## 1.7 Resumo e exercícios

- Sistemas ficam lentos de mudar porque as dependências crescem sem controle, não porque têm muito código.
- A meta é alta coesão e baixo acoplamento, escolhendo conscientemente quais dependências aceitar.
- A bola de lama nasce de pequenos atalhos razoáveis, e não de uma má decisão isolada.
- A lei de Conway liga a estrutura do software à comunicação das pessoas.

### Exercícios

1. Pense em um sistema que você conhece. Qual foi a última mudança pequena que exigiu mexer em muitos lugares? Que tipo de acoplamento da tabela causou isso?
2. Explique com suas palavras por que acoplamento por banco de dados costuma ser mais perigoso do que acoplamento por contrato.
3. Na lista de cruzamentos do Laçoos, escolha um e descreva como ele viraria um problema em um sistema sem fronteiras.

CAPÍTULO 2

# Do monolito aos microsserviços

*A pergunta certa não é "monolito ou microsserviços?". São duas perguntas diferentes, e confundi-las é o erro mais caro desta discussão.*


2.1 Duas perguntas que costumam virar uma
2.2 O monolito
2.3 Os microsserviços
2.4 O monolito modular
2.5 Como decidir
2.6 O que um monolito modular não resolve
2.7 Registrando a decisão de arquitetura
2.8 Resumo e exercícios


## 2.1 Duas perguntas que costumam virar uma

Muita gente descreve a escolha de arquitetura como uma linha: de um lado, o monolito "ruim"; do outro, os microsserviços "modernos". Essa visão mistura duas decisões independentes:

1. **Modularidade lógica:** o código tem fronteiras fortes entre as partes?
2. **Modularidade física:** as partes rodam e são implantadas separadamente?

Separar as duas muda toda a conversa. Elas formam quatro combinações, não duas.

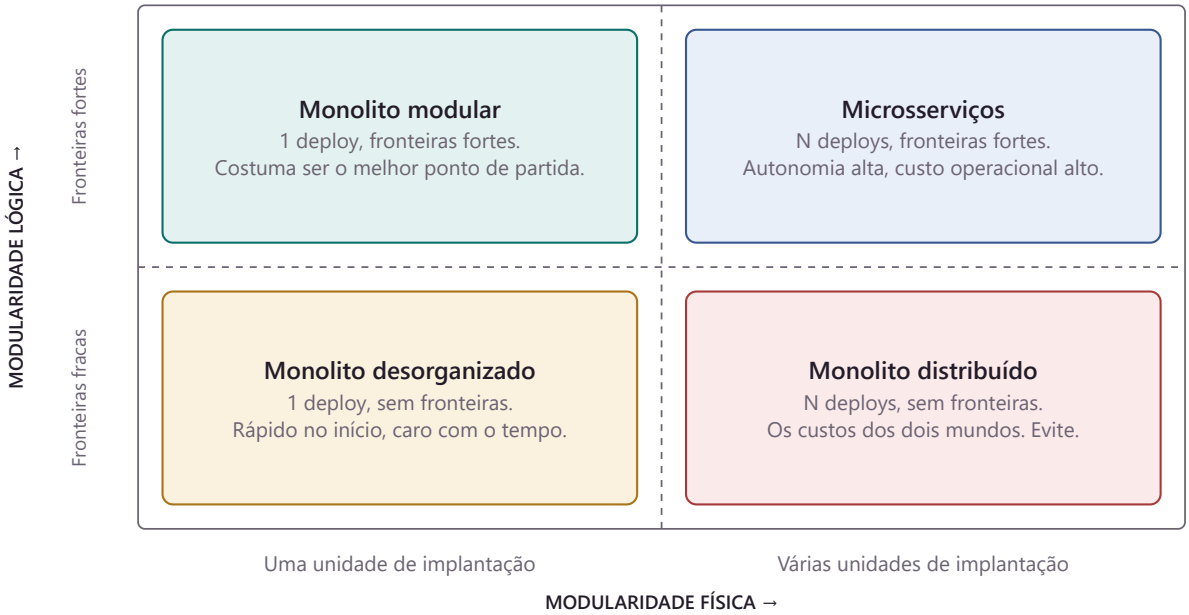



**Figura 2.1.** As duas decisões são independentes. Microsserviços resolvem a dimensão física, mas só entregam valor se a lógica também estiver certa.

## 2.2 O monolito

Um **monolito** é um sistema implantado como uma unidade só. Todo o código roda em um processo e é publicado de uma vez. A palavra não é um defeito. Ela descreve a forma de implantação, não a qualidade do código.

Sam Newman, em *Monolith to Microservices*, separa alguns tipos. O **monolito de processo único** é o caso clássico. O **monolito modular** é um subconjunto dele, com módulos separados dentro do mesmo processo. O **monolito distribuído** é uma armadilha: vários serviços que precisam ser implantados juntos.

Vantagens reais de um monolito:

- **Simplicidade operacional.** Um artefato, um pipeline, um lugar para olhar os logs.
- **Chamadas em memória.** São muito mais rápidas e confiáveis do que chamadas pela rede.
- **Transações reais.** Um único banco oferece ACID sem esforço.
- **Refatoração fácil.** A IDE renomeia tudo de uma vez, e o compilador avisa o que quebrou.
- **Depuração direta.** Você segue uma requisição com um único depurador.

O problema não é o monolito. É o **monolito sem fronteiras**, em que nada impede a bola de lama do capítulo anterior.

## 2.3 Os microsserviços

Na arquitetura de **microsserviços**, o sistema é formado por serviços pequenos, cada um com seu processo, seu ciclo de implantação e seus dados. Eles se comunicam pela rede.

O que eles oferecem de verdade:

- **Implantação independente.** Um time publica sem coordenar com os outros.
- **Escala por componente.** Só a parte com mais carga recebe mais recursos.
- **Isolamento de falhas.** Se bem desenhado, um serviço caído não derruba todos.
- **Liberdade tecnológica.** Cada serviço pode usar a pilha mais adequada.
- **Fronteiras físicas.** Fica difícil acessar o interior de outro serviço por acidente.

Esses ganhos são importantes para organizações grandes. Mas têm um preço que raramente aparece nas palestras.

### O preço da rede

Em 1994, Peter Deutsch reuniu as **falácias da computação distribuída**, suposições que quem começa em sistemas distribuídos costuma fazer e que são falsas. A lista final tem oito itens: a rede é confiável; a latência é zero; a banda é infinita; a rede é segura; a topologia não muda; há um único administrador; o custo de transporte é zero; a rede é homogênea.

Trocar uma chamada de método por uma chamada de rede muda a natureza do problema:

| Com chamada em memória | Com chamada pela rede |
|---|---|
| Funciona ou lança exceção | Pode dar timeout sem você saber se executou |
| Nanossegundos | Milissegundos, às vezes segundos |
| Transação cobre tudo | Consistência precisa ser projetada |
| Refatoração com a IDE | Mudança coordenada entre repositórios |
| Um depurador | Rastreamento distribuído |
| Um deploy | Vários pipelines, versões e compatibilidade |

Some a isso a infraestrutura: descoberta de serviços, filas, gateway, observabilidade distribuída, gestão de segredos e rede entre contêineres. Essas ferramentas resolvem problemas reais, mas exigem pessoas para operar.

> **Microsserviços não consertam um código desorganizado**
>
> Simon Brown resumiu bem na palestra *Modular Monoliths* (GOTO Berlin, 2018): se você não consegue construir um monolito bem estruturado, o que faz você pensar que microsserviços são a resposta? Se as fronteiras estão erradas no monolito, ao dividir você terá as mesmas fronteiras erradas, agora com latência e falhas de rede. Esse é o **monolito distribuído**.

## 2.4 O monolito modular

Um **monolito modular** é um sistema implantado como uma unidade, mas dividido internamente em módulos com fronteiras reais, verificadas e independentes.

Ele reúne as qualidades que costumam fazer falta em cada extremo:

| Característica | Monolito comum | Monolito modular | Microsserviços |
|---|---|---|---|
| Implantação | Uma | Uma | Várias e independentes |
| Comunicação interna | Livre | Em memória, só por contratos | Rede |
| Dados | Compartilhados | Mesmo banco, com posse por módulo | Um banco por serviço |
| Transações | Simples | Simples dentro do módulo | Sagas e consistência eventual |
| Escala | Do sistema inteiro | Do sistema inteiro | Por serviço |
| Autonomia de times | Baixa | Média | Alta |
| Custo operacional | Baixo | Baixo | Alto |
| Maior risco | Virar bola de lama | Fronteiras erodirem | Complexidade distribuída |

A aposta é esta: boa parte do que as empresas buscam nos microsserviços vem das **fronteiras**, não da **rede**. Se for assim, dá para obter o benefício principal sem pagar o preço principal.

> **Para lembrar**
>
> Monolito modular é **modularidade lógica forte com modularidade física simples**. Ele adia a decisão física até que exista um motivo concreto para tomá-la.

## 2.5 Como decidir

Martin Fowler defendeu em 2015, no texto *MonolithFirst*, que quase todos os casos de sucesso que ele conhecia com microsserviços começaram como monolito. O motivo é que acertar fronteiras exige conhecer o domínio, e isso só acontece com o tempo. Corrigir uma fronteira errada dentro de um processo é uma refatoração. Corrigir entre serviços é um projeto. Ele próprio avisa que a evidência é limitada, então trate a ideia como uma boa aposta, e não como lei.

Use a tabela abaixo como ponto de partida:

| Sinal | Aponta para |
|---|---|
| Time pequeno, ou um único time | Monolito modular |
| Domínio ainda sendo descoberto | Monolito modular |
| Transações fortes entre áreas | Monolito modular |
| Sem equipe de plataforma | Monolito modular |
| Vários times bloqueando uns aos outros em entregas | Considere extrair serviços |
| Uma parte com carga muito diferente das outras | Considere extrair essa parte |
| Exigência de isolamento regulatório ou de segurança | Considere extrair essa parte |
| Necessidade real de outra tecnologia em uma parte | Considere extrair essa parte |

Note que a lista fala em extrair **uma parte**, não em reescrever tudo. Um monolito modular com um ou dois serviços extraídos é uma arquitetura perfeitamente respeitável.

## 2.6 O que um monolito modular não resolve

Para não trocar um modismo por outro, seja honesto sobre os limites:

- **Implantação conjunta.** Um bug em um módulo pode bloquear a entrega de todos.
- **Falha compartilhada.** Um vazamento de memória em um módulo derruba o processo inteiro.
- **Escala conjunta.** Você escala tudo, mesmo que só uma parte precise.
- **Uma só tecnologia.** Em geral, todos os módulos usam a mesma linguagem e versão do runtime.
- **Disciplina obrigatória.** A fronteira não é física. Sem verificação automática, ela erode, e é por isso que o capítulo 13 existe.

CONSTRUINDO O LAÇOOS · PASSO 2

### Registrando a decisão de arquitetura

Vamos aplicar a tabela às restrições do Laçoos: um time pequeno, um domínio novo, regras financeiras, sem equipe de plataforma e hospedagem barata. A decisão é clara. Mas uma decisão importante precisa ficar escrita.

Uma prática muito usada é o **ADR** (*Architecture Decision Record*), proposta por Michael Nygard em 2011. É um documento curto que registra uma decisão, o contexto e as consequências. Daqui a dois anos, alguém vai perguntar "por que fizeram assim?", e o ADR responde. Veja como ficaria o primeiro do Laçoos:

**ADR 001: estilo de arquitetura** · Exemplo ilustrativo: não faz parte do repositório

```
# ADR 001: Monolito modular

Status: aceita

## Contexto
Sistema novo, equipe pequena, domínio em descoberta. Pagamentos, orçamento e
fornecedores têm regras que se cruzam. A entrega precisa funcionar com
`docker compose up --build`.

## Decisão
Construir um monolito modular em ASP.NET Core: um processo, um banco PostgreSQL,
nove módulos de negócio com fronteiras verificadas por testes.

## Consequências
+ Operação simples e barata.
+ Transações locais em cada módulo.
+ Módulos podem ser extraídos no futuro, se houver motivo.
- Implantação e falhas são compartilhadas.
- Fronteiras dependem de regras automáticas, não de separação física.
- Operações entre módulos não têm uma transação única.
```

Repare que a decisão lista o que perdemos. Um ADR que só mostra vantagens não está registrando uma decisão, está vendendo uma ideia.

Essa escolha também define uma regra do projeto, escrita desde o início: **não criar microsserviços**. Ela não significa "nunca". Significa que qualquer extração futura precisa de um motivo concreto, como veremos no capítulo 15.

## 2.8 Resumo e exercícios

- Modularidade lógica e modularidade física são decisões independentes.
- Microsserviços oferecem autonomia e escala seletiva ao custo da complexidade distribuída.
- Monolito modular combina fronteiras fortes com implantação simples.
- Ele tem limites reais: falha, escala e implantação compartilhadas, e depende de disciplina automatizada.

- Decisões importantes devem ser registradas em ADRs, com as consequências negativas.

## Exercícios

1. Em qual quadrante da Figura 2.1 está o sistema em que você trabalha? Que evidência sustenta sua resposta?
2. Dê um exemplo de falha que só existe em chamadas de rede e explique como você trataria essa falha.
3. Escreva um ADR de dez linhas para uma decisão recente do seu projeto. Liste pelo menos duas consequências negativas.

CAPÍTULO 3

# O que faz um módulo ser um módulo

*Uma pasta não é um módulo. Um namespace também não. Módulo é uma promessa sobre o que os outros podem e não podem saber.*


3.1 A ideia de Parnas
3.2 As propriedades de um bom módulo
3.3 Módulos profundos
3.4 Organizar por camada ou por capacidade
3.5 Regras sobre dependências
3.6 Como impor a fronteira
3.7 O esqueleto da solução
3.8 Resumo e exercícios


## 3.1 A ideia de Parnas

Em 1972, David Parnas publicou *On the Criteria To Be Used in Decomposing Systems into Modules*. Ele comparou duas maneiras de dividir o mesmo programa.

A primeira dividia pelos **passos do processamento**: ler a entrada, processar, gerar a saída. É a divisão que parece natural. A segunda dividia pelas **decisões de projeto que poderiam mudar**: cada módulo escondia uma decisão, como o formato de armazenamento.

As duas funcionavam. Mas, na primeira, uma mudança de formato espalhava alterações por todos os módulos. Na segunda, ela ficava contida em um só.

Daí vem o princípio do **ocultamento de informação** (*information hiding*): cada módulo esconde uma decisão que pode mudar. O que os outros veem é só uma interface que continua estável.

> **Para lembrar**
>
> Um módulo não é definido pelo que ele faz. É definido pelo que ele **esconde**. Se todos podem ver e usar o interior, não existe módulo, por mais organizadas que sejam as pastas.

## 3.2 As propriedades de um bom módulo

Reunindo o que a literatura e a prática ensinam, um módulo de verdade tem cinco propriedades:

1. **Responsabilidade clara.** Dá para dizer em uma frase de que assunto de negócio ele cuida.
2. **Interface explícita.** Existe uma lista conhecida do que os outros podem usar.
3. **Interior oculto.** O resto é inacessível, de preferência por força do compilador.
4. **Dados próprios.** Só ele lê e grava suas tabelas.
5. **Dependências declaradas.** Ao olhar o módulo, você vê de quem ele depende.

Compare com o que costuma ser chamado de módulo em muitos projetos:

| O que existe | Falta o quê |
|---|---|
| Uma pasta `Pagamentos/` com classes públicas | Interior oculto |
| Um projeto separado, mas que acessa as tabelas de outro | Dados próprios |
| Uma classe `PagamentoService` que todos chamam | Interface pensada e responsabilidade delimitada |
| Um namespace | Quase tudo: é só um nome |

## 3.3 Módulos profundos

John Ousterhout, em *A Philosophy of Software Design*, propõe uma imagem útil. Pense em um módulo como um retângulo: a largura é o tamanho da interface, a área é a funcionalidade que ele oferece. Um **módulo profundo** tem interface pequena e muita funcionalidade atrás dela. Um **módulo raso** tem uma interface quase tão grande quanto a implementação.

O melhor exemplo são as chamadas de arquivo do Unix: `open`, `read`, `write`, `close` escondem discos, cache, permissões e sistemas de arquivos. Cinco funções, décadas de complexidade atrás.

Essa imagem vale em dois níveis. Vale para classes e vale para módulos de negócio. Um contrato com três métodos que esconde mil linhas é ótimo. Um contrato com quarenta métodos que apenas repassam chamadas ao banco é um sinal de fronteira ruim.

## 3.4 Organizar por camada ou por capacidade

A escolha mais importante da estrutura de pastas é o primeiro nível. Durante anos, a organização mais comum foi por **camada técnica**, com `Controllers/`, `Services/`, `Repositories/`. Em um monolito modular, a organização é por **capacidade de negócio**.

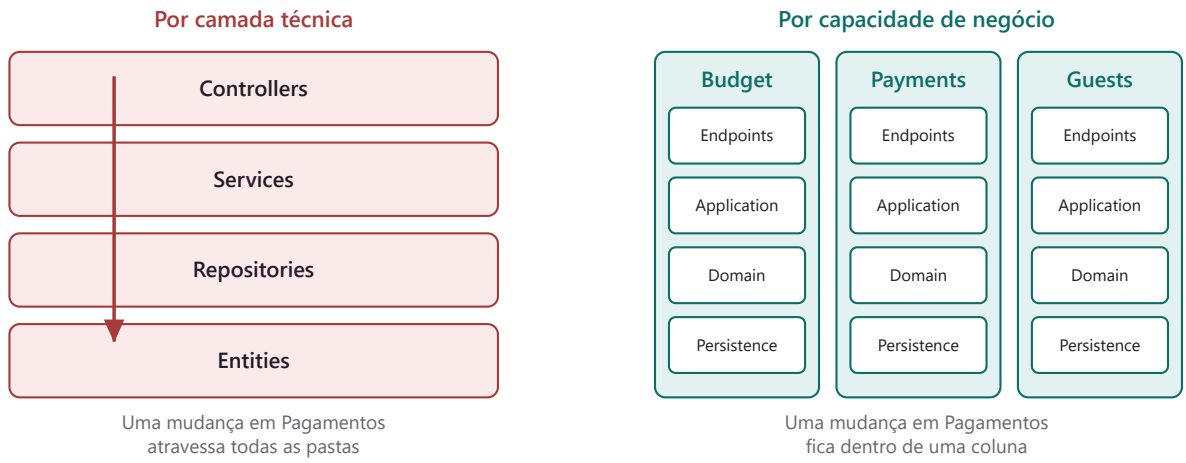



Figura 3.1. A organização por camada espalha cada assunto. A organização por capacidade deixa cada assunto em um lugar, e as camadas viram um detalhe interno.

A diferença é prática. Quase toda mudança de negócio é vertical: um novo campo em pagamentos toca API, regra e banco. Na organização por camada, isso significa mexer em muitas pastas, junto com código de outros assuntos. Na organização por capacidade, a mudança fica em uma coluna. A

Shopify fez esse mesmo movimento em sua base de código Rails: saiu de `models`, `views` e `controllers` para domínios como pedidos, envio, estoque e cobrança.

## 3.5 Regras sobre dependências

Saber quais módulos existem é metade do trabalho. A outra metade é definir como eles podem depender uns dos outros. Robert C. Martin formulou três princípios úteis.

**Princípio das dependências acíclicas (ADP).** O grafo de dependências entre módulos não deve ter ciclos. Se A depende de B e B depende de A, os dois se tornam um só para efeitos práticos: não dá para entender, testar ou mudar um sem o outro.

**Princípio das dependências estáveis (SDP).** Dependa na direção da estabilidade. O que muda muito deve depender do que muda pouco, e não o contrário.

**Princípio das abstrações estáveis (SAP).** O que é muito estável deve ser abstrato. É por isso que contratos tendem a ser interfaces.

Vamos voltar a esses princípios no capítulo 7, porque o Laçoos viola o primeiro de uma forma controlada, e essa é uma das lições mais interessantes do projeto.

## 3.6 Como impor a fronteira

Uma regra que depende só de boa vontade não dura. As linguagens oferecem níveis diferentes de imposição:

| Mecanismo | Exemplo | Força |
|---|---|---|
| Convenção | "Não acesse essa pasta" | Fraca |
| Revisão de código | Alguém precisa perceber | Fraca |
| Visibilidade da linguagem | `internal` em C#, sistema de módulos em Java | Forte |
| Separação em projetos | Sem referência, o código não compila | Forte |
| Testes de arquitetura | Falham se a regra for quebrada | Forte |
| Processo separado | Microsserviço | Muito forte, porém caro |

Em .NET, a combinação é poderosa. Um projeto separado define **quem pode referenciar**. A palavra `internal` define **o que fica visível** para quem referenciar. Com os dois, o compilador se torna guardião da arquitetura: você não precisa lembrar da regra, porque o código que a quebra simplesmente não compila.

CONSTRUINDO O LAÇOOS · PASSO 3

# O esqueleto da solução

Vamos criar o projeto. Mais adiante, cada módulo terá dois projetos: um **interior** e um **contrato**. Antes dos módulos, criamos a solução e as configurações compartilhadas.

## A solução e os projetos iniciais

Criando a solução

```
dotnet new sln -n LacoosApi --format sln
dotnet new globaljson --sdk-version 10.0.100 --roll-forward latestFeature

dotnet new classlib -n Lacoos.SharedKernel \
  -o src/BuildingBlocks/Lacoos.SharedKernel -f net10.0
dotnet new classlib -n Lacoos.Modules.Budget.Contracts \
  -o src/Modules/Budget/Lacoos.Modules.Budget.Contracts -f net10.0
dotnet new classlib -n Lacoos.Modules.Budget \
  -o src/Modules/Budget/Lacoos.Modules.Budget -f net10.0
dotnet new web -n Lacoos.Api -o src/Host/Lacoos.Api -f net10.0

dotnet sln add $(find src -name "*.csproj")
```

A opção `--format sln` é necessária porque, no .NET 10, o padrão passou a ser o novo formato `.slnx`. Os dois funcionam. Aqui mantemos o `.sln` tradicional, igual ao do repositório.

Agora ligamos os projetos. Observe a direção: o interior do módulo conhece o próprio contrato e o kernel, e o host conhece o módulo.

Ligando as referências

```
dotnet add src/Modules/Budget/Lacoos.Modules.Budget reference \
  src/BuildingBlocks/Lacoos.SharedKernel \
  src/Modules/Budget/Lacoos.Modules.Budget.Contracts

dotnet add src/Host/Lacoos.Api reference \
  src/BuildingBlocks/Lacoos.SharedKernel \
  src/Modules/Budget/Lacoos.Modules.Budget
```

## Configurações compartilhadas

Com mais de vinte projetos, repetir configurações em cada `.csproj` seria um problema. Um arquivo `Directory.Build.props` na raiz é aplicado automaticamente a todos os projetos abaixo dele:

**Configuração comum a todos os projetos** · Directory.Build.props

```
<Project>
  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <LangVersion>latest</LangVersion>
    <ManagePackageVersionsCentrally>true</ManagePackageVersionsCentrally>
    <NoWarn>$(NoWarn);CS1591</NoWarn>
  </PropertyGroup>
</Project>
```

Com `ManagePackageVersionsCentrally`, as versões dos pacotes ficam em um só arquivo. Em um monolito modular, isso tem uma razão arquitetural além da conveniência: como tudo roda em um processo, **dois módulos não podem usar versões diferentes da mesma biblioteca**. O arquivo central torna essa verdade explícita.

**Versões centralizadas (trecho)** · Directory.Packages.props · linhas 1–9

```
<Project>
  <ItemGroup>
    <PackageVersion Include="Microsoft.AspNetCore.OpenApi" Version="10.0.14" />
    <PackageVersion Include="Microsoft.EntityFrameworkCore" Version="10.0.14" />
    <PackageVersion Include="Microsoft.EntityFrameworkCore.Relational" Version="10.0.14" />
    <PackageVersion Include="Microsoft.EntityFrameworkCore.Design" Version="10.0.14" />
    <PackageVersion Include="Npgsql.EntityFrameworkCore.PostgreSQL" Version="10.0.3" />
    <PackageVersion Include="EFCore.NamingConventions" Version="10.0.1" />
    <PackageVersion Include="Microsoft.Extensions.Identity.Core" Version="10.0.14" />
```

Cada `.csproj` passa a declarar só o nome do pacote. Veja como o projeto do módulo Budget ficou pequeno. Ele é, na prática, uma lista das dependências permitidas:

src/Modules/Budget/Lacoos.Modules.Budget/Lacoos.Modules.Budget.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <ItemGroup>
    <ProjectReference
Include="..\..\..\BuildingBlocks\Lacoos.SharedKernel\Lacoos.SharedKernel.csproj" />
    <ProjectReference
Include="..\Lacoos.Modules.Budget.Contracts\Lacoos.Modules.Budget.Contracts.csproj" />
    <ProjectReference
Include="..\..\Suppliers\Lacoos.Modules.Suppliers.Contracts\Lacoos.Modules.Suppliers.Contracts.csproj
" />
    <ProjectReference
Include="..\..\Payments\Lacoos.Modules.Payments.Contracts\Lacoos.Modules.Payments.Contracts.csproj"
/>
  </ItemGroup>
</Project>
```

Esse arquivo é um documento de arquitetura. Basta lê-lo para saber que o Budget depende apenas do kernel, do próprio contrato e dos contratos de Suppliers e Payments. Nada mais.

O projeto de contrato é ainda mais revelador. Ele não tem nenhuma dependência:

src/Modules/Budget/Lacoos.Modules.Budget.Contracts/Lacoos.Modules.Budget.Contracts.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
</Project>
```

### A estrutura final

Ao final do livro, a solução terá esta forma:

Estrutura do repositório

```
LacoosApi.sln
Directory.Build.props        configuração comum
Directory.Packages.props     versões centralizadas
global.json                  versão do SDK
Dockerfile, docker-compose.yml
src/
  Host/Lacoos.Api/           host único: registra módulos e inicia
  BuildingBlocks/Lacoos.SharedKernel/
  Modules/
    Budget/
      Lacoos.Modules.Budget/            interior (internal)
      Lacoos.Modules.Budget.Contracts/  público
    Checklist/  Dashboard/  Gifts/  Guests/
    Identity/   Payments/   Suppliers/  Weddings/
tests/
  Lacoos.ArchitectureTests/  Lacoos.UnitTests/  Lacoos.IntegrationTests/
```

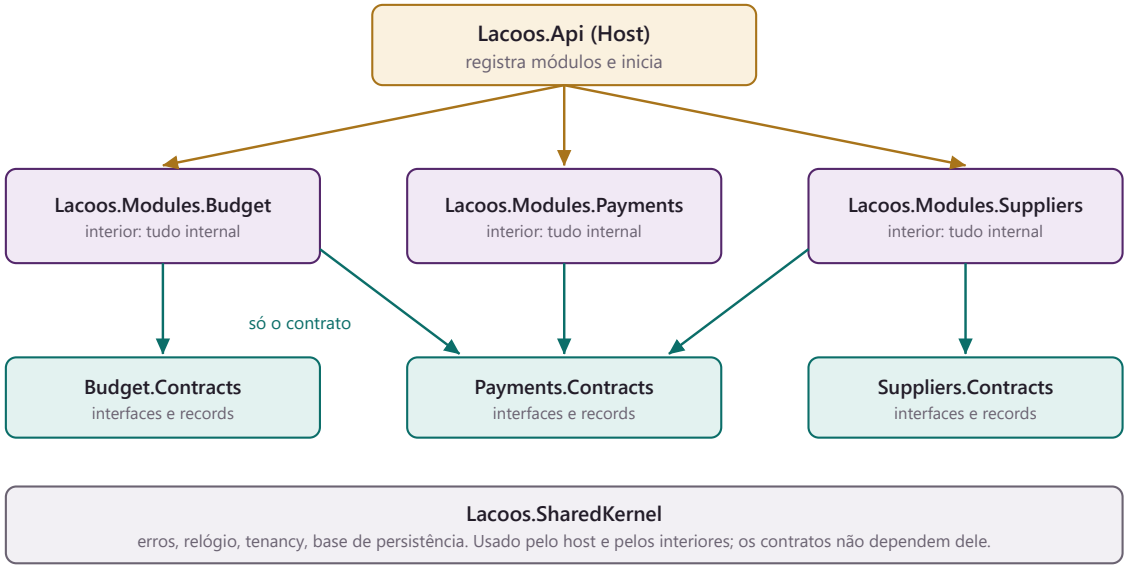



**Figura 3.2.** Como os projetos se referenciam. Os módulos enxergam apenas contratos uns dos outros e o kernel.

## 3.8 Resumo e exercícios

- Um módulo é definido pelo que esconde: decisões que podem mudar ficam atrás de uma interface estável.

- Bons módulos têm responsabilidade clara, interface explícita, interior oculto, dados próprios e dependências declaradas.
- Prefira módulos profundos: interface pequena, muita funcionalidade.
- Organize primeiro por capacidade de negócio; camadas vêm depois.
- Use o compilador para impor a fronteira: projetos separados mais `internal`.

## Exercícios

1. Na sua base de código, escolha uma pasta chamada de módulo. Quais das cinco propriedades ela tem?
2. Por que a centralização de versões de pacotes é especialmente importante em um monolito?
3. Crie os projetos do passo 3 no seu computador. Depois tente fazer um projeto usar uma classe `internal` de outro e observe o erro do compilador.

PARTE II

# Desenhando as fronteiras

A decisão mais cara de um monolito modular é onde cortar. Aqui usamos o Domain-Driven Design para encontrar as fronteiras e montamos o host que dará vida aos módulos.

CAPÍTULO 4

# Encontrando as fronteiras com DDD

*Toda a estrutura técnica deste livro serve para proteger fronteiras. Se elas estiverem no lugar errado, você terá protegido o erro.*


4.1 A decisão mais cara
4.2 Domínio e subdomínios
4.3 Linguagem ubíqua
4.4 Contexto delimitado
4.5 Mapa de contextos
4.6 Técnicas para descobrir fronteiras
4.7 O mapa de módulos do Laçoos
4.8 Resumo e exercícios


## 4.1 A decisão mais cara

Trocar uma biblioteca é barato. Reorganizar uma pasta também. Mas uma fronteira errada entre módulos gera, todos os dias, chamadas demais entre partes que deveriam ser uma só, e regras de negócio duplicadas.

O **Domain-Driven Design** (DDD), apresentado por Eric Evans em 2003, é a ferramenta mais usada para esse trabalho. Ele tem duas metades. O **DDD tático** trata de padrões de código, como entidades e agregados. O **DDD estratégico** trata de como dividir um sistema grande. É esta segunda metade que mais importa para um monolito modular, e ela não exige nenhuma linha de código.

## 4.2 Domínio e subdomínios

O **domínio** é a área de negócio que o sistema atende. No Laçoos, é o planejamento de casamentos. Domínios grandes se dividem em **subdomínios**, que têm tipos diferentes:

| Tipo | Significado | Exemplo no Laçoos | Como tratar |
|---|---|---|---|
| **Principal** (*core*) | Onde o produto se diferencia | Checklist, orçamento, pagamentos | Melhor design e mais atenção |
| **De suporte** | Necessário, específico, sem ser diferencial | Convidados, presentes | Bom o suficiente |
| **Genérico** | Problema já resolvido no mercado | Login, envio de e-mail | Use algo pronto, se puder |

Essa classificação ajuda a decidir onde investir. Não faz sentido gastar a mesma energia em um módulo de login e no cálculo de parcelas.

## 4.3 Linguagem ubíqua

Evans propõe que desenvolvedores e especialistas usem **a mesma linguagem** ao falar do negócio, e que essa linguagem apareça no código. Se o cliente diz "baixa" e o código diz `updateStatusPayment`, cada conversa exige uma tradução, e toda tradução perde informação.

O ponto mais útil é outro: **a linguagem não é global**. A mesma palavra pode significar coisas diferentes em lugares diferentes, e é exatamente aí que moram as fronteiras.

## 4.4 Contexto delimitado

Um **contexto delimitado** (*bounded context*) é a fronteira dentro da qual um modelo e sua linguagem têm um significado único e consistente.

Pense na palavra "cliente" em uma empresa. Para o comercial, é alguém com um funil e uma negociação. Para o financeiro, é um CNPJ com limite de crédito. Para o suporte, é quem abre chamados. Um modelo único com todos esses campos fica enorme e contraditório. A solução do DDD é aceitar **vários modelos**, cada um válido dentro do seu contexto, ligados por um identificador.

> **Para lembrar**
>
> Quando a mesma palavra muda de significado, ou quando dois grupos discutem sobre o que ela quer dizer, você provavelmente encontrou uma fronteira.

Em um monolito modular, a regra prática é: **um contexto delimitado vira um módulo**. Pode acontecer de um contexto grande virar mais de um módulo, mas um módulo não deve atravessar vários contextos.

## 4.5 Mapa de contextos

Definidos os contextos, é preciso descrever como eles se relacionam. O **mapa de contextos** usa alguns padrões conhecidos:

| Padrão | O que significa |
|---|---|
| **Cliente e fornecedor** | Um lado consome, o outro fornece, e existe negociação |
| **Conformista** | O consumidor aceita o modelo do fornecedor como está |
| **Camada anticorrupção (ACL)** | O consumidor traduz o modelo externo para proteger o próprio |
| **Serviço aberto + linguagem publicada** | Um contexto oferece uma API estável e bem documentada |
| **Kernel compartilhado** | Dois contextos dividem um pedaço pequeno de código |
| **Caminhos separados** | Os contextos não se integram: duplicar custa menos |

Esses nomes são úteis porque transformam uma dependência em uma decisão consciente. "Payments é cliente do contrato de Budget" diz muito mais do que "Payments chama Budget".

## 4.6 Técnicas para descobrir fronteiras

### Event Storming

Criado por Alberto Brandolini, o **Event Storming** é um workshop em que especialistas e desenvolvedores colam notas adesivas em uma parede, descrevendo os **eventos de negócio** no passado: "Pagamento registrado", "Convidado confirmou presença", "Presente reservado". Depois, ordenam por tempo e agrupam. Áreas com muita ligação interna e pouca externa sugerem contextos.

### Heurísticas úteis

Sem um workshop formal, estas perguntas ajudam bastante:

- **Quem muda isso?** Pessoas ou áreas diferentes indicam contextos diferentes.
- **O que muda junto?** Coisas que mudam na mesma entrega tendem a ficar no mesmo módulo.
- **Onde está a transação?** Dados que precisam mudar de forma atômica pedem para ficar juntos.
- **O ciclo de vida é o mesmo?** Se um objeto continua existindo depois que o outro acaba, talvez sejam contextos diferentes.
- **Quem é a fonte da verdade?** Se dois módulos gravam a mesma informação, há um erro de posse.

### Erros clássicos

> **Módulos por entidade**
>
> Criar um módulo por tabela, como `CustomerModule` e `OrderItemModule`, é o erro mais comum. Módulos viram CRUDs rasos, e toda regra de negócio passa a precisar de vários deles. Modele **capacidades** (o que o negócio faz), e não substantivos.

Outros erros frequentes:

- **Copiar o organograma sem pensar.** A estrutura das equipes influencia, mas nem sempre está certa.
- **Separar cedo demais.** Se você ainda não entende o domínio, fronteiras finas vão errar. É melhor começar com módulos maiores e dividir depois.
- **Criar módulos técnicos.** `Notifications` e `Reports` costumam acabar conhecendo tudo. Veremos uma exceção consciente com o Dashboard.

CONSTRUINDO O LAÇOOS · PASSO 4

### O mapa de módulos do Laçoos

Aplicando as heurísticas, chegamos a nove módulos. Cada linha abaixo é uma frase de responsabilidade, a primeira propriedade de um bom módulo:

| Módulo | Responsabilidade | É dono de |
|---|---|---|
| **Identity** | Saber quem é o usuário | Contas, sessões, redefinição de senha |
| **Weddings** | Saber quais casamentos existem e quem participa | Casamentos, membros, convites |
| **Checklist** | Organizar o que precisa ser feito | Tarefas e categorias |
| **Budget** | Planejar e acompanhar o que se vai gastar | Orçamento total, categorias, itens |
| **Guests** | Gerenciar quem foi convidado | Convidados, grupos, presença |
| **Suppliers** | Gerenciar com quem se contrata | Fornecedores, serviços, contratos |
| **Payments** | Controlar o que é pago e quando | Pagamentos, parcelas, baixas, comprovantes |
| **Gifts** | Gerenciar a lista de presentes | Presentes, reservas, recebimentos |
| **Dashboard** | Mostrar um resumo | Nada: só compõe dados |

## Três decisões difíceis

As decisões interessantes estão nos casos em que a resposta não é óbvia. Veja três delas.

**Budget e Payments são dois módulos ou um?** Ambos falam de dinheiro, mas têm linguagens diferentes. Orçamento fala em *planejado* e *contratado*: uma intenção. Pagamento fala em *parcela*, *baixa* e *estorno*: um fato. O ciclo de vida também difere, porque um item de orçamento existe antes de qualquer pagamento, e um pagamento pode existir sem item de orçamento. Por isso, são dois contextos.

**O que significa "pago"?** Essa pergunta revelou uma fronteira real. No Budget, "pago" soma só os pagamentos **ligados** a itens de orçamento. No Dashboard, "pago" soma **todos** os pagamentos do casamento. Os dois estão certos no próprio contexto, e o sistema assume a diferença em vez de forçar um número só.

**Quem é o dono do valor contratado?** Um fornecedor tem serviços com um valor contratado, e um item de orçamento também tem. Se os dois módulos fossem a fonte da verdade, os números divergiriam. A decisão: o valor **oficial** é o do `BudgetItem`. Cada informação precisa de um único dono.

> **Uma fronteira discutível**
>
> Budget, Payments e Suppliers conversam tanto que dependem uns dos outros em todas as direções, como veremos no capítulo 7. Isso sugere que os três formam um contexto maior, "Finanças". Mantê-los separados foi uma exigência do projeto e deixa cada um pequeno e claro, mas seria uma decisão defensável fundir os três. Fronteiras são hipóteses. Quando a evidência muda, elas devem mudar.

## O mapa de contextos real

As relações abaixo não são um desenho de intenções. Foram extraídas das referências entre projetos do repositório:

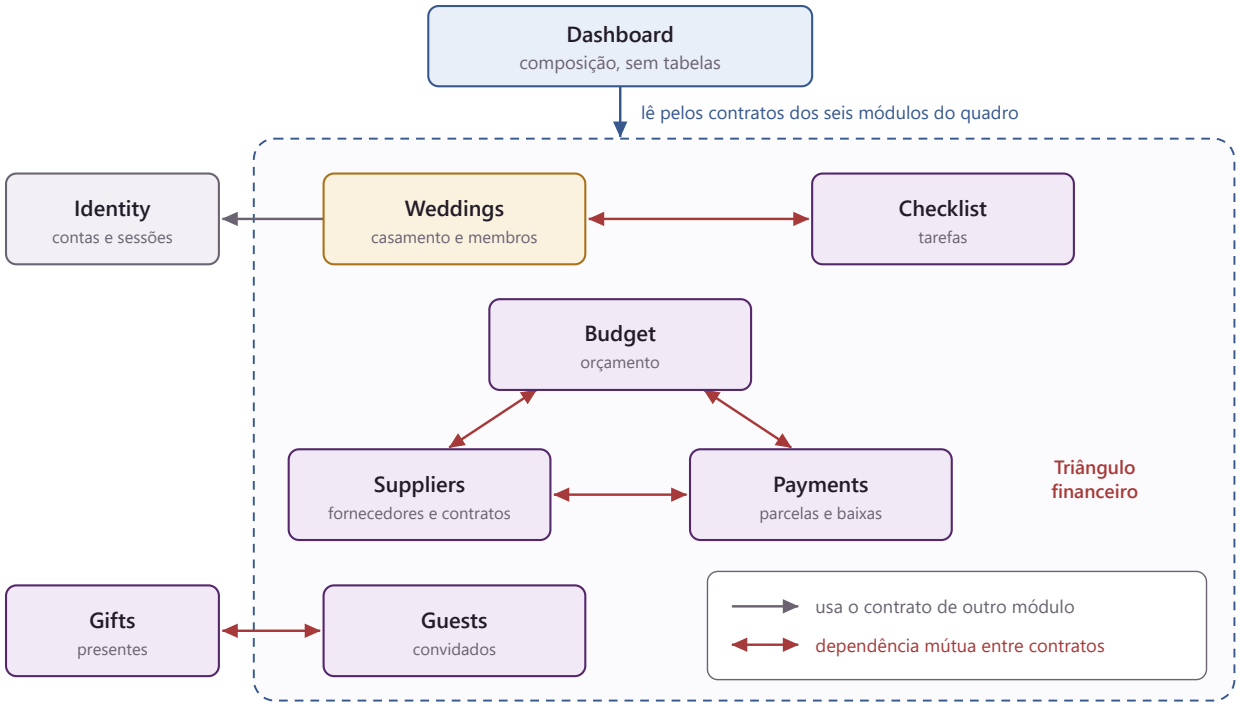



**Figura 4.1.** Dependências reais entre os módulos do Laçoos. Vermelho marca dependências mútuas entre contratos, que discutiremos no capítulo 7.

Note alguns padrões:

- **Identity não depende de ninguém.** É uma base estável, o que está de acordo com o princípio das dependências estáveis.
- **Dashboard depende de seis módulos, e ninguém depende dele.** Fica no topo do grafo, exatamente onde um módulo de composição deve estar.
- **O grupo financeiro é o mais denso.** Budget, Payments e Suppliers formam um triângulo.

### A regra sobre o que fica de fora

Delimitar também é dizer "não". O escopo do Laçoos exclui explicitamente site público, confirmação de presença por link, compra de presentes, integração com lojas e gateways de pagamento. Isso impede que os módulos cresçam na direção errada. Um módulo `Gifts` que controla uma lista administrativa é bem diferente de um que processa compras.

## 4.8 Resumo e exercícios

- Fronteiras erradas são o erro mais caro: nenhuma técnica compensa um corte ruim.
- O DDD estratégico separa subdomínios, define linguagens e delimita contextos.
- Em um monolito modular, um contexto delimitado costuma virar um módulo.
- Palavras que mudam de sentido revelam fronteiras.
- Modele capacidades de negócio, não tabelas.

## Exercícios

1. Escolha uma palavra importante no seu sistema, como "cliente" ou "pedido". Ela significa a mesma coisa em todas as áreas?
2. No Laçoos, por que "pago" pode ter dois valores corretos? Em que situação esses valores seriam diferentes?
3. Defenda a fusão de Budget, Payments e Suppliers em um módulo `Finances`. Depois defenda a separação. Qual argumento pesa mais para você?

CAPÍTULO 5

# O host: onde os módulos se encontram

*Vários módulos, um processo. Alguém precisa juntar tudo, e esse alguém deve saber o mínimo possível.*


5.1 O papel do host
5.2 Composition root
5.3 A porta de entrada do módulo
5.4 O ciclo de vida da aplicação
**5.5 A interface IModule e o Program.cs**
5.6 Resumo e exercícios


## 5.1 O papel do host

Em um monolito modular, existe um projeto executável que carrega os módulos e inicia a aplicação. Chamamos esse projeto de **host**. Ele é o único lugar que conhece todos os módulos, mas conhece apenas a porta de entrada de cada um.

Responsabilidades do host:

- Iniciar o servidor web.
- Registrar cada módulo.
- Configurar o que é realmente global: logs, tratamento de erros, CORS, serialização.
- Definir a ordem do pipeline de requisições.

O que ele **não** deve ter: regras de negócio, entidades, consultas ou endpoints de domínio. Um host que cresce é um sinal de que a lógica está vazando para o lugar errado.

> **Analogia**
> O host é como a recepção de um prédio comercial. Ela sabe em que andar fica cada empresa e controla a entrada. Mas não conhece os processos internos de nenhuma delas, nem atende os clientes de cada uma.

## 5.2 Composition root

Esse conceito tem nome. Mark Seemann chama de **composition root** o único ponto da aplicação em que as dependências são ligadas. Nos módulos, o código pede o que precisa por meio de abstrações. Só no início do programa alguém decide qual implementação atende cada abstração.

Isso é o que torna possível que `Payments` use `IBudgetQueries` sem conhecer a classe que a implementa. A resolução acontece na injeção de dependências, que em .NET é o `IServiceCollection`.

Em um monolito modular, o composition root é **distribuído de forma controlada**: cada módulo registra as próprias implementações, e o host chama esses registros. O host não precisa saber quais classes existem dentro de cada módulo.

## 5.3 A porta de entrada do módulo

Para isso funcionar, os módulos precisam seguir um contrato com o host. O padrão de mercado é uma interface pequena, implementada por uma classe pública em cada módulo. O host conversa com todos da mesma forma.

Três estilos são comuns para registrar os módulos:

| Estilo | Como funciona | Vantagem | Risco |
|---|---|---|---|
| **Lista explícita** | O host declara cada módulo no código | Simples, ordem visível, sem mágica | Uma linha a mais por módulo |
| **Descoberta por reflexão** | O host varre as DLLs procurando a interface | Adicionar um módulo não muda o host | Ordem e falhas menos claras |
| **Plugins** | Módulos carregados em tempo de execução | Instalação sem recompilar | Muito mais complexo |

Para um sistema com nove módulos, a lista explícita é a melhor escolha. Ela ocupa uma dúzia de linhas e deixa claro o que está rodando. A descoberta por reflexão resolve um problema que aqui não existe.

## 5.4 O ciclo de vida da aplicação

Quando o processo inicia, as etapas seguem uma ordem. Cada módulo participa de alguns momentos bem definidos:

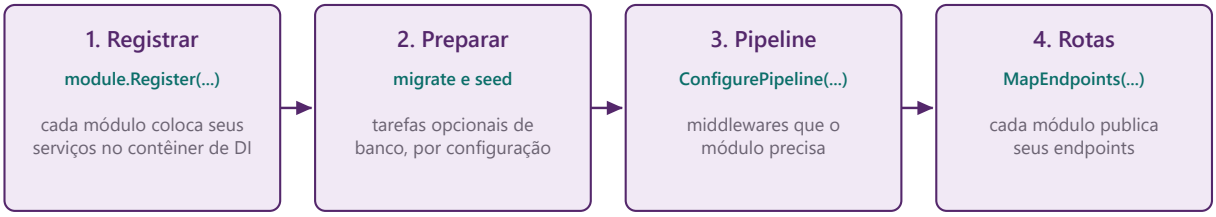



Figura 5.1. Os quatro momentos em que o host conversa com os módulos.

A ordem importa. Serviços precisam estar registrados antes de a aplicação ser construída. Middlewares precisam estar na posição certa: autenticação vem antes de autorização, e o tratamento de erros vem antes de tudo o que pode falhar.

### Configuração por módulo

Cada módulo pode ter a própria seção de configuração, como `Auth` para o Identity. O host carrega as fontes, como `appsettings.json` e variáveis de ambiente, e cada módulo lê apenas a sua parte. Esse arranjo evita que uma configuração vire um acoplamento escondido.

CONSTRUINDO O LAÇOOS · PASSO 5

## A interface IModule e o Program.cs

Vamos criar a porta de entrada. Ela fica no kernel, porque tanto o host quanto os módulos precisam conhecê-la.

**A porta de entrada de um módulo** · src/BuildingBlocks/Lacoos.SharedKernel/Modules/IModule.cs · linhas 8–19

```
/// <summary>Único ponto público de um módulo para o host.</summary>
public interface IModule
{
    void Register(IServiceCollection services, IConfiguration configuration);

    void MapEndpoints(IEndpointRouteBuilder app);

    /// <summary>Middlewares que o módulo precisa, executados após a autenticação.</summary>
    void ConfigurePipeline(IApplicationBuilder app)
    {
    }
}
```

Três métodos, cada um correspondendo a um momento da Figura 5.1. `Register` e `MapEndpoints` são obrigatórios. `ConfigurePipeline` tem uma implementação vazia por padrão, porque a maioria dos módulos não precisa de middleware. Só o Identity usa, para proteger contra CSRF. Esse recurso de C#, *default interface method*, evita que oito módulos declarem um método vazio.

### A primeira implementação

Toda a ligação do módulo Budget com o resto do mundo cabe em poucas linhas:

**A classe pública do módulo** · src/Modules/Budget/Lacoos.Modules.Budget/BudgetModule.cs · linhas 12–23

```
public sealed class BudgetModule : IModule
{
    public void Register(IServiceCollection services, IConfiguration configuration)
    {
        services.AddModuleDbContext<BudgetDbContext>(configuration, BudgetDbContext.SchemaName);
        services.AddScoped<BudgetService>();
        services.AddScoped<IBudgetQueries, BudgetQueries>();
        services.AddScoped<IModuleSeeder, BudgetSeeder>();
    }

    public void MapEndpoints(IEndpointRouteBuilder app) => BudgetEndpoints.Map(app);
}
```

Observe o que está acontecendo aqui:

- `BudgetModule` é **a única classe pública** do projeto.
- `BudgetService` e `BudgetQueries` são `internal`. O host nem consegue vê-las.
- A linha `AddScoped<IBudgetQueries, BudgetQueries>()` é onde o contrato público ganha sua implementação oculta.

O detalhe interessante é que o método `AddScoped` está **dentro do módulo**. Por isso uma classe `internal` pode ser registrada: quem conhece a classe é o próprio módulo.

## O Program.cs

Agora o host. A lista de módulos fica no início do arquivo, à vista de todos:

**Os módulos do sistema** · src/Host/Lacoos.Api/Program.cs · linhas 25–36

```
IModule[] modules =
[
    new IdentityModule(),
    new WeddingsModule(),
    new DashboardModule(),
    new ChecklistModule(),
    new BudgetModule(),
    new GuestsModule(),
    new SuppliersModule(),
    new PaymentsModule(),
    new GiftsModule(),
];
```

Em seguida, cada módulo se registra:

**Momento 1: registro** · src/Host/Lacoos.Api/Program.cs · linhas 73–76

```
foreach (var module in modules)
{
    module.Register(builder.Services, builder.Configuration);
}
```

Depois de construir a aplicação, vêm as tarefas de preparação. O argumento `migrate` permite usar a mesma imagem para aplicar migrations e encerrar, o que será útil no capítulo 14:

**Momento 2: preparação** · src/Host/Lacoos.Api/Program.cs · linhas 80–94

```
if (args.Contains("migrate"))
{
    await StartupTasks.MigrateAsync(app.Services, CancellationToken.None);
    return;
}

if (app.Configuration.GetValue("Database:MigrateOnStartup", false))
{
    await StartupTasks.MigrateAsync(app.Services, CancellationToken.None);
}

if (app.Configuration.GetValue("Seed:Enabled", false))
{
    await StartupTasks.SeedAsync(app.Services, app.Configuration, CancellationToken.None);
}
```

Então vem o pipeline. A ordem é uma decisão de segurança. Note que os middlewares dos módulos entram depois da autenticação, porque o CSRF do Identity precisa saber quem é o usuário:

**Momento 3: pipeline** · src/Host/Lacoos.Api/Program.cs · linhas 96–111

```
app.Use((http, next) =>
{
    // Impede o navegador de reinterpretar arquivos enviados como outro tipo.
    http.Response.Headers.XContentTypeOptions = "nosniff";
    return next(http);
});
app.UseExceptionHandler();
app.UseCors(CorsPolicy);
app.UseAuthentication();
foreach (var module in modules)
{
    module.ConfigurePipeline(app);
}

app.UseRateLimiter();
app.UseAuthorization();
```

Por fim, as rotas:

**Momento 4: rotas e início** · src/Host/Lacoos.Api/Program.cs · linhas 113–126

```
if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
    app.UseLacoosSwaggerUi();
}

app.MapHealthChecks("/health");

foreach (var module in modules)
{
    module.MapEndpoints(app);
}

await app.RunAsync();
```

Repare no bloco condicional: o Swagger e o documento OpenAPI só existem em desenvolvimento. O teste `Swagger_ui_is_not_exposed_outside_development` garante isso.

## O que o host conhece

Veja o início do `Lacoos.Api.csproj`. O host referencia todos os módulos, mas um teste de arquitetura impede que ele use algo além das portas de entrada:

**Referências do host** · src/Host/Lacoos.Api/Lacoos.Api.csproj · linhas 13–24

```
<ItemGroup>
  <ProjectReference Include="..\..\BuildingBlocks\Lacoos.SharedKernel\Lacoos.SharedKernel.csproj" />
  <ProjectReference
Include="..\..\Modules\Identity\Lacoos.Modules.Identity\Lacoos.Modules.Identity.csproj" />
  <ProjectReference
Include="..\..\Modules\Weddings\Lacoos.Modules.Weddings\Lacoos.Modules.Weddings.csproj" />
  <ProjectReference
Include="..\..\Modules\Dashboard\Lacoos.Modules.Dashboard\Lacoos.Modules.Dashboard.csproj" />
  <ProjectReference
Include="..\..\Modules\Checklist\Lacoos.Modules.Checklist\Lacoos.Modules.Checklist.csproj" />
  <ProjectReference Include="..\..\Modules\Budget\Lacoos.Modules.Budget\Lacoos.Modules.Budget.csproj"
/>
  <ProjectReference Include="..\..\Modules\Guests\Lacoos.Modules.Guests\Lacoos.Modules.Guests.csproj"
/>
  <ProjectReference
Include="..\..\Modules\Suppliers\Lacoos.Modules.Suppliers\Lacoos.Modules.Suppliers.csproj" />
  <ProjectReference
Include="..\..\Modules\Payments\Lacoos.Modules.Payments\Lacoos.Modules.Payments.csproj" />
  <ProjectReference Include="..\..\Modules\Gifts\Lacoos.Modules.Gifts\Lacoos.Modules.Gifts.csproj" />
</ItemGroup>
```

O projeto `.Contracts` não aparece aqui porque o host não precisa dele. Quem referencia contratos são os módulos.

> **O host ainda faz algumas escolhas de infraestrutura**
>
> O host também escolhe implementações como `SmtpEmailSender` e `LocalFileStorage`, porque são detalhes da aplicação inteira, e não de um módulo. Isso é correto, mas é preciso vigiar. O `Program.cs` tem 128 linhas, um tamanho saudável. Se ele chegar a 500, algo terá saído do lugar.

### O sistema já roda

Com o `IModule`, o `Program.cs` e um módulo vazio, a aplicação já inicia. Acrescentar um módulo novo significa criar dois projetos, implementar `IModule` e adicionar uma linha à lista. Nenhum outro módulo muda.

---

## 5.6 Resumo e exercícios

- O host é o único projeto que conhece todos os módulos, e só pelas portas de entrada.
- O composition root liga abstrações a implementações. Em um monolito modular, cada módulo registra o que é seu.
- Lista explícita de módulos é o caminho mais simples para sistemas de tamanho normal.
- A ordem do pipeline é uma decisão de segurança.
- Um host grande é sinal de lógica no lugar errado.

## Exercícios

1. Por que uma classe `internal` pode ser registrada no contêiner de injeção de dependências, mesmo que o host não a enxergue?
2. Se o middleware de CSRF estivesse antes de `UseAuthentication`, o que aconteceria? Olhe o código do `CsrfMiddleware` no repositório antes de responder.
3. Quais seriam as vantagens e desvantagens de trocar a lista explícita por descoberta via reflexão no Laçoos?

PARTE III

# Construindo os módulos

O interior de um módulo, as conversas entre módulos, a posse dos dados, a consistência e o que é compartilhado. É nesta parte que o Laçoos ganha corpo.

CAPÍTULO 6

# Anatomia interna de um módulo

*Por fora, todos os módulos são iguais. Por dentro, cada um pode ter a estrutura que o seu problema pede.*


6.1 Uma decisão local
6.2 Arquitetura em camadas
6.3 Arquitetura hexagonal e Clean Architecture
6.4 Vertical Slice
6.5 Onde mora a regra de negócio
6.6 Modelo anêmico: quando é problema?
6.7 Montando o interior do Budget
6.8 Resumo e exercícios


## 6.1 Uma decisão local

Um benefício pouco comentado do monolito modular é que **a arquitetura interna de um módulo é uma decisão local**. O resto do sistema só enxerga o contrato. Um módulo com regras ricas pode ter um modelo de domínio sofisticado. Um módulo de cadastro simples pode ser só um CRUD direto. Os dois convivem.

Isso liberta o time de um erro frequente: aplicar a mesma cerimônia a tudo. Quatro camadas e cinco classes para gravar um nome em uma tabela não deixam o sistema mais limpo, só mais longo.

Vejamos as opções que o mercado usa para organizar o interior.

## 6.2 Arquitetura em camadas

É o estilo mais tradicional. O código se divide em camadas horizontais, e cada uma só conhece a de baixo: apresentação, aplicação, domínio, dados.

- **Vantagem:** é simples, conhecida e fácil de explicar.
- **Risco:** na forma clássica, o domínio depende do banco, o que amarra as regras de negócio à tecnologia.

## 6.3 Arquitetura hexagonal e Clean Architecture

Alistair Cockburn propôs, em 2005, a **arquitetura hexagonal**, ou *ports and adapters*. A inversão é a ideia central: o domínio fica no centro e **não depende de nada**. Ele define **portas** (interfaces) do que precisa, e **adaptadores** fazem a ligação com o mundo externo.

A **Clean Architecture**, de Robert C. Martin, segue o mesmo princípio em anéis concêntricos: as dependências sempre apontam para dentro.

- **Vantagem:** regras testáveis sem banco, e tecnologia trocável.
- **Custo:** mais interfaces, mais mapeamentos, mais arquivos. Em um domínio simples, é cerimônia sem retorno.

## 6.4 Vertical Slice

Jimmy Bogard popularizou a **Vertical Slice Architecture**. Em vez de camadas, o código é organizado por funcionalidade: uma pasta `CriarPagamento/` contém a requisição, o handler e a resposta. Cada fatia decide como acessar dados.

- **Vantagem:** adicionar uma funcionalidade significa adicionar arquivos, e não editar vários.
- **Risco:** regras compartilhadas entre fatias podem acabar duplicadas.

Vale notar que vertical slice e monolito modular são conceitos diferentes. O primeiro organiza o **interior**. O segundo organiza o **sistema**. Eles combinam bem.

## 6.5 Onde mora a regra de negócio

Independentemente da estrutura, é preciso decidir onde as regras ficam. Martin Fowler, em *Patterns of Enterprise Application Architecture*, descreve dois extremos.

**Transaction Script.** Cada caso de uso é um procedimento que faz tudo em sequência: busca, valida, calcula, grava. É direto e funciona muito bem em lógica simples. Conforme as regras crescem, surge duplicação.

**Domain Model.** As regras moram nos objetos de domínio. Uma entidade protege seus próprios invariantes, e ninguém consegue colocá-la em um estado inválido. Exige mais design, mas escala melhor em regras complexas.

Na prática, há um meio-termo muito eficaz: **regras puras em funções sem dependências**. Elas recebem valores, devolvem valores e não acessam banco nem relógio. São simples de escrever e muito fáceis de testar.

> **Para lembrar**
>
> A pergunta útil não é "qual arquitetura é a certa?", mas "qual é a complexidade deste módulo?". Comece simples e adicione estrutura quando a dor aparecer.

## 6.6 Modelo anêmico: quando é problema?

Chama-se **modelo anêmico** a entidade que só tem dados, com propriedades públicas, enquanto a lógica fica em serviços. Fowler o considera um antipadrão quando se tenta fazer DDD, porque ele perde o principal benefício: proteger os invariantes.

Mas o contexto importa. Em um módulo simples, entidades com dados e um serviço claro são uma escolha razoável. O problema é quando a mesma regra aparece em cinco serviços, ou quando qualquer código consegue deixar uma entidade inválida.

CONSTRUINDO O LAÇOOS · PASSO 6

# Montando o interior do Budget

O Laçoos usa uma estrutura simples e igual em todos os módulos, com quatro pastas:

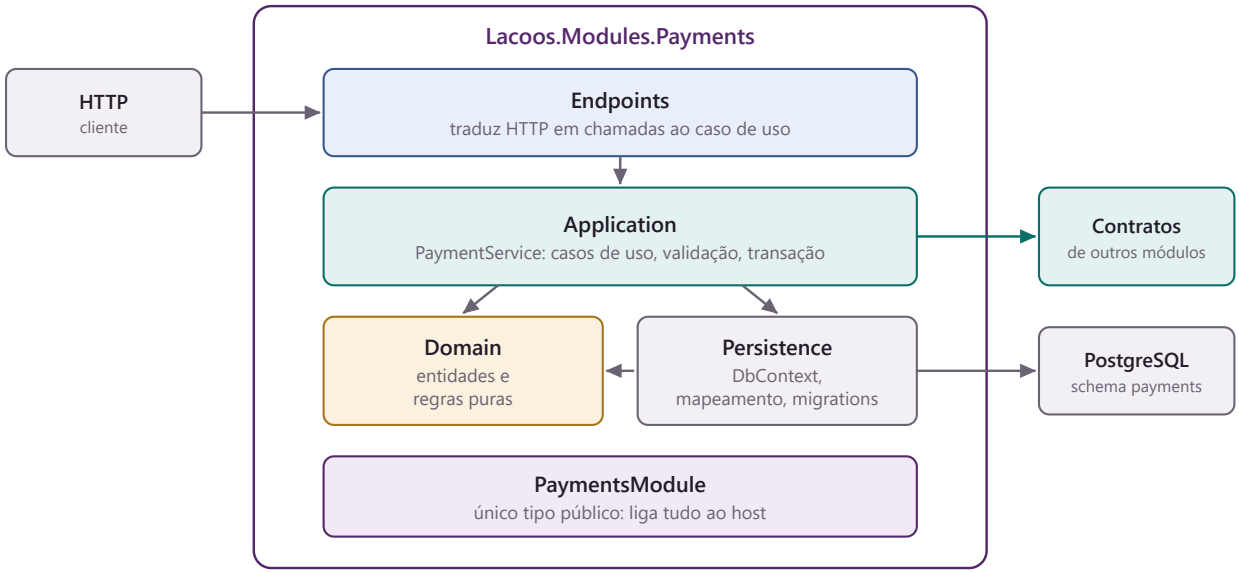



**Figura 6.1.** O interior do módulo Payments. Todas as caixas são internas: só o PaymentsModule é público.

| Pasta | Responsabilidade | Depende de |
|---|---|---|
| `Domain` | Entidades, enums e regras puras | Quase nada |
| `Application` | Casos de uso, validação, transação | Domain, Persistence, contratos |
| `Persistence` | `DbContext`, mapeamentos, migrations | Domain, EF Core |
| `Endpoints` | Rotas HTTP | Application |

Essa divisão fica mais perto de camadas simples do que de uma arquitetura hexagonal completa. A camada `Application` usa o `DbContext` diretamente, sem repositórios. O motivo é pragmático: o `DbContext` já é uma implementação dos padrões Repository e Unit of Work, e, como ele não sai do módulo, uma interface a mais só acrescentaria código.

## O domínio

Começamos pelo que é mais importante. Todas as classes são `internal`:

**Entidades do Budget** · src/Modules/Budget/Lacoos.Modules.Budget/Domain/BudgetItem.cs · linhas 31–62

```
internal sealed class BudgetItem : IWeddingScoped
{
    public Guid Id { get; set; } = Guid.CreateVersion7();

    public Guid WeddingId { get; set; }

    public Guid CategoryId { get; set; }

    public BudgetCategory? Category { get; set; }
```

```
    public Guid? SupplierId { get; set; }

    public required string Name { get; set; }

    public string? Description { get; set; }

    public decimal PlannedAmount { get; set; }

    public decimal ContractedAmount { get; set; }

    public BudgetItemStatus Status { get; set; } = BudgetItemStatus.Planned;

    /// <summary>Itens cancelados não contam como valor comprometido.</summary>
    public decimal CommittedAmount => Status == BudgetItemStatus.Cancelled ? 0 : ContractedAmount;
}

internal static class BudgetMath
{
    /// <summary>Sem valor definido, o orçamento total é a soma das categorias.</summary>
    public static decimal TotalBudget(decimal? definedTotal, decimal categoriesPlanned) =>
        definedTotal ?? categoriesPlanned;
}
```

Alguns detalhes merecem atenção:

- `Guid.CreateVersion7()` gera identificadores ordenados pelo tempo, que têm melhor comportamento em índices do que GUIDs aleatórios.
- `SupplierId` é só um `Guid?`. Não há nenhuma propriedade `Supplier`. Essa referência por identificador é assunto do capítulo 8.
- `CommittedAmount` é uma regra de negócio em uma linha: itens cancelados não comprometem o orçamento. Ela é **calculada**, não gravada.
- `BudgetMath` guarda uma regra pura em uma função estática.

### Regras puras em Payments

O módulo Payments mostra o valor de separar regras puras. Dividir R$ 100,00 em três parcelas parece trivial, até você lembrar que 33,33 × 3 = 99,99. A regra precisa garantir que a soma das parcelas seja exatamente o total:

**Regras puras de parcelas** · src/Modules/Payments/Lacoos.Modules.Payments/Domain/PaymentRules.cs · linhas 9–26

```
/// <summary>
/// Divide igualmente, truncando em 2 casas, e deixa o resto na última parcela.
/// Truncar garante que a última nunca fica menor que as demais.
/// </summary>
public static IReadOnlyList<decimal> SplitAmount(decimal total, int count)
{
    var baseAmount = Math.Truncate(total * 100 / count) / 100;
    var amounts = Enumerable.Repeat(baseAmount, count).ToArray();
    amounts[^1] = total - (baseAmount * (count - 1));
    return amounts;
}

/// <summary>
/// Mensal sempre a partir do primeiro vencimento: 31/01 gera 28/02 e depois 31/03.
/// Somar ao mês anterior perderia o dia original.
/// </summary>
public static IReadOnlyList<DateOnly> MonthlyDueDates(DateOnly first, int count) =>
    Enumerable.Range(0, count).Select(first.AddMonths).ToList();
```

O comentário explica o **porquê**, a parte que o código sozinho não conta. E o método `MonthlyDueDates` resolve uma armadilha real: somar um mês a 31/01 dá 28/02. Se você somar outro mês a partir daí, obtém 28/03, perdendo o dia 31. Calcular sempre a partir da primeira data evita o erro.

Como a função não depende de nada, o teste é simples e cobre muitos casos:

**Teste da divisão de parcelas** · tests/Lacoos.UnitTests/PaymentRulesTests.cs · linhas 10–23

```
[Theory]
[InlineData("100.00", 3, "33.33", "33.34")]
[InlineData("200.00", 3, "66.66", "66.68")]
[InlineData("0.10", 3, "0.03", "0.04")]
[InlineData("1000.00", 4, "250.00", "250.00")]
[InlineData("99.99", 7, "14.28", "14.31")]
public void Split_truncates_and_leaves_the_remainder_in_the_last(string total, int count, string
regular, string last)
{
    var amounts = PaymentRules.SplitAmount(decimal.Parse(total), count);

    Assert.Equal(count, amounts.Count);
    Assert.All(amounts.SkipLast(1), amount => Assert.Equal(decimal.Parse(regular), amount));
    Assert.Equal(decimal.Parse(last), amounts[^1]);
}
```

Outra decisão do domínio: o valor pago de uma parcela nunca é gravado. Ele é **calculado a partir do histórico** de baixas e estornos:

**Pago é derivado do histórico** · src/Modules/Payments/Lacoos.Modules.Payments/Domain/Payment.cs · linhas 72–76

```
/// <summary>Calculado do histórico; nunca é salvo, para não ficar desatualizado.</summary>
public decimal PaidAmount =>
    Transactions.Sum(t => t.Type == TransactionType.Settlement ? t.Amount : -t.Amount);

public decimal RemainingAmount => PaymentRules.Remaining(Amount, PaidAmount);
```

Um estorno não apaga uma baixa, ele adiciona uma nova transação. O histórico fica imutável, como em um livro contábil. E, como o total é sempre recalculado, ele não tem como ficar desatualizado.

## A camada de aplicação

O `BudgetService` contém os casos de uso. Veja a criação de um item:

**Um caso de uso** · src/Modules/Budget/Lacoos.Modules.Budget/Application/BudgetService.cs · linhas 182–190

```
public async Task<ItemResponse> CreateItemAsync(SaveItemRequest request, CancellationToken ct)
{
    await ValidateItemAsync(request, ct);
    var item = new BudgetItem { Name = request.Name!.Trim() };
    Apply(item, request);
    db.Items.Add(item);
    await db.SaveChangesAsync(ct);
    return await GetItemAsync(item.Id, ct);
}
```

E a validação que ele chama:

**Validação em duas etapas** · src/Modules/Budget/Lacoos.Modules.Budget/Application/BudgetService.cs · linhas 220–239

```
private async Task ValidateItemAsync(SaveItemRequest request, CancellationToken ct)
{
    new Validator()
        .Required(request.Name, "name", "Nome")
        .MaxLength(request.Name, 200, "name", "Nome")
        .Required(request.CategoryId, "categoryId", "Categoria")
        .Money(request.PlannedAmount, "plannedAmount", "Valor planejado")
        .Money(request.ContractedAmount, "contractedAmount", "Valor contratado")
        .ThrowIfInvalid();

    if (!await db.Categories.AnyAsync(c => c.Id == request.CategoryId, ct))
    {
        throw new BusinessRuleException("Categoria não encontrada neste casamento.");
    }

    if (request.SupplierId is { } supplierId && !await suppliers.ExistsAsync(supplierId, ct))
    {
        throw new BusinessRuleException("Fornecedor não encontrado neste casamento.");
    }
}
```

A validação tem duas etapas com significados diferentes. Primeiro, o formato dos campos: um erro aqui vira **HTTP 400**, com a lista de todos os campos inválidos. Depois, as regras que dependem do estado: um erro aqui vira **HTTP 422**. Já a chamada `suppliers.ExistsAsync` é o primeiro encontro com outro módulo, e vamos estudá-la no próximo capítulo.

## Os endpoints

Os endpoints usam Minimal APIs e são finos de propósito. Eles só traduzem HTTP em uma chamada de serviço:

**Endpoints do Budget (início)** · src/Modules/Budget/Lacoos.Modules.Budget/Endpoints/BudgetEndpoints.cs · linhas 10–19

```
public static void Map(IEndpointRouteBuilder app)
{
    var budget = app.MapWeddingGroup("Orçamento").MapGroup("/budget");

    budget.MapGet("", (BudgetService service, CancellationToken ct) => service.GetOverviewAsync(ct));

    budget.MapPut("", (SetTotalBudgetRequest request, BudgetService service, CancellationToken ct) =>
        service.SetTotalAsync(request, ct));

    budget.MapGet("/by-category", (BudgetService service, CancellationToken ct) =>
service.GetByCategoryAsync(ct));
```

A linha mais importante é a que cria o grupo: `MapWeddingGroup`. Ela aplica autenticação, papéis e isolamento a todas as rotas do grupo. Nenhum endpoint do Budget repete essa lógica, o que veremos no capítulo 10.

## Um arquivo de usings por módulo

O arquivo `Usings.cs` lista os `using` globais do módulo. Ele tem um efeito colateral interessante: mostra de relance do que o módulo depende.

src/Modules/Budget/Lacoos.Modules.Budget/Usings.cs

```
global using Lacoos.Modules.Budget.Contracts;
global using Lacoos.Modules.Payments.Contracts;
global using Lacoos.Modules.Suppliers.Contracts;
global using Lacoos.SharedKernel.Errors;
global using Lacoos.SharedKernel.Tenancy;
global using Lacoos.SharedKernel.Validation;
global using Microsoft.EntityFrameworkCore;
```

> **Entidades com setters públicos**
>
> As entidades do Laçoos têm propriedades com `set` público, e parte das regras vive nos serviços. É um modelo próximo do anêmico, combinado com regras puras. Funciona porque tudo é `internal`, então só o próprio módulo consegue mexer nelas. Mas não impede que o `PaymentService` deixe um pagamento em um estado inválido. Em um domínio mais complexo, métodos como `payment.Settle(amount)`, com setters privados, protegeriam melhor os invariantes.

> **Serviços que crescem**
>
> O `PaymentService` tem 534 linhas, e o `GiftService`, 493. Eles ainda são legíveis, mas estão no limite. Uma evolução natural seria dividir esses serviços por caso de uso, no estilo vertical slice. Como tudo é interno, a mudança não afetaria nenhum outro módulo. Esse é o benefício da decisão local.

## 6.8 Resumo e exercícios

- A arquitetura interna é uma decisão local: cada módulo pode ter a sua.
- Camadas, hexagonal e vertical slice são opções para o interior, não para o sistema.
- Transaction script serve para lógica simples, domain model para lógica rica. Funções puras são um meio-termo eficaz.
- Valores derivados não devem ser gravados quando podem ser calculados com segurança.
- Comece simples, porque o contrato protege o resto do sistema de uma mudança interna.

### Exercícios

1. Por que `SplitAmount` usa truncamento e deixa o resto na última parcela, em vez de arredondar? Calcule R$ 200,00 em 3 parcelas nos dois métodos.
2. Reescreva mentalmente a baixa de uma parcela como um método `installment.Settle(...)`. O que mudaria nos testes?
3. Que vantagem há em manter a validação de formato separada da validação de estado?

CAPÍTULO 7

# Contratos e comunicação entre módulos

*Um contrato é uma promessa. Quanto menor a promessa, mais fácil é cumpri-la por muito tempo.*


7.1 O que é a API pública de um módulo
7.2 Três formas de comunicação
7.3 Tipos do contrato
7.4 O projeto de contratos
7.5 Dependências circulares
7.6 Evitando o N+1 entre módulos
7.7 Tradução na fronteira
7.8 Evolução de contratos
7.9 Criando os contratos
7.10 Resumo e exercícios


## 7.1 O que é a API pública de um módulo

Até aqui falamos de esconder. Só que módulos precisam conversar. A **API pública** de um módulo é tudo aquilo de que os outros podem depender. Ela precisa ser pequena, estável e pensada para quem consome.

Essa API tem três componentes típicos:

- **Interfaces:** o que pode ser pedido ao módulo.
- **Tipos de dados:** o que entra e o que sai, em geral records simples.
- **Eventos**, quando existem: o que o módulo anuncia ao mundo. São o tema do capítulo 9.

## 7.2 Três formas de comunicação

| Estilo | Como funciona | Acoplamento | Consistência |
|---|---|---|---|
| **Chamada síncrona em memória** | Um método de interface | Temporal: o chamador espera | Imediata |
| **Mediador** | Mensagem enviada por um despachante | Reduz a referência direta | Imediata |
| **Eventos assíncronos** | O módulo publica, outros reagem | O mais baixo | Eventual |

A chamada síncrona é a mais simples e a primeira opção em um monolito modular. Sua maior vantagem é o que parece um defeito: o compilador valida tudo, e você segue a execução com um depurador.

### Um aviso sobre mediadores

Muitos exemplos de .NET usam a biblioteca MediatR. Um `Send(new GetBudgetQuery())` esconde quem atende o pedido. Parece desacoplado, mas o acoplamento continua: o chamador ainda precisa conhecer o tipo da mensagem e confiar que alguém responde, e o compilador perde a capacidade de mostrar quem chama quem.

Um mediador tem bom uso em comportamentos transversais, como logs e validação. Ele não substitui um contrato bem definido. Além disso, é importante lembrar que o MediatR passou a ter licença comercial a partir da versão 13, em 2025. O Laçoos usa interfaces simples, e não precisa disso.

## 7.3 Tipos do contrato

### O que retornar

Um contrato nunca deve expor uma entidade interna. Se `Payments` devolvesse sua entidade `Payment`, todos os consumidores ficariam presos ao formato interno, inclusive às propriedades de navegação do EF Core. Em vez disso, o contrato define **tipos próprios**, em geral imutáveis, com o mínimo de dados necessário.

Isso vale para tipos da infraestrutura também. Um contrato que expõe `IQueryable` entrega aos consumidores a capacidade de montar consultas nas suas tabelas, e destrói a fronteira de uma vez.

### Consultas e comandos

Bertrand Meyer propôs o princípio **CQS**: um método ou consulta um estado, ou altera, mas não os dois. Em contratos, essa separação ajuda muito. Consultas são seguras para chamar em qualquer momento. Comandos têm efeitos e merecem mais atenção.

> Para lembrar
>
> Prefira contratos **orientados ao uso**, e não à estrutura. `HasPaymentsForSupplierAsync(id)` é melhor do que `GetAllPayments()`, porque revela uma necessidade e esconde todo o resto.

## 7.4 O projeto de contratos

Para o compilador proteger a fronteira, o contrato precisa estar fisicamente separado do interior. Em .NET, há dois arranjos comuns:

1. **Tipos públicos dentro do projeto do módulo.** Simples, mas o projeto inteiro precisa ser referenciado, e pode haver ciclos.
2. **Um projeto separado para contratos.** Um pouco mais de arquivos, mas você só referencia o contrato.

A segunda opção tem uma vantagem decisiva. Sistemas de build como o do .NET **não permitem referência circular entre projetos**. Se Budget precisa de Payments e Payments precisa de Budget, com projetos únicos isso nem compila. Com projetos de contrato, cada interior depende apenas do contrato do outro, e o grafo de projetos continua acíclico.

## 7.5 Dependências circulares

O compilador permite o arranjo acima, mas o ciclo lógico continua existindo. Quando dois módulos dependem um do outro, o que fazer? O mercado usa algumas estratégias:

| Estratégia | Quando usar |
|---|---|
| **Inverter a dependência** | Um dos lados só precisa ser avisado: passe a reagir a eventos |
| **Mover o dado ou a regra** | A informação está no módulo errado |
| **Extrair um terceiro módulo** | Os dois compartilham um conceito que merece existir |
| **Fundir os módulos** | Estão tão ligados que a fronteira não ajuda |
| **Aceitar, com contratos pequenos** | O custo de resolver é maior do que o benefício, por enquanto |

A última opção é válida, desde que seja consciente e limitada.

## 7.6 Evitando o N+1 entre módulos

O problema N+1 é um clássico de ORMs, e reaparece nas fronteiras. Imagine listar 50 pagamentos mostrando o nome de cada fornecedor. O código ingênuo chama `GetNameAsync(id)` 50 vezes. Em memória isso ainda gera 50 consultas ao banco. Em microsserviços, seriam 50 chamadas de rede.

A solução é pensar em **lotes** ao desenhar o contrato: uma chamada que recebe vários identificadores e devolve um dicionário.

## 7.7 Tradução na fronteira

Quando um módulo recebe dados de outro, ele pode precisar traduzi-los. Essa é a **camada anticorrupção** do DDD em tamanho pequeno. Se o contrato de outro módulo mudar, só a tradução muda, e não todo o seu módulo. Em um monolito, muitas vezes não vale criar uma camada formal. Mas vale manter a ideia: **não deixe tipos de outro módulo se espalharem pelo seu domínio**.

## 7.8 Evolução de contratos

Em um monolito, tudo compila junto. Isso é uma grande vantagem: se você mudar um contrato, o compilador mostra todos os consumidores na hora, e você atualiza tudo em um único commit. Não há versionamento, nem coordenação entre repositórios.

Essa facilidade também é uma tentação. Como mudar é fácil, contratos tendem a crescer sem critério. Antes de adicionar um método, vale a pergunta: isso é uma necessidade real do consumidor ou estou expondo meu interior?

CONSTRUINDO O LAÇOOS · PASSO 7

## Criando os contratos

Vamos criar o contrato do Budget. O arquivo inteiro cabe em uma tela:

**Tudo o que o resto do sistema sabe sobre o Budget** ·
src/Modules/Budget/Lacoos.Modules.Budget.Contracts/BudgetContracts.cs

```
namespace Lacoos.Modules.Budget.Contracts;

public sealed record BudgetFinancials(decimal TotalBudget, decimal Committed, decimal Balance,
decimal CommittedPercent);

public interface IBudgetQueries
{
    Task<bool> ItemExistsAsync(Guid budgetItemId, CancellationToken ct);

    Task<bool> HasItemsForSupplierAsync(Guid supplierId, CancellationToken ct);

    Task<IReadOnlyDictionary<Guid, string>> GetItemNamesAsync(IReadOnlyCollection<Guid> itemIds,
CancellationToken ct);

    Task<IReadOnlyList<Guid>> SearchItemIdsAsync(string search, CancellationToken ct);

    Task<BudgetFinancials> GetFinancialsAsync(CancellationToken ct);
}
```

São cinco métodos, para um módulo com mais de 500 linhas. Cada método existe porque alguém precisa dele:

| Método | Quem usa | Para quê |
|---|---|---|
| `ItemExistsAsync` | Payments | Validar o vínculo ao criar um pagamento |
| `HasItemsForSupplierAsync` | Suppliers | Bloquear a exclusão de fornecedor em uso |
| `GetItemNamesAsync` | Payments | Mostrar nomes em lote |
| `SearchItemIdsAsync` | Payments | Buscar pagamentos pelo nome do item |
| `GetFinancialsAsync` | Dashboard | Exibir o resumo financeiro |

O contrato é composto de tipos simples: `Guid`, `string`, `decimal` e um record. O projeto não referencia nada, como vimos no capítulo 3, e um teste de arquitetura garante que continue assim.

## A implementação é interna

**Implementação oculta do contrato** · src/Modules/Budget/Lacoos.Modules.Budget/Application/BudgetQueries.cs · linhas 7–18

```
internal sealed class BudgetQueries(BudgetDbContext db) : IBudgetQueries
{
    public Task<bool> ItemExistsAsync(Guid budgetItemId, CancellationToken ct) =>
        db.Items.AnyAsync(i => i.Id == budgetItemId, ct);

    public Task<bool> HasItemsForSupplierAsync(Guid supplierId, CancellationToken ct) =>
        db.Items.AnyAsync(i => i.SupplierId == supplierId, ct);

    public async Task<IReadOnlyDictionary<Guid, string>> GetItemNamesAsync(IReadOnlyCollection<Guid>
itemIds, CancellationToken ct) =>
        await db.Items.AsNoTracking()
            .Where(i => itemIds.Contains(i.Id))
            .ToDictionaryAsync(i => i.Id, i => i.Name, ct);
```

Essa classe é `internal`. O `Payments` nunca saberá que ela existe, nem que os dados vêm de uma tabela chamada `budget_items`.

## Consumindo o contrato

Veja como Payments usa o contrato ao validar uma requisição:

**Validação que cruza a fronteira** · src/Modules/Payments/Lacoos.Modules.Payments/Application/PaymentService.cs · linhas 455–463

```
if (request.SupplierId is { } supplierId && !await suppliers.ExistsAsync(supplierId, ct))
{
    throw new BusinessRuleException("Fornecedor não encontrado neste casamento.");
}

if (request.BudgetItemId is { } itemId && !await budget.ItemExistsAsync(itemId, ct))
{
    throw new BusinessRuleException("Item de orçamento não encontrado neste casamento.");
}
```

O resultado é uma chamada comum de método. Mas a mesma linha faz mais do que parece: como a consulta do Budget passa pelo filtro global de casamento, `ItemExistsAsync` só retorna `true` se o item pertencer **ao casamento atual**. Um usuário não consegue vincular um pagamento a um item de outro casamento, e nenhum código extra foi preciso. Veremos como isso funciona no capítulo 10.

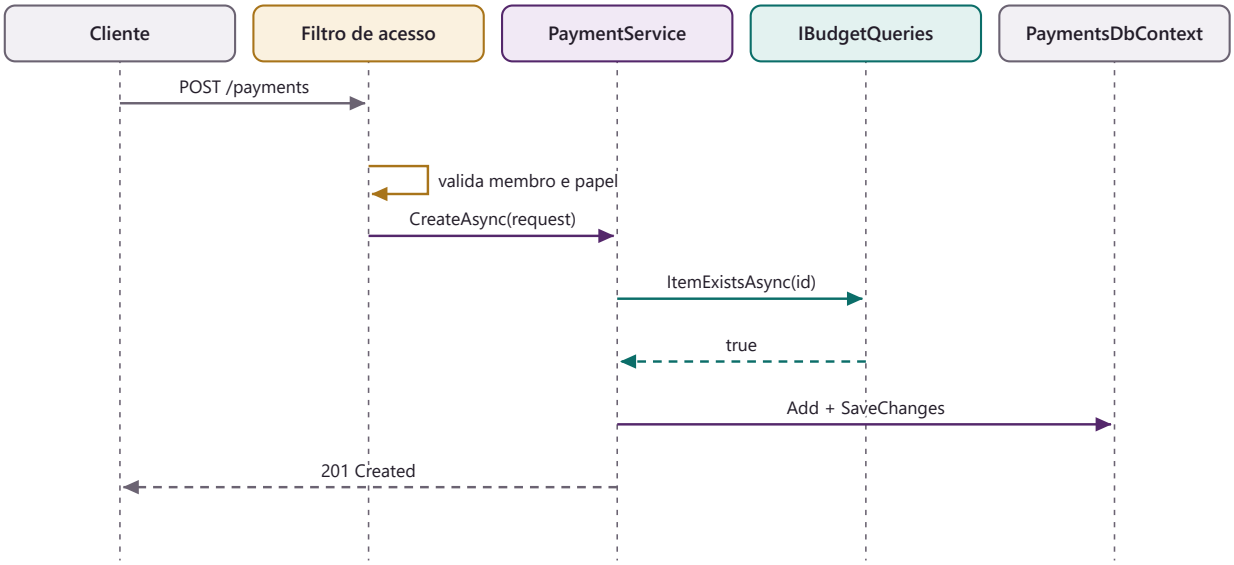



**Figura 7.1.** Criação de um pagamento com vínculo a um item de orçamento. A chamada entre módulos é comum, em memória.

## Consultas em lote

O contrato prevê lotes. Ao montar uma lista de pagamentos, são feitas duas consultas no total, não importa quantos pagamentos existam:

**Buscando nomes em lote** · src/Modules/Payments/Lacoos.Modules.Payments/Application/PaymentService.cs · linhas 488–493

```
private async Task<IReadOnlyList<PaymentResponse>> ToResponsesAsync(IReadOnlyList<Payment> payments,
CancellationToken ct)
{
    var supplierIds = payments.Where(p => p.SupplierId is not null).Select(p =>
p.SupplierId!.Value).Distinct().ToList();
    var itemIds = payments.Where(p => p.BudgetItemId is not null).Select(p =>
p.BudgetItemId!.Value).Distinct().ToList();
    var supplierNames = await suppliers.GetNamesAsync(supplierIds, ct);
    var itemNames = await budget.GetItemNamesAsync(itemIds, ct);
```

## Contratos de comando

Nem todo contrato é uma consulta. Às vezes um módulo precisa pedir uma ação a outro. Observe que a interface tem um nome diferente, porque tem outra natureza:

**Um contrato de comando** · src/Modules/Checklist/Lacoos.Modules.Checklist.Contracts/ChecklistContracts.cs · linhas 15–19

```
public interface IChecklistAssignments
{
    /// <summary>Deixa sem responsável as tarefas de um membro que será removido.</summary>
    Task UnassignMemberAsync(Guid memberId, CancellationToken ct);
}
```

O nome descreve a **intenção do negócio**, e não uma operação de banco. Essa separação entre `IChecklistQueries` e `IChecklistAssignments` aplica o princípio CQS no nível do contrato.

### As dependências mútuas do Laçoos

Agora vem a parte menos bonita. Listando as referências entre projetos, encontramos cinco pares que dependem um do outro:

| Par | Motivo de cada direção |
|---|---|
| **Budget ↔ Payments** | Budget mostra o valor pago. Payments valida o item de orçamento |
| **Budget ↔ Suppliers** | Budget mostra o nome. Suppliers bloqueia exclusão com itens |
| **Suppliers ↔ Payments** | Suppliers mostra pagamentos. Payments valida o fornecedor |
| **Weddings ↔ Checklist** | Weddings libera as tarefas ao remover um membro. Checklist valida responsáveis |
| **Guests ↔ Gifts** | Gifts mostra nomes. Guests preserva os vínculos ao excluir |

Pelo princípio das dependências acíclicas, esses pares são um problema. Por que aceitá-los?

Primeiro, cada direção tem um motivo real de negócio. Segundo, a dependência é sobre **contratos pequenos**, e não sobre interiores. Terceiro, os ciclos ficam visíveis e limitados: um teste impede que alguém acesse o interior de outro módulo, e o `.csproj` deixa cada dependência explícita.

> **O preço dos ciclos**
>
> Os ciclos são a maior fraqueza do desenho. O triângulo Budget, Payments e Suppliers não pode ser entendido nem extraído por partes: qualquer um dos três precisa dos outros dois. Há três caminhos para reduzir isso: fundir os três; publicar eventos para que uma das direções desapareça; ou introduzir um módulo acima dos três para compor os dados de leitura, assim como o Dashboard faz. Para o tamanho atual do projeto, nenhum deles se pagaria ainda. Mas esse é o primeiro lugar a revisar conforme o sistema crescer.

## 7.10 Resumo e exercícios

- A API pública de um módulo tem interfaces, tipos e, às vezes, eventos. Ela deve ser pequena e orientada ao uso.
- Nunca exponha entidades internas, `IQueryable` ou detalhes de infraestrutura em um contrato.
- Projetos de contrato permitem que o compilador proteja a fronteira e quebram ciclos de build.
- Ciclos lógicos continuam sendo um cheiro: resolva ou aceite de forma consciente.
- Desenhe contratos pensando em lotes para evitar o N+1 entre módulos.

### Exercícios

1. Imagine que `IBudgetQueries` retornasse `IQueryable<BudgetItem>`. Liste três coisas que deixariam de funcionar como fronteira.
2. A tela de fornecedores mostra o total pago a cada um. Olhe `GetPaidBySupplierAsync` em `IPaymentQueries`: por que ela devolve um dicionário, em vez de receber um `supplierId`?
3. Escolha um dos cinco pares de dependência mútua e proponha como eliminar uma das direções.

CAPÍTULO 8

# Dados: cada módulo é dono dos seus

*O banco de dados é onde a maioria dos monolitos "modulares" deixa de ser modular. É aqui que a fronteira fica séria.*


8.1 Por que os dados são o ponto crítico
8.2 Níveis de isolamento
8.3 Um DbContext por módulo
8.4 Referência por identificador
8.5 Sem JOIN, como fazer consultas?
8.6 Migrations em vários módulos
8.7 A persistência do Laçoos
8.8 Resumo e exercícios


## 8.1 Por que os dados são o ponto crítico

Você pode ter projetos separados, classes internas e contratos impecáveis. Mas, se um módulo faz `SELECT` nas tabelas de outro, todo esse trabalho se perde. Um `JOIN` entre tabelas de módulos diferentes é um acoplamento invisível:

- Ele **não aparece nas referências de projeto**.
- O compilador **não o detecta**.
- Uma coluna renomeada quebra outro módulo **só em tempo de execução**.
- Ele impede a extração do módulo no futuro.

A regra fundamental é a **posse dos dados**. Toda tabela tem exatamente um módulo dono. Só ele lê e grava. Os outros pedem pelo contrato.

> **Para lembrar**
>
> Se dois módulos gravam na mesma tabela, a fronteira está errada. Se um lê diretamente a tabela do outro, a fronteira existe só no papel.

## 8.2 Níveis de isolamento

Isolar dados não é uma escolha binária. Existe um espectro de opções:

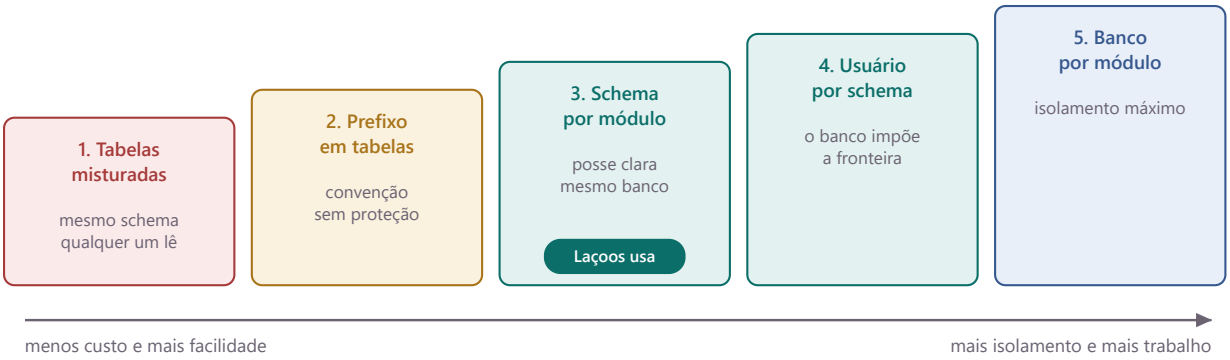



Figura 8.1. Níveis de isolamento de dados. Cada passo à direita aumenta a proteção e o custo.

**1. Tabelas misturadas.** Tudo no mesmo schema. Nada separa o que é de quem. Evite.

**2. Prefixo nas tabelas.** Uma convenção como `pay_installments`. Melhor do que nada, mas não impede nada.

**3. Schema por módulo.** Cada módulo tem seu schema no mesmo banco. A posse fica explícita, e backups e transações continuam simples. É a escolha padrão do mercado para monolitos modulares.

**4. Usuário por schema.** Além do schema, cada módulo se conecta com um usuário que só tem permissão nas próprias tabelas. Agora **o banco** impõe a fronteira. Custa mais configuração e mais strings de conexão.

**5. Banco por módulo.** Separação total. Permite até tecnologias diferentes, mas perde transações locais entre módulos e multiplica o trabalho de operação.

## 8.3 Um DbContext por módulo

Em EF Core, o caminho natural é ter um `DbContext` por módulo. Um contexto único para o sistema inteiro faria todas as entidades conviverem no mesmo modelo, com navegações entre elas, e destruiria a fronteira no primeiro dia.

Com contextos separados:

- Cada módulo mapeia só as próprias entidades.
- Cada um tem seu histórico de migrations.
- Duas equipes podem criar migrations sem conflito.
- Um módulo pode ser extraído levando seu contexto.

## 8.4 Referência por identificador

Se o módulo A precisa apontar para algo no módulo B, ele guarda **apenas o identificador**. Sem propriedade de navegação, sem `Include`, sem classe compartilhada. Essa técnica vem de uma regra de DDD para agregados: um agregado referencia outro pelo identificador, e não pelo objeto.

### E a chave estrangeira?

Uma das decisões mais discutidas é: devemos ter chave estrangeira (FK) entre schemas de módulos diferentes? Tecnicamente, o PostgreSQL permite.

| | Com FK entre módulos | Sem FK entre módulos |
|---|---|---|
| **Integridade** | O banco garante | A aplicação garante |
| **Extração futura** | Precisa remover a FK | Já está pronto |
| **Ordem de migrations** | Importa | Não importa |
| **Autonomia do módulo** | Menor | Maior |
| **Risco** | Exclusão bloqueada por outro módulo | Referência órfã em condições de corrida |

Não há resposta universal. Muitos projetos aceitam FKs no início, por segurança. A posição mais comum entre quem desenha para extração é **não usar FK entre módulos**, e compensar na aplicação.

## 8.5 Sem JOIN, como fazer consultas?

Quando uma tela precisa de dados de dois módulos, há algumas opções:

| Técnica | Como funciona | Custo |
|---|---|---|
| **Composição em memória** | Busca em A, depois em B, junta no código | Mais uma consulta |
| **Dados copiados** | A guarda uma cópia do que precisa de B | Sincronização |
| **Modelo de leitura** | Uma tabela própria alimentada por eventos | Infraestrutura e atraso |
| **View entre schemas** | Uma visão de banco com o JOIN | Acoplamento no banco |

A primeira é a mais simples e resolve a maioria dos casos em sistemas médios. Voltaremos a isso no capítulo 11.

## 8.6 Migrations em vários módulos

Cada módulo mantém suas migrations. É preciso decidir como aplicá-las. Em desenvolvimento, é cômodo rodar tudo ao iniciar. Em produção, é mais seguro um passo explícito, antes de subir a nova versão, para evitar que várias réplicas tentem migrar ao mesmo tempo.

Sem FKs entre módulos, a ordem de aplicação deixa de importar. Essa é uma vantagem discreta e bem-vinda.

CONSTRUINDO O LAÇOOS · PASSO 8

# A persistência do Laçoos

O Laçoos usa o nível 3 do espectro: um banco PostgreSQL com 8 schemas. Cada módulo tem seu próprio `DbContext`, seu histórico de migrations e nenhuma FK para fora. O Dashboard não tem persistência.

## A base comum

Todos os contextos herdam de uma classe no kernel, para garantir que nenhum módulo esqueça as regras importantes:

**Base dos contextos: schema próprio e precisão decimal** · src/BuildingBlocks/Lacoos.SharedKernel/Persistence/ModuleDbContext.cs · linhas 14–16, 23–29

```
public abstract class ModuleDbContext(DbContextOptions options, IWeddingContext wedding, string
schema)
    : DbContext(options)
{
// ...
    protected override void ConfigureConventions(ModelConfigurationBuilder builder) =>
        builder.Properties<decimal>().HavePrecision(14, 2);

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.HasDefaultSchema(schema);
        modelBuilder.ApplyConfigurationsFromAssembly(GetType().Assembly);
```

O construtor recebe o **schema** como parâmetro. Não há como criar um contexto sem definir o schema. A convenção de decimais garante `numeric(14,2)` para valores em reais, sem que ninguém precise lembrar disso a cada propriedade.

## O contexto do módulo

Com a base pronta, cada contexto fica curto:

**O contexto do Budget** · src/Modules/Budget/Lacoos.Modules.Budget/Persistence/BudgetDbContext.cs · linhas 7–19

```
internal sealed class BudgetDbContext(DbContextOptions<BudgetDbContext> options, IWeddingContext
wedding)
    : ModuleDbContext(options, wedding, SchemaName)
{
    public const string SchemaName = "budget";

    public DbSet<BudgetSettings> Settings => Set<BudgetSettings>();

    public DbSet<BudgetCategory> Categories => Set<BudgetCategory>();

    public DbSet<BudgetItem> Items => Set<BudgetItem>();
}

internal sealed class BudgetDbContextFactory() : ModuleDesignTimeFactory<BudgetDbContext>
(BudgetDbContext.SchemaName);
```

O nome do schema é uma constante pública. Um teste de arquitetura usa essa constante para verificar que cada módulo tem exatamente um contexto, e com o schema certo.

## O registro

Um método de extensão no kernel configura tudo do mesmo jeito:

**Registro padronizado de um contexto** · src/BuildingBlocks/Lacoos.SharedKernel/Persistence/ModuleDatabase.cs · linhas 24–46

```
public static class ModulePersistence
{
    public const string ConnectionStringName = "Postgres";

    public static IServiceCollection AddModuleDbContext<TContext>(
        this IServiceCollection services, IConfiguration configuration, string schema)
        where TContext : ModuleDbContext
    {
        services.AddDbContext<TContext>((provider, options) =>
        {
            var connectionString = provider.GetRequiredService<IConfiguration>
().GetConnectionString(ConnectionStringName)
                ?? throw new InvalidOperationException($"ConnectionStrings:{ConnectionStringName} não
configurada.");
            Configure(options, connectionString, schema);
        });
        services.AddScoped<IModuleDatabase, ModuleDatabase<TContext>>();
        return services;
    }

    internal static void Configure(DbContextOptionsBuilder options, string connectionString, string
schema) =>
        options
            .UseNpgsql(connectionString, npgsql =>
npgsql.MigrationsHistoryTable("__ef_migrations_history", schema))
            .UseSnakeCaseNamingConvention();
}
```

Dois detalhes importam. A tabela de histórico fica **dentro do schema do módulo**, então cada módulo tem a sua `__ef_migrations_history`. E, ao registrar `IModuleDatabase`, o contexto se junta a uma lista que o host percorre para aplicar as migrations, sem saber quais módulos existem.

Veja como é simples esse percurso:

**Aplicando as migrations de todos os módulos** · src/Host/Lacoos.Api/Infrastructure/StartupTasks.cs · linhas 8–17

```
public static async Task MigrateAsync(IServiceProvider services, CancellationToken ct)
{
    await using var scope = services.CreateAsyncScope();
    var logger = scope.ServiceProvider.GetRequiredService<ILogger<Program>>();
    foreach (var database in scope.ServiceProvider.GetServices<IModuleDatabase>())
    {
        logger.LogInformation("Aplicando migrations de {Module}", database.Name);
        await database.MigrateAsync(ct);
    }
}
```

## Criando uma migration

Com vários contextos, o comando precisa dizer qual módulo está sendo alterado:

Nova migration no módulo Budget

```
dotnet ef migrations add NomeDaMigration \
  --project src/Modules/Budget/Lacoos.Modules.Budget \
  --startup-project src/Host/Lacoos.Api \
  --context BudgetDbContext \
  --output-dir Persistence/Migrations
```

Para esse comando não precisar de um banco ligado, cada módulo tem uma pequena fábrica de tempo de projeto, a `BudgetDbContextFactory`, que você viu no código acima.

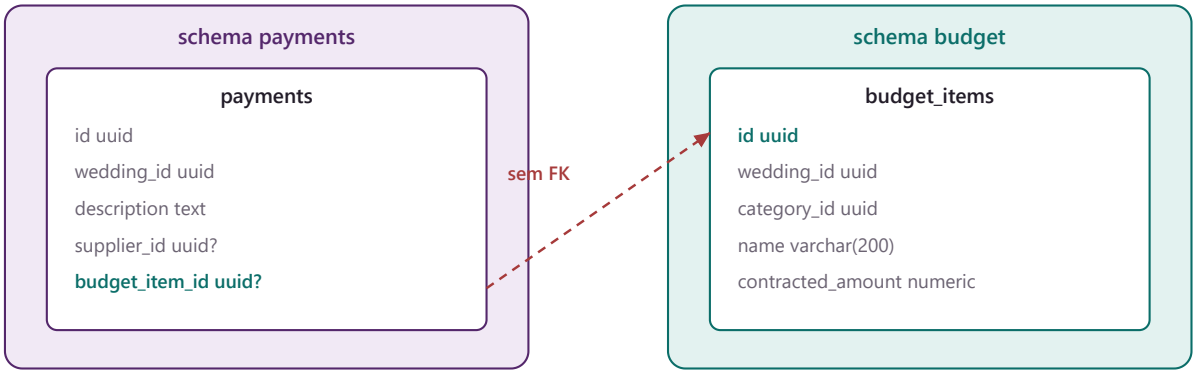
## A referência por identificador em ação

Compare as duas colunas abaixo. O item de orçamento tem um relacionamento **interno** com a categoria, do mesmo módulo, e a FK existe. O mesmo item aponta para o fornecedor, de **outro** módulo, só com um valor:

**FK dentro do módulo, valor simples para fora** · src/Modules/Budget/Lacoos.Modules.Budget/Persistence/BudgetDbContext.cs · linhas 39–49

```
internal sealed class BudgetItemConfiguration : IEntityTypeConfiguration<BudgetItem>
{
    public void Configure(EntityTypeBuilder<BudgetItem> builder)
    {
        builder.ToTable("budget_items");
        builder.Ignore(i => i.CommittedAmount);
        builder.Property(i => i.Name).HasMaxLength(200);
        builder.HasIndex(i => i.SupplierId);
        builder.HasOne(i => i.Category).WithMany().HasForeignKey(i =>
i.CategoryId).OnDelete(DeleteBehavior.Restrict);
    }
}
```

A linha `HasIndex(i => i.SupplierId)` mostra um cuidado importante. Sem FK, ninguém cria o índice por você, mas ele continua necessário para responder depressa a `HasItemsForSupplierAsync`.




O vínculo é só um valor. Quem garante a existência é o contrato: IBudgetQueries.ItemExistsAsync(id).

**Figura 8.2.** Um pagamento aponta para um item de orçamento apenas por um valor. A existência é garantida pelo contrato.

### Números reais

Vale olhar como o banco ficou. Estes números vêm do catálogo do PostgreSQL:

| Schema | Tabelas |
|---|---|
| `identity` | 3: `users`, `sessions`, `password_reset_tokens` |
| `weddings` | 3: `weddings`, `wedding_members`, `member_invitations` |
| `checklist` | 2: `checklist_categories`, `checklist_items` |
| `budget` | 3: `budget_settings`, `budget_categories`, `budget_items` |
| `guests` | 2: `guest_groups`, `guests` |
| `suppliers` | 3: `suppliers`, `supplier_services`, `supplier_contracts` |
| `payments` | 4: `payments`, `payment_installments`, `installment_transactions`, `payment_receipts` |
| `gifts` | 4: `gift_categories`, `gifts`, `gift_reservations`, `gift_receipts` |

São 24 tabelas de negócio e 17 chaves estrangeiras. **Nenhuma cruza schemas.** Toda relação entre módulos é um valor comum.

> **O que se perdeu sem a FK**
>
> Sem chave estrangeira, o banco não protege contra uma condição de corrida. Imagine que uma pessoa exclui um fornecedor no mesmo instante em que outra cria um pagamento para ele. O serviço de exclusão consulta `HasPaymentsForSupplierAsync`, recebe `false` e segue. O outro pedido consulta `ExistsAsync`, recebe `true` e grava. Resultado: um pagamento aponta para um fornecedor que não existe mais. A janela é pequena e o impacto é baixo: a tela mostra um nome vazio. Em um sistema financeiro com requisitos fortes, esse risco justificaria uma FK ou uma trava mais forte.

> **Usuário único no banco**
>
> Todos os módulos se conectam com o mesmo usuário. Nada no banco impede que um módulo leia o schema de outro, e a regra é protegida pelo código e pelos testes. Evoluir para o nível 4 aumentaria a proteção com um custo modesto.

## 8.8 Resumo e exercícios

- Toda tabela tem um único módulo dono, e só ele acessa diretamente.
- Schema por módulo é o melhor equilíbrio para a maioria dos monolitos modulares.
- Um `DbContext` e um histórico de migrations por módulo mantêm a independência.
- Entre módulos, guarde apenas identificadores. FK entre módulos é uma decisão consciente.
- Uma integridade que sai do banco precisa ser garantida pela aplicação, e isso tem custo.

## Exercícios

1. No catálogo do seu banco, consulte quantas FKs cruzam áreas de negócio diferentes. O que elas impediriam em uma extração?
2. Crie mentalmente uma coluna `nickname` em `guests.guests`. Quais arquivos mudam? Há algum fora do módulo Guests?
3. Como você implementaria o nível 4, com um usuário de banco por módulo, usando a estrutura do `AddModuleDbContext`?

CAPÍTULO 9

# Consistência e comunicação assíncrona

*Separar dados tem uma consequência: uma operação que toca dois módulos deixa de ser atômica. O que fazemos a respeito é uma das decisões mais importantes do sistema.*


9.1 O problema
9.2 ACID, transações e o que dá para fazer
9.3 Eventos
9.4 O problema da gravação dupla
9.5 O padrão Outbox
9.6 Idempotência e Inbox
9.7 Sagas
9.8 Consistência eventual
9.9 Concorrência dentro do módulo
9.10 Consistência no Laçoos
9.11 Resumo e exercícios


## 9.1 O problema

Com um único `DbContext`, a transação resolve tudo. Com vários, cada módulo faz o próprio `SaveChanges`. Imagine a exclusão de um convidado:

1. O módulo Gifts atualiza as reservas.
2. O módulo Guests remove o convidado.

Se o passo 2 falhar depois do passo 1, as reservas já mudaram. Não existe `ROLLBACK` que desfaça os dois, porque são duas transações. Bem-vindo ao problema central da consistência entre fronteiras.

## 9.2 ACID, transações e o que dá para fazer

Uma transação **ACID** garante atomicidade, consistência, isolamento e durabilidade. É um presente enorme dos bancos relacionais, e dá vontade de manter esse presente em tudo.

Em um monolito modular com banco único, isso ainda é **tecnicamente** possível. Os contextos podem compartilhar conexão e transação. Algumas equipes fazem isso, e a escolha tem lógica: é simples e consistente. O preço é que a fronteira deixa de valer em tempo de execução. Se o módulo for extraído, a transação compartilhada desaparece, e todo o código que dependia dela precisará ser repensado.

Temos, então, um leque de escolhas:

| Estratégia | Consistência | Complexidade | Prepara para extração |
|---|---|---|---|
| **Transação compartilhada** | Forte | Baixa | Não |
| **Passos ordenados e seguros** | Boa, com pequenos riscos | Baixa | Parcialmente |
| **Eventos em memória** | Fraca, se o processo cair | Baixa | Parcialmente |
| **Outbox e eventos** | Eventual, mas garantida | Média | Sim |
| **Saga** | Eventual, com compensação | Alta | Sim |

## 9.3 Eventos

Um **evento** é um fato que aconteceu, descrito no passado: `PagamentoRegistrado`, `ConvidadoExcluido`. É diferente de um **comando**, que é um pedido: `RegistrarPagamento`.

| | Comando | Evento |
|---|---|---|
| **Sentido** | Pedido, algo que deve acontecer | Fato, algo que já aconteceu |
| **Destinatários** | Um | Zero ou vários |
| **Pode ser recusado?** | Sim | Não: já aconteceu |
| **Quem conhece quem** | Quem envia conhece quem recebe | Quem publica não conhece quem reage |

Essa última linha é a mais importante. Eventos **invertem a dependência**. Com uma chamada, Guests precisa conhecer Gifts. Com um evento, Guests só anuncia "convidado excluído", e Gifts reage. A dependência muda de direção, o que ajuda a quebrar ciclos.

### Eventos de domínio e de integração

Costuma-se separar dois tipos:

- **Evento de domínio.** Interno a um módulo. Pode carregar tipos internos e acontece na mesma transação.
- **Evento de integração.** Atravessa a fronteira. Faz parte do contrato público e precisa ser estável.

## 9.4 O problema da gravação dupla

A primeira ideia costuma ser: grave no banco e, em seguida, publique o evento. Mas e se o processo cair entre os dois passos? O dado foi gravado e o evento se perdeu. Se você inverter a ordem, pode publicar um evento sobre algo que nunca foi gravado. Isso é o **problema da gravação dupla** (*dual write*).

## 9.5 O padrão Outbox

O **Transactional Outbox** resolve o problema usando o único recurso confiável disponível: a transação local.

1. Na **mesma transação** que grava o dado, grave também o evento em uma tabela `outbox`.
2. Um processo em segundo plano lê os eventos pendentes.
3. Ele entrega cada evento e marca como publicado.

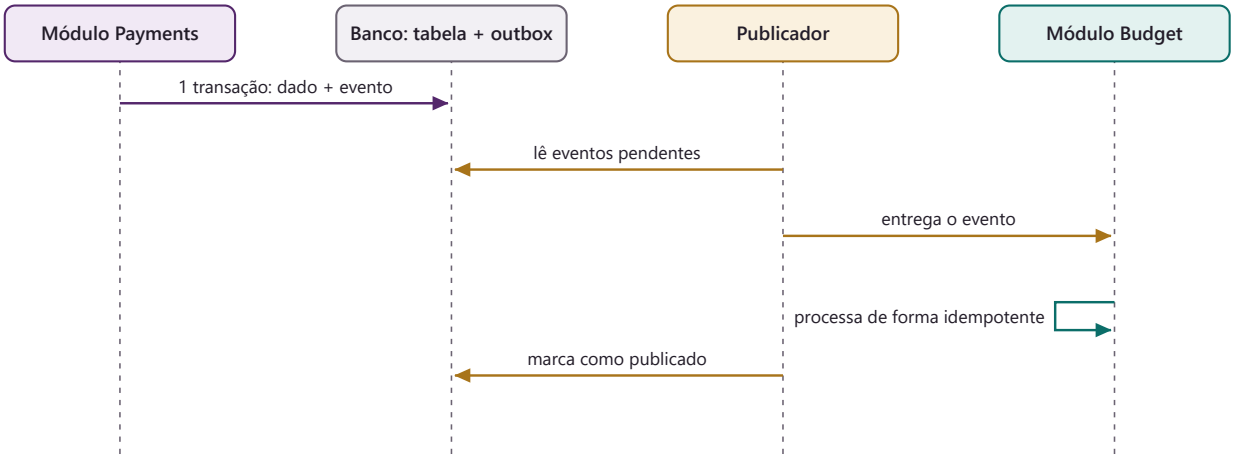



**Figura 9.1.** O padrão Outbox. Dado e evento nascem na mesma transação, e a entrega acontece depois.

O ponto importante: o evento só existe se a transação tiver sido confirmada. Em compensação, o publicador pode cair logo depois de entregar e antes de marcar, e entregará de novo. Por isso, a garantia é **pelo menos uma vez** (*at-least-once*). Quem recebe precisa lidar com repetições.

**Gravando o evento na mesma transação** · Exemplo ilustrativo: não faz parte do repositório

```
public async Task DeleteAsync(Guid id, CancellationToken ct)
{
    var guest = await LoadAsync(id, ct);
    db.Guests.Remove(guest);

    // O evento entra na mesma transação do SaveChanges.
    db.Outbox.Add(OutboxMessage.From(new GuestDeleted(guest.Id, guest.WeddingId, guest.FullName)));

    await db.SaveChangesAsync(ct);
}
```

## 9.6 Idempotência e Inbox

Uma operação é **idempotente** se executá-la duas vezes tem o mesmo efeito de executá-la uma. Se a entrega é "pelo menos uma vez", o consumidor precisa ser idempotente. Há dois caminhos:

- **Por natureza.** A operação já é segura para repetir: "limpar o `guest_id` das reservas do convidado X".
- **Por registro.** O padrão **Inbox**: o consumidor grava o identificador da mensagem processada, na mesma transação do efeito, e ignora repetidas.

**Consumidor idempotente com Inbox** · Exemplo ilustrativo: não faz parte do repositório

```
public async Task HandleAsync(GuestDeleted message, Guid messageId, CancellationToken ct)
{
    if (await db.Inbox.AnyAsync(m => m.Id == messageId, ct))
    {
        return; // Já processada: a entrega repetida não tem efeito.
    }

    await db.Reservations.Where(r => r.GuestId == message.GuestId)
        .ExecuteUpdateAsync(set => set
            .SetProperty(r => r.PersonName, message.FullName)
            .SetProperty(r => r.GuestId, (Guid?)null), ct);

    db.Inbox.Add(new InboxMessage { Id = messageId });
    await db.SaveChangesAsync(ct);
}
```

Em .NET, bibliotecas como **Wolverine** oferecem Outbox e Inbox integrados ao EF Core. O **MassTransit** também oferece, mas sua versão 9 foi anunciada com licença comercial, então confira as condições antes de adotar.

## 9.7 Sagas

Quando um processo de negócio atravessa vários módulos e precisa de um desfecho coordenado, usamos uma **saga**. O conceito vem de um artigo de 1987, de Garcia-Molina e Salem: uma transação longa é dividida em passos, cada um com uma **ação de compensação** que desfaz seu efeito.

Compensar não é um `ROLLBACK`: é uma ação de negócio. Não dá para "desfazer" um e-mail enviado, mas dá para enviar outro explicando.

Existem duas formas:

- **Coreografia.** Cada módulo reage a eventos e publica outros. Não há coordenador. É simples de começar, mas difícil de acompanhar.
- **Orquestração.** Um coordenador explícito envia comandos e registra o estado. É mais fácil de entender, mas tem mais código.

## 9.8 Consistência eventual

Com eventos, o sistema fica em estados intermediários por um tempo. O dado do módulo A já mudou, e o do módulo B ainda não. Isso se chama **consistência eventual**.

Pat Helland, em *Life beyond Distributed Transactions*, mostrou que sistemas grandes convivem com isso, e o mundo real também: um cheque leva dias para compensar. A pergunta certa para o negócio é: **quanto tempo de diferença é aceitável aqui?** Para o saldo exibido em um dashboard, alguns segundos não importam. Para evitar a venda dupla do último ingresso, importam muito.

> **Não adote eventos por moda**
>
> Eventos trazem depuração mais difícil, ordem de entrega, reprocessamento, mensagens mortas e um fluxo que não dá para ler de cima para baixo. Em um sistema pequeno, com poucos fluxos entre módulos, chamadas diretas costumam ser melhores. Use eventos quando houver um ganho concreto.

## 9.9 Concorrência dentro do módulo

A consistência também tem um lado local: duas requisições ao mesmo tempo. Imagine uma parcela com R$ 100 restantes e duas baixas simultâneas de R$ 80. Ambas leem "100 restantes", ambas passam na validação, e o sistema aceita R$ 160.

Há duas técnicas principais:

- **Concorrência otimista.** Uma coluna de versão detecta o conflito ao gravar. Quem perde tenta de novo.
- **Trava pessimista.** `SELECT ... FOR UPDATE` bloqueia a linha até o fim da transação. Serializa o acesso. Útil quando conflitos são prováveis ou quando a regra depende de uma soma.

CONSTRUINDO O LAÇOOS · PASSO 9

### Consistência no Laçoos

O Laçoos não usa eventos. Com poucos fluxos que cruzam módulos, a equipe escolheu a estratégia de **passos ordenados e seguros**. A ideia é simples: ordene os passos de forma que, se a operação falhar no meio, o sistema fique em um estado **aceitável**.

#### Excluindo um convidado

**Passos ordenados com um comentário que explica o porquê** ·
src/Modules/Guests/Lacoos.Modules.Guests/Application/GuestService.cs · linhas 149–158

```
public async Task DeleteAsync(Guid id, CancellationToken ct)
{
    var guest = await LoadAsync(id, ct);

    // Primeiro preserva o nome nos presentes: se a exclusão falhar, nenhum vínculo fica quebrado.
    await gifts.DetachGuestAsync(guest.Id, guest.FullName, ct);

    db.Guests.Remove(guest);
    await db.SaveChangesAsync(ct);
}
```

Observe a ordem e pense nas falhas possíveis:

| Se falhar... | Estado final | Aceitável? |
|---|---|---|
| Antes de `DetachGuestAsync` | Nada mudou | Sim |

| Se falhar... | Estado final | Aceitável? |
|---|---|---|
| Em `DetachGuestAsync` | Nada mudou no convidado | Sim |
| Depois, ao remover o convidado | Presentes preservam o nome, e o convidado continua existindo | Sim: é só repetir |

Se a ordem fosse a inversa, uma falha no segundo passo deixaria reservas apontando para um convidado inexistente. Com esta ordem, a pior situação é inofensiva, e **a operação pode ser repetida**.

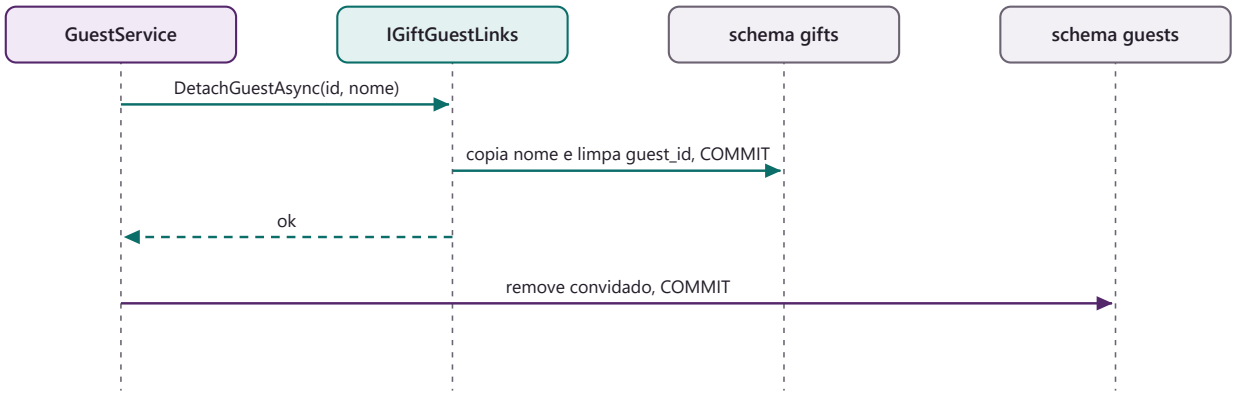



Figura 9.2. A exclusão acontece em duas transações. A ordem foi escolhida para que uma falha no meio seja segura.

O outro lado da operação é idempotente por natureza. Rodar duas vezes não causa nada de errado, porque a segunda vez não encontra registros:

**Operação segura para repetir** · src/Modules/Gifts/Lacoos.Modules.Gifts/Application/GiftGuestLinks.cs · linhas 7–16

```
internal sealed class GiftGuestLinks(GiftsDbContext db) : IGiftGuestLinks
{
    public async Task DetachGuestAsync(Guid guestId, string guestName, CancellationToken ct)
    {
        await db.Reservations.Where(r => r.GuestId == guestId)
            .ExecuteUpdateAsync(set => set.SetProperty(r => r.PersonName, guestName).SetProperty(r =>
r.GuestId, (Guid?)null), ct);
        await db.Receipts.Where(r => r.GuestId == guestId)
            .ExecuteUpdateAsync(set => set.SetProperty(r => r.PersonName, guestName).SetProperty(r =>
r.GuestId, (Guid?)null), ct);
    }
}
```

Note também o que esse método preserva. Excluir um convidado **não apaga** o histórico de quem deu o presente: o nome é copiado para `PersonName`. É uma forma de pequena duplicação de dados, feita de propósito, porque esse registro histórico precisa sobreviver.

## Removendo um membro

O mesmo padrão aparece em `Weddings` e `Checklist`:

**Liberar as tarefas antes de remover** · src/Modules/Weddings/Lacoos.Modules.Weddings/Application/MemberService.cs · linhas 145–162

```
public async Task RemoveAsync(Guid memberId, CancellationToken ct)
{
    await using var transaction = await db.Database.BeginTransactionAsync(ct);
    await db.LockRowAsync(WeddingsDbContext.WeddingsTable, WeddingId, ct);

    var member = await LoadAsync(memberId, ct);
    if (member.Role == WeddingRole.Owner)
    {
        await EnsureNotLastOwnerAsync(member, ct);
    }

    // Primeiro libera as tarefas: se a exclusão falhar depois, nada aponta para um membro
inexistente.
    await checklist.UnassignMemberAsync(member.Id, ct);

    db.Members.Remove(member);
    await db.SaveChangesAsync(ct);
    await transaction.CommitAsync(ct);
}
```

Esse trecho tem dois níveis de consistência. A transação e a trava protegem a regra **interna** do módulo: um casamento nunca pode ficar sem nenhum owner. Já `UnassignMemberAsync` roda em outro `DbContext`, com a própria transação, e não é desfeita se o resto falhar. Se isso acontecer, as tarefas ficam sem responsável, mas o membro continua lá. É um estado aceitável.

## Travas contra concorrência

Regras de limite usam trava pessimista. A extensão fica no kernel, porque vários módulos precisam dela:

**Trava de linha** · src/BuildingBlocks/Lacoos.SharedKernel/Persistence/RowLock.cs · linhas 5–17

```
public static class RowLock
{
    /// <summary>
    /// Bloqueia uma linha até o fim da transação atual, para que validações de limite
    /// (baixas, reservas, último owner) não sejam furadas por requisições simultâneas.
    /// </summary>
    public static Task LockRowAsync(this DbContext context, string qualifiedTable, Guid id,
CancellationToken ct)
    {
#pragma warning disable EF1002 // O nome da tabela vem de constantes do código; o ID segue como
parâmetro.
        return context.Database.ExecuteSqlRawAsync($"SELECT 1 FROM {qualifiedTable} WHERE id = {{0}}
FOR UPDATE", [id], ct);
#pragma warning restore EF1002
    }
}
```

Veja como a baixa de uma parcela usa a trava. Dentro da transação, `LoadLockedInstallmentAsync` bloqueia a linha, e só então o saldo é verificado:

**Baixa protegida contra requisições simultâneas** ·
src/Modules/Payments/Lacoos.Modules.Payments/Application/PaymentService.cs · linhas 274–293

```
await using var transaction = await db.Database.BeginTransactionAsync(ct);
var installment = await LoadLockedInstallmentAsync(installmentId, ct);

if (request.Amount!.Value > installment.RemainingAmount)
{
    throw new BusinessRuleException($"A baixa excede o valor restante da parcela
({Text.Brl(installment.RemainingAmount)}).");
}

var method = request.PaymentMethod
    ?? await db.Payments.Where(p => p.Id == installment.PaymentId).Select(p =>
p.PaymentMethod).FirstAsync(ct);

var entry = NewTransaction(installment.Id, TransactionType.Settlement, request.Amount.Value);
entry.PaidAt = request.PaidAt ?? clock.Today;
entry.PaymentMethod = method;
entry.Notes = Text.Clean(request.Notes);
db.Transactions.Add(entry);

await db.SaveChangesAsync(ct);
await transaction.CommitAsync(ct);
return ToResponse(entry);
```

Uma segunda requisição fica parada até a primeira terminar, e então enxerga o saldo atualizado. Testes de integração no repositório disparam baixas simultâneas para confirmar esse comportamento.

## Como seria evoluir para eventos

Se a exclusão de um convidado precisasse notificar vários módulos, ou se Gifts fosse extraído, trocaríamos a chamada por um evento. Essa mudança eliminaria uma direção da dependência mútua Guests ↔ Gifts, como vimos no capítulo 7:

**Contrato de evento no projeto Guests.Contracts** · Exemplo ilustrativo: não faz parte do repositório

```
public sealed record GuestDeleted(Guid GuestId, Guid WeddingId, string FullName);
```

Guests deixaria de conhecer Gifts. O preço seria uma tabela de outbox, um processo em segundo plano, consumidores idempotentes e um pequeno atraso. Para dois módulos em um sistema pequeno, ainda não compensa. Quando esses fluxos se multiplicarem, compensará.

> ### O que ainda pode dar errado
>
> A estratégia aceita um cenário imperfeito: um arquivo apagado do disco e uma transação são coisas separadas. Em `SupplierAppService.DeleteAsync`, o fornecedor é removido do banco primeiro e os arquivos dos contratos depois. Se a exclusão do arquivo falhar, sobra um arquivo órfão. A escolha é consciente: arquivo órfão ocupa disco, enquanto um registro apontando para um arquivo inexistente quebraria o sistema.

## 9.11 Resumo e exercícios

- Separar dados significa perder a atomicidade entre módulos, e essa perda precisa ser tratada.
- Passos ordenados são uma solução simples: escolha a ordem que torna as falhas seguras e torne a repetição inofensiva.
- Eventos invertem dependências, mas trazem custos reais.
- O padrão Outbox resolve a gravação dupla. A entrega é pelo menos uma vez, então consumidores precisam ser idempotentes.
- Sagas coordenam processos longos com ações de compensação.
- Regras de limite precisam de proteção contra concorrência, como `SELECT FOR UPDATE`.

### Exercícios

1. Em uma operação que você conhece e que toca duas áreas, liste todos os pontos de falha. Existe uma ordem mais segura?
2. Por que a limpeza de `guest_id` é idempotente por natureza, enquanto "somar 1 ao contador de presentes" não é?
3. Desenhe, em papel, o fluxo da exclusão de um convidado usando Outbox. Quantos estados intermediários o usuário poderia ver?

CAPÍTULO 10

# O que é compartilhado: kernel e temas transversais

*Compartilhar demais recria o acoplamento que você evitou. Compartilhar de menos multiplica a duplicação. A arte está no critério.*


10.1 Preocupações transversais
10.2 Shared Kernel
10.3 Portas compartilhadas
10.4 Inversão de dependência para o kernel
10.5 Multi-tenancy
10.6 Tratamento de erros
10.7 Tempo
10.8 O kernel do Laçoos
10.9 Resumo e exercícios


## 10.1 Preocupações transversais

Algumas necessidades atravessam todos os módulos: autenticação, autorização, tratamento de erros, validação, logs, data e hora. São as **preocupações transversais** (*cross-cutting concerns*). Se cada módulo as resolver sozinho, haverá nove respostas de erro diferentes e nove formas de checar permissão. Se tudo for para um lugar comum, esse lugar vira um novo centro de acoplamento.

## 10.2 Shared Kernel

No DDD, o **kernel compartilhado** é uma parte pequena do modelo que dois ou mais contextos concordam em dividir. Ele tem uma regra de ouro: mudar o kernel afeta todo mundo, então ele precisa ser **pequeno, estável e sem regras de negócio**.

Um teste simples para decidir se algo entra:

1. **É necessário em vários módulos?** Se é usado por um só, deve ficar nele.
2. **É infraestrutura, e não negócio?** "Validar e-mail" pode entrar. "Calcular juros" não.
3. **Muda raramente?** Algo que muda toda semana não deve ser compartilhado.
4. **O significado é o mesmo em todos os lugares?** Se cada módulo precisa de uma variação, não é a mesma coisa.

> **O kernel como lixeira**
>
> Uma pasta `Common` ou `Utils` é onde as fronteiras vão morrer. A tentação é sempre a mesma: "isso pode ser útil em outro lugar". Sem critério, o kernel acumula conceitos de negócio, todos passam a depender deles, e o sistema volta a ser uma bola de lama, agora com uma camada extra.

### Duplicar pode ser melhor

O princípio DRY (*Don't Repeat Yourself*) é frequentemente mal entendido. Duplicar **conhecimento** é ruim. Mas duas partes de código parecidas podem representar decisões diferentes, que vão divergir com o tempo. Sandi Metz resumiu isso em uma frase conhecida: duplicação é muito mais barata do que a abstração errada. Entre módulos, um pouco de repetição costuma custar menos do que uma dependência nova.

## 10.3 Portas compartilhadas

Um tipo útil de compartilhamento é o de **portas**, como `IFileStorage` e `IEmailSender`. O kernel define a interface, e a implementação fica fora, no host. Assim, os módulos dependem de uma abstração estável, e a tecnologia pode mudar. No Laçoos, trocar o disco local pelo Azure Blob exigiria uma classe nova e uma linha no `Program.cs`, sem alterar nenhum módulo.

## 10.4 Inversão de dependência para o kernel

Existe um caso que parece sem solução: o kernel precisa de uma informação que só um módulo tem. O kernel não pode depender do módulo. A saída é o **princípio da inversão de dependência**: o kernel define a interface, o módulo a implementa, e o contêiner liga as duas partes.

## 10.5 Multi-tenancy

Um sistema **multi-tenant** atende vários clientes isolados. No Laçoos, o "cliente" é um casamento. Há três estratégias clássicas:

| Estratégia | Isolamento | Custo por cliente | Uso típico |
|---|---|---|---|
| **Banco por cliente** | Máximo | Alto | Poucos clientes grandes, exigência regulatória |
| **Schema por cliente** | Alto | Médio | Número moderado de clientes |
| **Coluna de cliente** | Lógico | Baixo | Muitos clientes pequenos |

A estratégia da coluna é a mais escalável, mas exige que **toda** consulta filtre pelo cliente. Esquecer um `WHERE` em uma única consulta causa vazamento de dados. Por isso, o filtro não deve depender de memória humana. Deve ser automático e, se possível, defensivo: a ausência de contexto precisa resultar em **nada**, e não em tudo.

Um reforço possível é o **Row-Level Security** do PostgreSQL, que aplica o filtro no próprio banco.

## 10.6 Tratamento de erros

Erros são parte do contrato da API. Um padrão de mercado é o **Problem Details** (RFC 9457, que substituiu a RFC 7807), um formato JSON padronizado para erros HTTP. Em uma arquitetura modular, uma boa divisão é:

- Os módulos **lançam** exceções com significado de negócio.
- Um tratador central **traduz** essas exceções em respostas HTTP.

## 10.7 Tempo

Código que chama `DateTime.Now` é difícil de testar e tem um risco sutil: qual é "hoje"? Em um servidor em UTC, às 22h de São Paulo já é amanhã. Uma abstração de relógio resolve os dois problemas. O .NET oferece `TimeProvider` para isso desde a versão 8. O Laçoos optou por uma interface própria, porque precisa de um conceito de domínio: "hoje" significa hoje em São Paulo.

CONSTRUINDO O LAÇOOS · PASSO 10

### O kernel do Laçoos

O kernel tem 754 linhas, organizadas por assunto. Não existe pasta `Utils`:

| Pasta | Conteúdo |
|---|---|
| `Modules` | `IModule`, `IModuleSeeder` |
| `Persistence` | `ModuleDbContext`, `AddModuleDbContext`, `RowLock` |
| `Tenancy` | `WeddingContext`, `WeddingAccessFilter`, `ICurrentUser` |
| `Errors`, `Web` | Exceções de negócio e `AppExceptionHandler` |
| `Validation` | `Validator`, `Text`, `EnumText` |
| `Time` | `IClock` e o relógio de São Paulo |
| `Files`, `Email` | Portas `IFileStorage` e `IEmailSender` |

### Isolamento por casamento

Esse é o exemplo mais rico do kernel, e talvez a parte mais importante da segurança do sistema. O objetivo: nenhum casamento pode ver dados de outro, e **nenhum módulo precisa lembrar disso**.




Figura 10.1. O caminho de uma requisição. Depois do filtro, o isolamento se aplica sozinho em todas as consultas.

**Passo 1: agrupar as rotas.** Todas as rotas de negócio passam por este método, que exige autenticação e aplica o filtro:

**Grupo de rotas protegido** · src/BuildingBlocks/Lacoos.SharedKernel/Tenancy/WeddingAccessFilter.cs · linhas 51–67

```
public static class WeddingRoutes
{
    public const string Prefix = "/api/weddings/{weddingId:guid}";

    /// <summary>Grupo autenticado, com checagem de papel e isolamento por casamento.</summary>
    public static RouteGroupBuilder MapWeddingGroup(this IEndpointRouteBuilder app, string tag)
    {
        var group = app.MapGroup(Prefix);
        group.RequireAuthorization();
        group.AddEndpointFilter<WeddingAccessFilter>();
        group.WithTags(tag);
        return group;
    }

    public static T RequireRole<T>(this T builder, WeddingRole role) where T :
IEndpointConventionBuilder =>
        builder.WithMetadata(new RequiredRole(role));
}
```

**Passo 2: verificar o acesso.** O filtro roda em toda requisição:

**O filtro de acesso** · src/BuildingBlocks/Lacoos.SharedKernel/Tenancy/WeddingAccessFilter.cs · linhas 15–49

```
public sealed class WeddingAccessFilter : IEndpointFilter
{
    public async ValueTask<object?> InvokeAsync(EndpointFilterInvocationContext context,
EndpointFilterDelegate next)
    {
        var http = context.HttpContext;
        var services = http.RequestServices;

        if (!Guid.TryParse(http.Request.RouteValues["weddingId"]?.ToString(), out var weddingId))
        {
            throw new NotFoundException("Casamento não encontrado.");
        }

        var userId = services.GetRequiredService<ICurrentUser>().RequiredUserId;
        var role = await services.GetRequiredService<IWeddingMembershipResolver>()
            .GetActiveRoleAsync(weddingId, userId, http.RequestAborted);

        // Quem não é membro recebe 404, para não revelar que o casamento existe.
        if (role is null)
        {
            throw new NotFoundException("Casamento não encontrado.");
        }

        var required = http.GetEndpoint()?.Metadata.GetMetadata<RequiredRole>()?.Role ??
DefaultRole(http.Request.Method);
        if (role < required)
        {
            throw new ForbiddenException("Seu papel neste casamento não permite esta ação.");
        }

        services.GetRequiredService<WeddingContext>().Set(weddingId, role);
        return await next(context);
    }

    private static WeddingRole DefaultRole(string method) =>
```

```
        HttpMethods.IsGet(method) || HttpMethods.IsHead(method) ? WeddingRole.Viewer :
WeddingRole.Organizer;
}
```

Vale ler com calma:

- **Quem não é membro recebe 404, e não 403.** Um 403 confirmaria que o casamento existe. O comentário explica o motivo.
- **O papel padrão vem do verbo HTTP.** `GET` exige `viewer`, e os demais exigem `organizer`. Uma rota nova já nasce protegida.
- **Rotas que exigem mais** declaram isso com `.RequireRole(WeddingRole.Owner)`.
- `role < required` funciona porque o enum foi definido em ordem crescente de poder.

**Passo 3: inversão de dependência.** O filtro precisa saber o papel do usuário, informação que pertence ao módulo Weddings. Mas o kernel não pode conhecer esse módulo. A interface fica no kernel:

**A interface é declarada no kernel** · src/BuildingBlocks/Lacoos.SharedKernel/Tenancy/WeddingContext.cs · linhas 31–35

```
/// <summary>Implementado pelo módulo Weddings: papel de um usuário ativo no casamento.</summary>
public interface IWeddingMembershipResolver
{
    Task<WeddingRole?> GetActiveRoleAsync(Guid weddingId, Guid userId, CancellationToken ct);
}
```

E a implementação, no módulo:

**A implementação fica no módulo Weddings** ·
src/Modules/Weddings/Lacoos.Modules.Weddings/Application/WeddingQueries.cs · linhas 6–13

```
internal sealed class MembershipResolver(WeddingsDbContext db) : IWeddingMembershipResolver
{
    public async Task<WeddingRole?> GetActiveRoleAsync(Guid weddingId, Guid userId, CancellationToken
ct) =>
        await db.Members.AsNoTracking()
            .Where(m => m.WeddingId == weddingId && m.UserId == userId && m.Status ==
MemberStatus.Active)
            .Select(m => (WeddingRole?)m.Role)
            .FirstOrDefaultAsync(ct);
}
```

**Passo 4: filtro automático.** Depois da verificação, o casamento atual fica guardado em um serviço com escopo de requisição. Qualquer entidade que implementa `IWeddingScoped` recebe um filtro global:

**O filtro global por casamento** · src/BuildingBlocks/Lacoos.SharedKernel/Persistence/ModuleDbContext.cs · linhas 20–21, 59–64

```
/// <summary>Sem casamento na requisição, o filtro usa um ID vazio e nada é retornado.</summary>
public Guid CurrentWeddingId => wedding.WeddingId ?? Guid.Empty;
// ...
private void ApplyWeddingFilter<T>(ModelBuilder modelBuilder) where T : class, IWeddingScoped
{
    Expression<Func<T, bool>> filter = e => e.WeddingId == CurrentWeddingId;
    modelBuilder.Entity<T>().HasQueryFilter(filter);
    modelBuilder.Entity<T>().HasIndex(e => e.WeddingId);
}
```

O comentário da primeira linha descreve a escolha defensiva: **sem casamento na requisição, o filtro usa um ID vazio, e nada é retornado**. Se alguém esquecer de definir o contexto, a consulta devolve zero linhas, e não todas.

**Passo 5: gravação automática.** O mesmo vale para gravar. O `WeddingId` é preenchido sozinho, e a gravação falha se não houver casamento definido:

**O WeddingId é preenchido automaticamente** · src/BuildingBlocks/Lacoos.SharedKernel/Persistence/ModuleDbContext.cs · linhas 45–57

```
public override Task<int> SaveChangesAsync(bool acceptAllChangesOnSuccess, CancellationToken
cancellationToken = default)
{
    foreach (var entry in ChangeTracker.Entries<IWeddingScoped>().Where(e => e.State ==
EntityState.Added))
    {
        if (entry.Entity.WeddingId == Guid.Empty)
        {
            entry.Entity.WeddingId = wedding.WeddingId
                ?? throw new InvalidOperationException("Gravação sem casamento definido na
requisição.");
        }
    }

    return base.SaveChangesAsync(acceptAllChangesOnSuccess, cancellationToken);
}
```

Agora volte ao capítulo 7. Quando o Payments chama `ItemExistsAsync`, a consulta do Budget aplica esse mesmo filtro. Como o `WeddingContext` tem escopo de requisição, ele é **o mesmo objeto** em todos os módulos. O isolamento atravessa as fronteiras sem nenhum parâmetro extra.

> **Para lembrar**
>
> Segurança que depende de cada desenvolvedor lembrar de algo vai falhar um dia. No Laçoos, o caminho mais fácil é o caminho seguro: para vazar dados, alguém precisaria desligar o filtro de propósito.

## Erros

Os módulos lançam exceções com significado. Cada uma carrega o status HTTP correspondente:

**Erros de negócio** · src/BuildingBlocks/Lacoos.SharedKernel/Errors/AppExceptions.cs · linhas 3–25

```
/// <summary>Erro esperado de negócio, convertido em ProblemDetails com mensagem em português.
</summary>
public abstract class AppException(string message, int statusCode) : Exception(message)
{
    public int StatusCode { get; } = statusCode;
}

public sealed class NotFoundException(string message) : AppException(message, 404);

public sealed class ForbiddenException(string message) : AppException(message, 403);

public sealed class UnauthorizedException(string message) : AppException(message, 401);

/// <summary>A operação conflita com o estado atual, como excluir um registro em uso.</summary>
public sealed class ConflictException(string message) : AppException(message, 409);

/// <summary>A requisição é bem formada, mas viola uma regra de negócio.</summary>
public sealed class BusinessRuleException(string message) : AppException(message, 422);

public sealed class ValidationException(IReadOnlyDictionary<string, string[]> errors)
    : AppException("Um ou mais campos são inválidos.", 400)
{
    public IReadOnlyDictionary<string, string[]> Errors { get; } = errors;
}
```

Um único tratador central traduz tudo para Problem Details, com mensagens em português:

**Tradução central de erros** · src/BuildingBlocks/Lacoos.SharedKernel/Web/AppExceptionHandler.cs · linhas 14–34

```
public async ValueTask<bool> TryHandleAsync(HttpContext http, Exception exception, CancellationToken
ct)
{
    var (status, detail) = exception switch
    {
        AppException app => (app.StatusCode, app.Message),
        BadHttpRequestException bad when bad.StatusCode == StatusCodes.Status413PayloadTooLarge =>
            (413, "O arquivo enviado é maior que o limite permitido."),
        BadHttpRequestException => (400, "O corpo da requisição é inválido. Verifique os campos e os
formatos enviados."),
        DbUpdateConcurrencyException => (409, "O registro foi alterado por outra pessoa. Atualize e
tente novamente."),
        DbUpdateException { InnerException: PostgresException { SqlState:
PostgresErrorCodes.UniqueViolation } } =>
            (409, "Já existe um registro com estes dados."),
        DbUpdateException { InnerException: PostgresException { SqlState:
PostgresErrorCodes.ForeignKeyViolation } } =>
            (409, "O registro está em uso por outros dados."),
        _ => (500, "Ocorreu um erro inesperado. Tente novamente."),
    };

    if (status >= 500)
    {
        logger.LogError(exception, "Erro não tratado em {Method} {Path}", http.Request.Method,
http.Request.Path);
    }
```

Observe a mensagem de 500: ela não revela detalhes internos, mas o erro completo vai para o log. O cliente recebe o que pode ver, e a equipe recebe o que precisa.

## O relógio

**"Hoje" é um conceito de negócio** · src/BuildingBlocks/Lacoos.SharedKernel/Time/Clock.cs · linhas 3–21

```
public interface IClock
{
    DateTimeOffset UtcNow { get; }

    /// <summary>Data de hoje em America/Sao_Paulo, nunca a data UTC.</summary>
    DateOnly Today { get; }
}

public sealed class SaoPauloClock : IClock
{
    private static readonly TimeZoneInfo SaoPaulo =
TimeZoneInfo.FindSystemTimeZoneById("America/Sao_Paulo");

    public DateTimeOffset UtcNow => DateTimeOffset.UtcNow;

    public DateOnly Today => ToSaoPauloDate(UtcNow);

    public static DateOnly ToSaoPauloDate(DateTimeOffset instant) =>
        DateOnly.FromDateTime(TimeZoneInfo.ConvertTime(instant, SaoPaulo).DateTime);
}
```

Parcelas atrasadas dependem da data de hoje. Com essa abstração, os testes fixam o tempo, e o sistema calcula "hoje" corretamente, mesmo com o servidor em UTC.

## Portas: arquivos e e-mail

As portas ficam no kernel, e as implementações, no host:

**A porta de armazenamento** · src/BuildingBlocks/Lacoos.SharedKernel/Files/FileStorage.cs · linhas 6–13

```
public interface IFileStorage
{
    Task SaveAsync(string key, Stream content, CancellationToken ct);

    Task<Stream?> OpenReadAsync(string key, CancellationToken ct);

    Task DeleteAsync(string key, CancellationToken ct);
}
```

O `Program.cs` faz a ligação, em duas linhas:

**Implementações escolhidas pelo host** · src/Host/Lacoos.Api/Program.cs · linhas 52–53

```
builder.Services.AddSingleton<IEmailSender, SmtpEmailSender>();
builder.Services.AddSingleton<IFileStorage, LocalFileStorage>();
```

### Autenticação como módulo

A autenticação é uma preocupação transversal, mas não está no kernel. Fica no módulo Identity, por ter regras próprias: senhas, sessões, recuperação. O que o kernel conhece é só `ICurrentUser`, que lê o usuário da requisição:

**O kernel só conhece o usuário atual** · src/BuildingBlocks/Lacoos.SharedKernel/Tenancy/CurrentUser.cs · linhas 7–20

```
public interface ICurrentUser
{
    Guid? UserId { get; }

    Guid RequiredUserId { get; }
}

public sealed class HttpCurrentUser(IHttpContextAccessor accessor) : ICurrentUser
{
    public Guid? UserId =>
        Guid.TryParse(accessor.HttpContext?.User.FindFirstValue(ClaimTypes.NameIdentifier), out var
id) ? id : null;

    public Guid RequiredUserId => UserId ?? throw new UnauthorizedException("Autenticação
necessária.");
}
```

Esse é um bom exemplo de critério: o **conceito de usuário atual** é transversal, enquanto a **forma de autenticar** é um subdomínio com dono.

> #### O kernel depende de tecnologia
>
> O projeto do kernel referencia ASP.NET Core, EF Core e o provedor do PostgreSQL. Puristas diriam que um kernel deveria ser independente de frameworks. Aqui a escolha foi pragmática: as partes mais valiosas do kernel, como o filtro de acesso e a base dos contextos, dependem desses frameworks, e todos os módulos usam a mesma pilha. A consequência é que mudar de banco afetaria todos os módulos. Se isso incomodar, a saída seria dividir o kernel em projetos menores, como `Kernel.Abstractions` e `Kernel.Persistence`.

## 10.9 Resumo e exercícios

- O kernel compartilhado deve ser pequeno, estável e sem regras de negócio.
- Duplicar um pouco entre módulos costuma custar menos do que uma abstração errada.
- Portas no kernel e implementações no host mantêm os módulos independentes da tecnologia.
- A inversão de dependência permite que o kernel use informações de um módulo sem conhecê-lo.
- Em multi-tenancy por coluna, o filtro precisa ser automático e falhar de forma segura.
- Trate os erros de forma central, e lance exceções com significado de negócio.

### Exercícios

1. Na sua base de código, escolha cinco itens de uma pasta `Common`. Quantos passam nos quatro critérios do kernel?

2. Por que é melhor o filtro global usar `Guid.Empty` do que simplesmente não filtrar quando não há casamento?
3. Que riscos surgem se um desenvolvedor usar `IgnoreQueryFilters()`? Que regra de arquitetura ajudaria a controlar isso?

CAPÍTULO 11

# Telas que precisam de vários módulos

*Quando os dados estão separados, uma tela que mostra tudo vira um problema de arquitetura. Felizmente, é um problema conhecido.*


11.1 O desafio da leitura
11.2 API Composition
11.3 Backend for Frontend
11.4 CQRS
11.5 Modelos de leitura
11.6 Como escolher
11.7 Módulos de composição e a regra da dependência
11.8 O Dashboard
11.9 Resumo e exercícios


## 11.1 O desafio da leitura

Escrever em um módulo é simples: o módulo é dono do dado. Ler é mais difícil, porque as telas não respeitam fronteiras. Um dashboard, um relatório financeiro ou uma busca global querem dados de vários lugares ao mesmo tempo.

A tentação é abrir uma exceção: "só esse relatório faz `JOIN` em tudo". Essa exceção costuma ser o começo da erosão. Vale conhecer as alternativas.

## 11.2 API Composition

Na **composição de API**, um componente chama os contratos de cada módulo e monta a resposta em memória. É o padrão mais simples.

- **Vantagens:** nenhuma infraestrutura nova, dados sempre atuais, sem cópias.
- **Desvantagens:** várias consultas por requisição, e filtros que cruzam módulos ficam difíceis. Não dá para ordenar eficientemente por um campo de outro módulo quando há muitos registros.

## 11.3 Backend for Frontend

O **BFF** (*Backend for Frontend*) é uma variação: um componente feito sob medida para uma interface, que devolve exatamente o que uma tela precisa. Sam Newman descreveu esse padrão a partir da experiência na SoundCloud. Em um monolito modular, um módulo de composição faz esse papel, sem exigir um serviço separado.

## 11.4 CQRS

**CQRS** (*Command Query Responsibility Segregation*), popularizado por Greg Young, separa o modelo de escrita do modelo de leitura. A premissa é que as duas necessidades são diferentes: a escrita protege regras, e a leitura busca velocidade e formato prático.

CQRS tem níveis, e muitas pessoas associam a ideia apenas ao mais complexo:

| Nível | Como funciona | Complexidade |
|---|---|---|
| **1. Classes separadas** | Consultas e comandos em código diferente, mesmo banco | Baixa |
| **2. Consultas diretas** | Leituras usam projeções otimizadas, sem carregar entidades | Baixa |
| **3. Modelo de leitura** | Tabelas próprias, atualizadas por eventos | Média |
| **4. Banco separado** | Armazenamento específico para leitura | Alta |

## 11.5 Modelos de leitura

Um **modelo de leitura** (ou *read model*) é uma estrutura pensada para uma consulta: uma tabela desnormalizada, atualizada em reação a eventos. O dashboard deixa de calcular e passa a ler uma linha pronta.

- **Vantagens:** consultas muito rápidas e possibilidade de filtrar e ordenar à vontade.
- **Custos:** consistência eventual, reconstrução dos dados e mais código.

Faz sentido quando a composição fica lenta, ou quando você precisa de buscas e filtros cruzados que a composição não consegue atender.

## 11.6 Como escolher

| Situação | Solução recomendada |
|---|---|
| Resumo com poucos dados por módulo | Composição |
| Lista de um módulo enriquecida com nomes de outro | Busca em lote |
| Filtro por um campo de outro módulo | Busca de identificadores, depois filtro |
| Relatório pesado e histórico | Modelo de leitura ou banco analítico |
| Busca global em texto | Índice dedicado, como Elasticsearch ou busca do PostgreSQL |

> **Para lembrar**
>
> Comece com composição. Só avance quando houver uma medida de lentidão ou um requisito que ela não atende. Um modelo de leitura sem necessidade é consistência eventual gratuita.

## 11.7 Módulos de composição e a regra da dependência

Um módulo que só agrega dados tem um lugar especial no grafo. Ele depende de muitos, e **ninguém depende dele**. Isso é saudável, porque uma mudança nele não afeta nenhum outro módulo. O risco seria o contrário: se outros módulos começarem a depender da agregação, ele vira um centro de acoplamento.

CONSTRUINDO O LAÇOOS · PASSO 11

# O Dashboard

O dashboard do Laçoos mostra um resumo: dias restantes, progresso do checklist, situação financeira, convidados, parcelas próximas e atrasadas, e tarefas próximas. Os dados vêm de seis módulos.

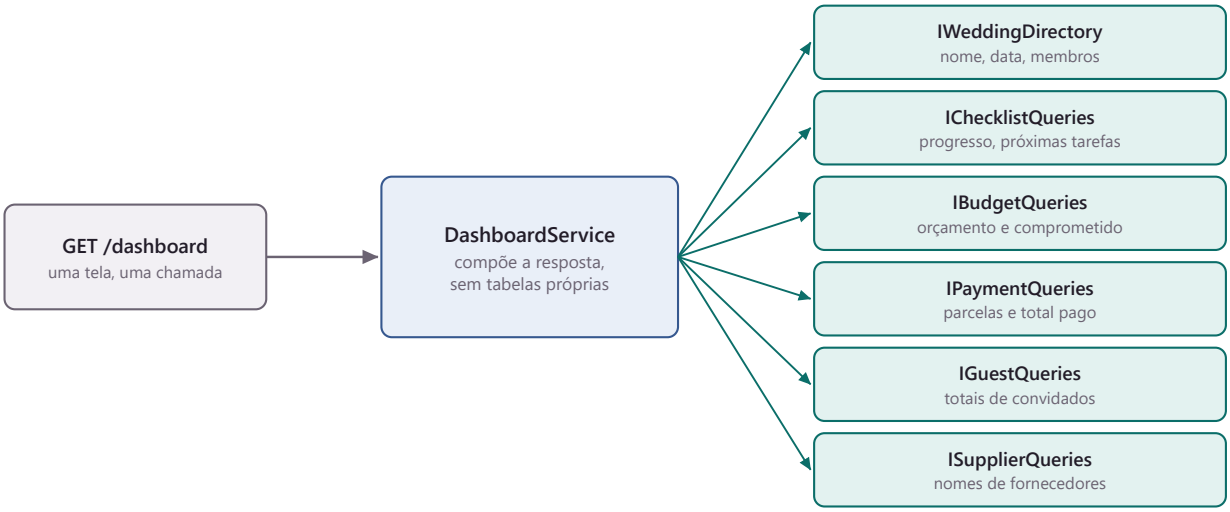



**Figura 11.1.** O Dashboard compõe a resposta chamando seis contratos, sem tabelas próprias.

## Um módulo em um arquivo

O módulo inteiro tem 103 linhas, em um único arquivo. Sem `Domain`, sem `Persistence`, sem migrations. Essa é a decisão local do capítulo 6 em ação: a estrutura se adapta ao problema.

**O serviço depende só de contratos** · src/Modules/Dashboard/Lacoos.Modules.Dashboard/DashboardModule.cs · linhas 43–51

```
/// <summary>Compõe os dados só pelos contratos públicos dos módulos. Não tem tabelas próprias.
</summary>
internal sealed class DashboardService(
    IWeddingDirectory weddings,
    IChecklistQueries checklist,
    IBudgetQueries budget,
    IPaymentQueries payments,
    IGuestQueries guests,
    ISupplierQueries suppliers,
    IClock clock)
```

A lista de dependências é o próprio desenho da Figura 11.1. O comentário declara a intenção: compor os dados **só pelos contratos públicos**.

## A composição

**Buscando os dados em cada módulo** · src/Modules/Dashboard/Lacoos.Modules.Dashboard/DashboardModule.cs · linhas 57–75

```
public async Task<DashboardResponse> GetAsync(CancellationToken ct)
{
    var today = clock.Today;
    var wedding = await weddings.GetCurrentAsync(ct);
    var summary = await checklist.GetSummaryAsync(ct);
    var financials = await budget.GetFinancialsAsync(ct);
    var overview = await payments.GetOverviewAsync(UpcomingInstallmentsLimit, ct);
    var guestTotals = await guests.GetTotalsAsync(ct);
    var tasks = await checklist.GetUpcomingTasksAsync(today.AddDays(TaskWindowDays),
UpcomingTasksLimit, ct);

    var supplierIds = overview.UpcomingInstallments.Concat(overview.OverdueInstallments)
        .Where(i => i.SupplierId is not null)
        .Select(i => i.SupplierId!.Value)
        .Distinct()
        .ToList();
    var supplierNames = await suppliers.GetNamesAsync(supplierIds, ct);

    var memberIds = tasks.Where(t => t.AssignedToMemberId is not null).Select(t =>
t.AssignedToMemberId!.Value).Distinct().ToList();
    var memberNames = await weddings.GetMemberNamesAsync(memberIds, ct);
```

Note a sequência em duas fases. Primeiro, os dados principais. Depois, os identificadores de fornecedores e membros que apareceram são reunidos, e os nomes são buscados **em lote**. São duas consultas adicionais, não uma por parcela.

Por que as chamadas não rodam em paralelo com `Task.WhenAll`? Porque os serviços de um mesmo módulo compartilham um `DbContext` com escopo de requisição, e um `DbContext` não é seguro para uso simultâneo. Para o volume de dados de um casamento, a execução em sequência é suficiente.

## O registro é mínimo

**Toda a porta de entrada do módulo** · src/Modules/Dashboard/Lacoos.Modules.Dashboard/DashboardModule.cs · linhas 95–103

```
public sealed class DashboardModule : IModule
{
    public void Register(IServiceCollection services, IConfiguration configuration) =>
        services.AddScoped<DashboardService>();

    public void MapEndpoints(IEndpointRouteBuilder app) =>
        app.MapWeddingGroup("Dashboard")
            .MapGet("/dashboard", (DashboardService service, CancellationToken ct) =>
service.GetAsync(ct));
}
```

Como `MapWeddingGroup` aplica a proteção, o Dashboard herda autenticação, papéis e isolamento sem nenhuma linha extra.

## Uma regra verificada

Um teste de arquitetura garante que o Dashboard continue sem persistência:

**O Dashboard não pode ter persistência** · tests/Lacoos.ArchitectureTests/ModuleBoundaryTests.cs · linhas 127–136

```
[Fact]
public void Dashboard_has_no_persistence()
{
    var dashboard = Modules["Dashboard"];

    Assert.DoesNotContain(dashboard.GetTypes(), type => typeof(DbContext).IsAssignableFrom(type));
    Assert.DoesNotContain(
        dashboard.GetReferencedAssemblies(),
        assembly => assembly.Name!.StartsWith("Microsoft.EntityFrameworkCore",
StringComparison.Ordinal));
}
```

Isso é mais do que um detalhe. Se alguém adicionar uma tabela de cache ao Dashboard, o teste falha, e a equipe é obrigada a discutir a decisão, em vez de ela passar despercebida.

## Filtro cruzado em Payments

Outro caso comum: buscar pagamentos pelo nome do fornecedor. A informação está em outro módulo. A solução é pedir os **identificadores** que combinam com o texto e filtrar localmente:

**Busca que cruza módulos sem JOIN** · src/Modules/Payments/Lacoos.Modules.Payments/Application/PaymentService.cs · linhas 90–98

```
if (Text.Clean(filter.Search) is { } search)
{
    var supplierIds = (await suppliers.SearchIdsAsync(search, ct)).ToHashSet();
    var itemIds = (await budget.SearchItemIdsAsync(search, ct)).ToHashSet();
    matches = matches.Where(p =>
        p.Description.Contains(search, StringComparison.OrdinalIgnoreCase)
        || (p.SupplierId is { } supplierId && supplierIds.Contains(supplierId))
        || (p.BudgetItemId is { } itemId && itemIds.Contains(itemId)));
}
```

### Os limites desta solução

O `PaymentService` carrega todos os pagamentos do casamento e filtra em memória. Para um casamento, com dezenas ou centenas de pagamentos, isso funciona muito bem, e permite usar a regra de status calculada. Em um sistema com milhões de registros por cliente, seria inviável, e o caminho passaria por paginação no banco e um modelo de leitura. Boas decisões de arquitetura dependem do tamanho real dos dados.

### Dois significados, de propósito

O Dashboard mostra `TotalPaid` de `IPaymentQueries`, que soma todos os pagamentos. A tela de orçamento soma só os pagamentos ligados a itens. Quem ler os dois números pode achar que há um erro. A diferença é correta e foi decidida no capítulo 4, mas precisa estar clara na interface para o usuário.

## 11.9 Resumo e exercícios

- Telas não respeitam fronteiras. A leitura entre módulos exige um padrão consciente.
- A composição de API é simples, atual e deve ser a primeira opção.
- CQRS tem níveis: os mais simples não exigem eventos nem outro banco.
- Modelos de leitura trocam velocidade por consistência eventual e mais código.
- Um módulo de composição deve ficar no topo do grafo: depende de muitos, e ninguém depende dele.

### Exercícios

1. A tela de pagamentos passa a precisar ordenar por nome do fornecedor, com paginação no banco e milhões de registros. Quais opções existem?
2. Por que é arriscado chamar vários contratos em paralelo com `Task.WhenAll` neste projeto?
3. Liste os prós e contras de transformar o Dashboard em um modelo de leitura alimentado por eventos.

PARTE IV

# Mantendo a arquitetura viva

Fronteira que ninguém verifica desaparece. Testes, regras automáticas e uma entrega bem feita mantêm o desenho de pé conforme o código cresce.

CAPÍTULO 12

# Testes em um monolito modular

*Módulos bem separados mudam o que vale a pena testar e onde. A pirâmide continua útil, mas pede ajustes.*


12.1 Para que servem os testes aqui
12.2 Formatos de suíte
12.3 O que testar em cada nível
12.4 Testes sociáveis e solitários
12.5 Banco real ou falso?
12.6 Dublês nas fronteiras certas
12.7 Testes de contrato
12.8 Testes por varredura
12.9 A suíte do Laçoos
12.10 Resumo e exercícios


## 12.1 Para que servem os testes aqui

Testes costumam ser descritos como uma forma de encontrar erros. Em um sistema modular, eles têm três funções adicionais:

- **Proteger as regras de negócio** contra regressões.
- **Permitir refatorar o interior** de um módulo com segurança.
- **Proteger a própria arquitetura**, assunto do próximo capítulo.

## 12.2 Formatos de suíte

A **pirâmide de testes**, popularizada por Mike Cohn, sugere muitos testes de unidade, alguns de integração e poucos de ponta a ponta. A ideia é preferir o que é rápido e barato.

Com o tempo, surgiram alternativas. O **favo de mel** (*honeycomb*), proposto por engenheiros do Spotify para microsserviços, dá mais peso a testes de integração, porque boa parte da complexidade fica nas interações. O **troféu**, de Kent C. Dodds, tem uma ideia parecida para o front-end.

Essas figuras divergem porque os sistemas divergem. Em uma API de negócio, muito do risco está em consultas, transações e permissões. Um teste de unidade com banco falso não vê nada disso. A regra prática é: **teste no nível em que o risco aparece**.

## 12.3 O que testar em cada nível

| Nível | Melhor para | Evite usar para |
|---|---|---|
| **Unidade** | Regras puras, cálculos, casos de borda | Consultas ao banco, fluxo HTTP |
| **Integração do módulo** | Casos de uso, consultas, transações | Combinações exaustivas de valores |
| **API completa** | Rotas, permissões, serialização, fluxos entre módulos | Toda variação de regra |

| Nível | Melhor para | Evite usar para |
|---|---|---|
| **Arquitetura** | Fronteiras, dependências, convenções | Comportamento |
| **Ponta a ponta com front-end** | Poucas jornadas críticas | Regras detalhadas |

## 12.4 Testes sociáveis e solitários

Martin Fowler descreve dois estilos de teste de unidade. O **solitário** substitui todas as dependências por dublês. O **sociável** deixa os objetos reais colaborarem, substituindo apenas o que é externo ou lento.

Um erro comum é usar *mocks* em tudo. O teste fica acoplado à implementação: você refatora o interior sem mudar o comportamento, e os testes quebram. Isso ataca justamente a liberdade que o módulo deveria oferecer.

> **Para lembrar**
>
> Teste pelo **contrato público** sempre que possível. Se os testes conhecem o interior, o interior deixa de ser livre para mudar.

## 12.5 Banco real ou falso?

Provedores em memória parecem convenientes, mas não se comportam como o banco real: não têm a mesma transação, não têm `FOR UPDATE` e tratam restrições de outra forma. O **Testcontainers** resolve isso: ele sobe um contêiner descartável do banco real durante os testes. Custa alguns segundos, e o resultado é confiável.

Recomendação: use **a mesma versão** do banco de produção nos testes.

## 12.6 Dublês nas fronteiras certas

Dublês continuam importantes, mas nos pontos certos. Substitua o que é **externo ao sistema**, como e-mail, relógio e serviços de terceiros. Evite substituir o que é seu e pode rodar de verdade, como o banco.

Módulos também podem ser testados isoladamente, com dublês dos contratos de outros módulos. Isso é especialmente útil em sistemas grandes, em que subir tudo fica lento.

## 12.7 Testes de contrato

Com microsserviços, testes de contrato como os do **Pact** verificam se provedor e consumidor continuam compatíveis. Em um monolito modular, o compilador já faz boa parte desse trabalho. O que ele não verifica é o **comportamento**. Por exemplo, "`GetPaidBySupplierAsync` não retorna

fornecedores sem pagamentos" está documentado em um comentário, e um teste de integração pode travar essa promessa.

## 12.8 Testes por varredura

Uma técnica pouco usada e muito poderosa é o teste que **descobre o que testar**. Em vez de listar as rotas à mão, ele lê do próprio sistema todas as rotas registradas e aplica a mesma regra a cada uma. Uma rota nova entra na verificação sem que ninguém precise lembrar. É o tipo de teste que protege **o futuro**.

CONSTRUINDO O LAÇOOS · PASSO 12

### A suíte do Laçoos

O projeto tem 236 casos de teste, em três projetos:

| Projeto | Casos | Tempo aproximado | Depende de |
|---|---|---|---|
| `Lacoos.UnitTests` | 53 | 2 segundos | Nada |
| `Lacoos.ArchitectureTests` | 64 | 3 segundos | Nada |
| `Lacoos.IntegrationTests` | 119 | 1 minuto | Docker |

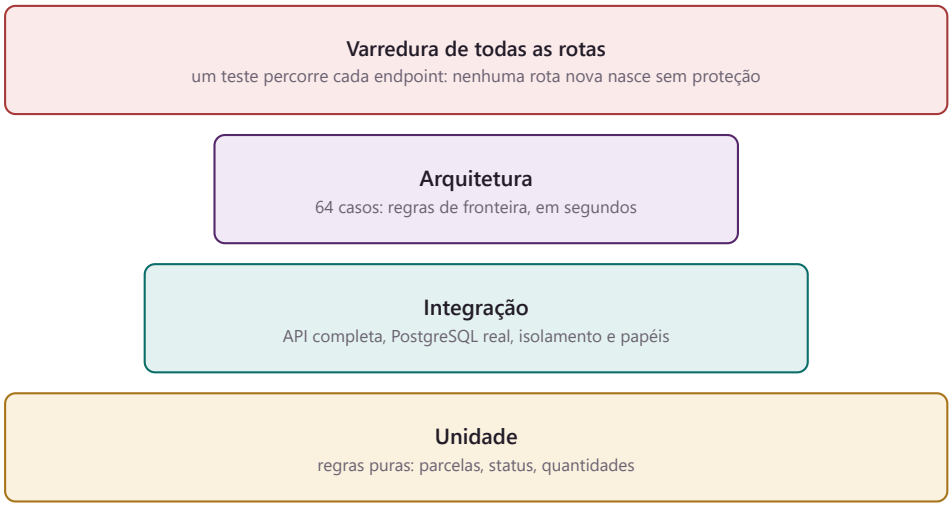



**Figura 12.1.** Os níveis de teste do Laçoos. A varredura de rotas é um tipo de teste de integração com papel especial.

### Testes de unidade

Só regras puras, como vimos no capítulo 6. Para testar classes internas, o projeto do módulo expõe seu interior ao projeto de testes. O benefício dos testes puros aparece em casos de borda, como o ano bissexto:

**Casos de borda de datas** · tests/Lacoos.UnitTests/PaymentRulesTests.cs · linhas 43–58

```
[Fact]
public void Due_dates_clamp_to_the_last_day_of_february_in_a_common_year()
{
    var dates = PaymentRules.MonthlyDueDates(new DateOnly(2027, 1, 31), 4);

    Assert.Equal(
        [new DateOnly(2027, 1, 31), new DateOnly(2027, 2, 28), new DateOnly(2027, 3, 31), new
DateOnly(2027, 4, 30)],
        dates);
}

[Fact]
public void Due_dates_clamp_to_february_29_in_a_leap_year()
{
    var dates = PaymentRules.MonthlyDueDates(new DateOnly(2028, 1, 31), 2);

    Assert.Equal(new DateOnly(2028, 2, 29), dates[1]);
```

## A infraestrutura de integração

Uma fixture sobe um PostgreSQL real, na mesma versão do `docker-compose.yml`, e uma instância completa da API:

**PostgreSQL real por meio do Testcontainers** · tests/Lacoos.IntegrationTests/Support/ApiFixture.cs · linhas 51–71

```
public sealed class ApiFixture : WebApplicationFactory<Program>, IAsyncLifetime
{
    public const string PostgresImage = "postgres:17.6-alpine";

    private readonly PostgreSqlContainer _database = new PostgreSqlBuilder(PostgresImage).Build();
    private readonly string _uploads = Path.Combine(Path.GetTempPath(), $"lacoos-tests-
{Guid.NewGuid():N}");

    public TestClock Clock { get; } = new();

    public CapturingEmailSender Emails { get; } = new();

    public string ConnectionString => _database.GetConnectionString();

    public async ValueTask InitializeAsync()
    {
        await _database.StartAsync();
        Environment.SetEnvironmentVariable("ConnectionStrings__Postgres",
_database.GetConnectionString());
        Environment.SetEnvironmentVariable("Auth__RateLimitPerMinute", "1000000");
        Environment.SetEnvironmentVariable("Auth__SessionDays", "36500");
        Environment.SetEnvironmentVariable("FileStorage__RootPath", _uploads);
    }
```

A API de teste é a mesma do sistema real, incluindo todos os módulos e migrations. Só duas portas são trocadas por dublês: o relógio e o e-mail.

**Dublês só para o que é externo** · tests/Lacoos.IntegrationTests/Support/ApiFixture.cs · linhas 86–96

```
protected override void ConfigureWebHost(IWebHostBuilder builder)
{
    builder.UseEnvironment("Development");
    builder.ConfigureServices(services =>
    {
        services.RemoveAll<IClock>();
        services.AddSingleton<IClock>(Clock);
        services.RemoveAll<IEmailSender>();
        services.AddSingleton<IEmailSender>(Emails);
    });
}
```

O `TestClock` permite testar parcelas vencidas sem esperar o tempo passar. O `CapturingEmailSender` guarda os e-mails, e os testes leem o token do convite direto da mensagem, como um usuário faria.

Repare no que **não** é substituído: nenhum contrato entre módulos. Os testes exercitam o sistema de ponta a ponta, sem mocks.

## Fluxos entre módulos

Testes pela API descrevem o comportamento do jeito que o usuário vê. Veja o dashboard: o teste cria dados por rotas de vários módulos e confere a composição.

**Um teste que atravessa quatro módulos** · tests/Lacoos.IntegrationTests/DashboardTests.cs · linhas 21–51

```
[Fact]
public async Task Aggregates_checklist_financial_and_guests()
{
    var (owner, w, _) = await NewOwner();
    await owner.Post($"{w}/checklist/items", new { title = "A", status = "done" });
    await owner.Post($"{w}/checklist/items", new { title = "B" });
    await owner.Post($"{w}/checklist/items", new { title = "C", status = "in_progress" });
    await owner.Post($"{w}/checklist/items", new { title = "D", status = "done" });
    var category = (await owner.Post($"{w}/budget/categories", new { name = "A", plannedAmount = 4000
})).Json.Id();
    await owner.Post($"{w}/budget/items", new { name = "Item", categoryId = category,
contractedAmount = 1000 });
    await owner.Post($"{w}/budget/items", new { name = "Cancelado", categoryId = category,
contractedAmount = 900, status = "cancelled" });
    await owner.Post($"{w}/guests", new { firstName = "A", lastName = "B", attendanceStatus =
"confirmed" });
    await owner.Post($"{w}/guests", new { firstName = "C", lastName = "D", attendanceStatus =
"declined" });
    await owner.Post($"{w}/guests", new { firstName = "E", lastName = "F" });

    var dashboard = await owner.Get($"{w}/dashboard");

    Assert.Equal(HttpStatusCode.OK, dashboard.Status);
    Assert.Equal(4, dashboard.Json.Int("checklist.total"));
    Assert.Equal(2, dashboard.Json.Int("checklist.done"));
    Assert.Equal(2, dashboard.Json.Int("checklist.open"));
    Assert.Equal(50, dashboard.Json.Dec("checklist.completionPercent"));
    Assert.Equal(4000, dashboard.Json.Dec("financial.totalBudget"));
    Assert.Equal(1000, dashboard.Json.Dec("financial.committed"));
    Assert.Equal(3000, dashboard.Json.Dec("financial.balance"));
    Assert.Equal(25, dashboard.Json.Dec("financial.committedPercent"));
    Assert.Equal(3, dashboard.Json.Int("guests.total"));
```

```
    Assert.Equal(1, dashboard.Json.Int("guests.confirmed"));
    Assert.Equal(1, dashboard.Json.Int("guests.pending"));
    Assert.Equal(1, dashboard.Json.Int("guests.declined"));
}
```

## Varredura de todas as rotas

Esse é provavelmente o teste mais valioso do projeto. Ele percorre as rotas de casamento registradas na aplicação e garante que cada uma esconda o casamento de quem não é membro:

**Todas as rotas escondem o casamento** · tests/Lacoos.IntegrationTests/AccessControlTests.cs · linhas 61–74

```
[Fact]
public async Task Every_wedding_route_hides_the_wedding_from_non_members()
{
    var (_, _, weddingId) = await NewOwner();
    var stranger = await Api.NewUser();

    foreach (var route in WeddingRoutes())
    {
        var response = await Call(stranger, route, weddingId);

        Assert.True(response.Status == HttpStatusCode.NotFound, $"{route.Method} {route.Pattern}
retornou {response.Code}");
        Assert.Equal("Casamento não encontrado.", response.Json.Str("detail"));
    }
}
```

Há variantes para autenticação e para papéis. Esta, por exemplo, garante que um `viewer` nunca consiga escrever:

**Nenhuma escrita para quem só pode ver** · tests/Lacoos.IntegrationTests/AccessControlTests.cs · linhas 76–88

```
[Fact]
public async Task Every_write_route_is_forbidden_for_viewers()
{
    var (owner, _, weddingId) = await NewOwner();
    var (viewer, _) = await Api.AddMember(owner, weddingId, WeddingRole.Viewer);

    foreach (var route in WeddingRoutes().Where(r => r.Method != "GET"))
    {
        var response = await Call(viewer, route, weddingId);

        Assert.True(response.Status == HttpStatusCode.Forbidden, $"{route.Method} {route.Pattern}
retornou {response.Code}");
    }
}
```

Imagine que alguém crie um módulo novo e esqueça `MapWeddingGroup`, ou registre uma rota fora do grupo seguro. Se a rota usar o padrão de caminho de casamento, o teste falha, mesmo que ninguém tenha escrito um teste para ela.

## Isolamento entre casamentos

O teste de isolamento simula um ataque real. Um usuário é dono de um casamento, e tenta acessar, **pela própria rota**, registros do casamento alheio:

**Tentando acessar dados de outro casamento** · tests/Lacoos.IntegrationTests/TenantIsolationTests.cs · linhas 10–16, 34–38, 62–70

```
[Fact]
public async Task Records_of_another_wedding_are_not_found_through_your_own_wedding()
{
    var (a, wa, _) = await NewOwner();
    var (b, wb, _) = await NewOwner();

    var task = await a.Post($"{wa}/checklist/items", new { title = "Segredo" });
// ...
    var attempts = new Dictionary<string, Task<ApiResponse>>
    {
        ["GET tarefa"] = b.Get($"{wb}/checklist/items/{task.Json.Id()}"),
        ["PUT tarefa"] = b.Put($"{wb}/checklist/items/{task.Json.Id()}", new { title = "Invadido" }),
        ["DELETE tarefa"] = b.Delete($"{wb}/checklist/items/{task.Json.Id()}"),
// ...
    };

    foreach (var (name, attempt) in attempts)
    {
        var response = await attempt;
        Assert.True(response.Status == HttpStatusCode.NotFound, $"{name} retornou {response.Code}:
{response.Text}");
    }

    Assert.Equal("Segredo", (await a.Get($"
{wa}/checklist/items/{task.Json.Id()}")).Json.Str("title"));
```

Esse caso é mais sutil do que acessar a rota de outro casamento, porque o usuário **tem permissão** na rota. O que o protege é o filtro global. O teste percorre 26 tentativas em módulos diferentes, com leituras e escritas.

> **O que ainda não está coberto**
>
> Não há testes de carga, e os testes de integração usam o mesmo banco para todos os casos, sem limpar dados entre eles. Eles funcionam porque cada teste cria um usuário e um casamento próprios. Isso é rápido, mas exige cuidado com verificações globais. Também não há testes automatizados de ponta a ponta com o front-end em Angular.

## 12.10 Resumo e exercícios

- Teste no nível em que o risco aparece: regras puras em unidade, comportamento e segurança em integração.
- Prefira testar pelo contrato público, para que o interior continue livre.
- Use o banco real com Testcontainers. Substitua apenas o que é externo.
- Testes por varredura protegem o que ainda nem foi escrito.
- Em sistemas multi-tenant, teste explicitamente o acesso a registros de outro cliente.

## Exercícios

1. Quantas regras de segurança do seu projeto dependem de alguém lembrar de escrever um teste? Qual poderia virar uma varredura?
2. Por que um banco em memória não conseguiria testar a proteção de `SELECT FOR UPDATE` contra baixas simultâneas?
3. Escreva um teste de varredura que verifique se toda rota de escrita exige o cabeçalho CSRF.

CAPÍTULO 13

# Arquitetura como código

*Uma regra de arquitetura que só existe em um documento é uma sugestão. Quando vira teste, passa a ser uma lei.*


13.1 A erosão
13.2 Funções de aptidão
13.3 Camadas de defesa
13.4 Ferramentas do mercado
13.5 Como adotar em um sistema existente
13.6 Documentação viva
13.7 As regras do Laçoos
13.8 Resumo e exercícios


## 13.1 A erosão

Toda arquitetura sofre **erosão**: a distância entre o desenho pretendido e o código real aumenta aos poucos. Não acontece por má-fé. Acontece porque, sob pressão, alguém adiciona uma referência "só desta vez", e nada impede.

Em microsserviços, a rede cria uma barreira física. Em um monolito modular, a fronteira é lógica. A verificação automática é o que substitui essa barreira. **Sem isso, um monolito modular tende a virar um monolito comum.**

> **Para lembrar**
>
> A verificação automática não é um extra do monolito modular. Ela é **parte da definição**. Sem ela, você tem apenas um monolito organizado hoje.

## 13.2 Funções de aptidão

O livro *Building Evolutionary Architectures*, de Neal Ford, Rebecca Parsons e Patrick Kua, chama esse tipo de verificação de **função de aptidão arquitetural** (*fitness function*). O termo vem da computação evolutiva: uma medida objetiva de quanto o sistema atende a uma característica desejada.

Funções de aptidão podem medir muitas coisas: dependências, tempo de resposta, cobertura, vulnerabilidades ou complexidade. Rodam no pipeline de integração. Quando uma regra quebra, o build falha, e a discussão acontece antes de a mudança entrar no sistema.

## 13.3 Camadas de defesa

A proteção de fronteiras funciona melhor em camadas, cada uma cobrindo os limites da anterior:

| Camada | O que detecta | Quando |
|---|---|---|
| **Compilador** (`internal`, referências) | Acesso a tipos ocultos | Ao compilar |

| Camada | O que detecta | Quando |
|---|---|---|
| **Analisadores** (Roslyn, `BannedApiAnalyzers`) | Uso de APIs proibidas | Ao compilar |
| **Testes de arquitetura** | Dependências, nomes, convenções | No build ou no CI |
| **Revisão de código** | Decisões que uma regra não capta | No pull request |
| **Dono de código** (`CODEOWNERS`) | Quem precisa aprovar mudanças em contratos | No pull request |

## 13.4 Ferramentas do mercado

### .NET

- **NetArchTest.** API fluente e leve, fácil de começar. É a usada no Laçoos.
- **ArchUnitNET.** Porta do ArchUnit para .NET, com regras mais expressivas e licença Apache 2.0.

**Regra equivalente com ArchUnitNET** · Exemplo ilustrativo: não faz parte do repositório

```
private static readonly Architecture Architecture =
    new ArchLoader().LoadAssemblies(typeof(BudgetModule).Assembly).Build();

[Fact]
public void Budget_nao_acessa_o_interior_de_Payments() =>
    Types().That().ResideInNamespace("Lacoos.Modules.Budget")
        .Should().NotDependOnAny(Types().That().ResideInNamespace("Lacoos.Modules.Payments.Domain"))
        .Check(Architecture);
```

### Java

**ArchUnit** é a referência no ecossistema Java. O **Spring Modulith** vai além: ele entende os pacotes como módulos e oferece verificação com uma linha, documentação gerada e um registro de publicação de eventos.

**Verificação da estrutura com Spring Modulith** · Exemplo ilustrativo: não faz parte do repositório

```
@Test
void verifiesModularStructure() {
    ApplicationModules.of(Application.class).verify();
}
```

### Ruby

A Shopify criou o **Packwerk** para definir pacotes e suas dependências em uma base Rails. O texto *A Packwerk Retrospective*, de 2024, traz lições valiosas sobre esse tipo de ferramenta:

- As verificações de **privacidade** foram removidas na versão 3.0, porque transformavam a ferramenta em algo que ela não deveria ser: um desenhador de API.
- Desenvolvedores agrupavam código pelo **nome**, e não pelo comportamento em tempo de execução.
- A ferramenta impõe o grafo que você definir, mesmo que seja um grafo ruim.

- A lição final da equipe é que **código rodando vale mais do que métricas**: um pacote com zero violações ainda pode falhar em execução.

> **Por que esse retrospecto importa**
>
> Ferramentas verificam regras, mas não dizem se as regras são boas. A Shopify aprendeu isso em uma das maiores bases Rails do mundo. Medir violações é útil, mas não substitui pensar sobre fronteiras.

## 13.5 Como adotar em um sistema existente

Em um projeto legado, ligar todas as regras de uma vez gera milhares de erros. Duas técnicas ajudam:

- **Lista de exceções.** Registre as violações atuais e proíba apenas as novas. É o que o Packwerk faz com os arquivos de pendências.
- **Catraca** (*ratchet*). A meta só pode melhorar: o número de violações pode cair, nunca subir.

## 13.6 Documentação viva

Regras em teste são uma documentação que não envelhece. Elas se somam a duas outras práticas:

- **ADRs** registram o porquê.
- **Diagramas gerados do código** mostram o estado real. O modelo **C4**, de Simon Brown, é um padrão muito usado para descrever sistemas em níveis, e o Spring Modulith gera diagramas assim automaticamente.

CONSTRUINDO O LAÇOOS · PASSO 13

### As regras do Laçoos

O projeto `Lacoos.ArchitectureTests` tem um único arquivo com 199 linhas, que geram 64 casos de teste. Ele declara os módulos uma vez e aplica as regras a cada um:

**Os módulos e as camadas internas** · tests/Lacoos.ArchitectureTests/ModuleBoundaryTests.cs · linhas 22–41

```
public sealed class ModuleBoundaryTests
{
    private static readonly Dictionary<string, Assembly> Modules = new()
    {
        ["Identity"] = typeof(IdentityModule).Assembly,
        ["Weddings"] = typeof(WeddingsModule).Assembly,
        ["Dashboard"] = typeof(DashboardModule).Assembly,
        ["Checklist"] = typeof(ChecklistModule).Assembly,
        ["Budget"] = typeof(BudgetModule).Assembly,
        ["Guests"] = typeof(GuestsModule).Assembly,
        ["Suppliers"] = typeof(SuppliersModule).Assembly,
        ["Payments"] = typeof(PaymentsModule).Assembly,
        ["Gifts"] = typeof(GiftsModule).Assembly,
    };

    private static readonly string[] InternalLayers = ["Domain", "Application", "Persistence",
"Endpoints"];

    public static TheoryData<string> ModuleNames => new(Modules.Keys);

    public static TheoryData<string> DataModuleNames => new(Modules.Keys.Where(m => m !=
"Dashboard"));
```

## Regra 1: só contratos entre módulos

A primeira regra olha o binário compilado:

**Nenhuma referência a interiores** · tests/Lacoos.ArchitectureTests/ModuleBoundaryTests.cs · linhas 43–54

```
[Theory]
[MemberData(nameof(ModuleNames))]
public void Module_only_references_contracts_of_other_modules(string module)
{
    var offending = Modules[module].GetReferencedAssemblies()
        .Select(a => a.Name!)
        .Where(name => name.StartsWith("Lacoos.Modules.", StringComparison.Ordinal))
        .Where(name => !name.EndsWith(".Contracts", StringComparison.Ordinal))
        .ToList();

    Assert.Empty(offending);
}
```

Existe uma regra irmã que lê o `.csproj` como XML. Por que as duas? Porque o compilador de .NET remove referências que não são usadas. Um `.csproj` com uma referência proibida, mas ainda sem uso, passaria pela primeira regra. A segunda pega isso antes de o problema crescer.

## Regra 2: nenhum tipo do interior alheio

Esta regra usa o NetArchTest para analisar os tipos, caso algo escape da regra anterior:

**Dependência proibida em nível de tipo** · tests/Lacoos.ArchitectureTests/ModuleBoundaryTests.cs · linhas 71–83

```
[Theory]
[MemberData(nameof(ModuleNames))]
public void Module_does_not_depend_on_the_interior_of_other_modules(string module)
{
    var forbidden = Modules.Keys
        .Where(other => other != module)
        .SelectMany(other => InternalLayers.Select(layer => $"Lacoos.Modules.{other}.{layer}"))
        .ToArray();

    var result =
Types.InAssembly(Modules[module]).ShouldNot().HaveDependencyOnAny(forbidden).GetResult();

    Assert.True(result.IsSuccessful, Failures(result));
}
```

## Regra 3: só a porta de entrada é pública

Esta é a mais simples, e talvez a mais poderosa. Exige que a lista de tipos públicos do módulo seja **exatamente** o `XModule`:

**Interior oculto** · tests/Lacoos.ArchitectureTests/ModuleBoundaryTests.cs · linhas 85–95

```
[Theory]
[MemberData(nameof(ModuleNames))]
public void Module_exposes_only_its_entry_point(string module)
{
    var exposed = Modules[module].GetExportedTypes()
        .Where(type => !typeof(Migration).IsAssignableFrom(type))
        .Select(type => type.Name)
        .ToList();

    Assert.Equal([$"{module}Module"], exposed);
}
```

A exceção para `Migration` existe porque o EF Core gera as migrations como públicas. Sem esta regra, uma única palavra `public` esquecida abriria um buraco. Com ela, o buraco quebra o build.

## Regra 4: contratos limpos

Um projeto de contrato não pode depender de nada interno, nem do EF Core:

**Contratos sem dependências** · tests/Lacoos.ArchitectureTests/ModuleBoundaryTests.cs · linhas 97–107

```
[Theory]
[MemberData(nameof(DataModuleNames))]
public void Contracts_do_not_depend_on_persistence_or_on_module_interiors(string module)
{
    var contracts = Assembly.Load($"Lacoos.Modules.{module}.Contracts");

    var references = contracts.GetReferencedAssemblies().Select(a => a.Name!).ToList();

    Assert.DoesNotContain(references, name => name.StartsWith("Lacoos.", StringComparison.Ordinal));
    Assert.DoesNotContain(references, name => name.StartsWith("Microsoft.EntityFrameworkCore",
StringComparison.Ordinal));
}
```

## Regra 5: dados isolados

Cada módulo com dados precisa ter exatamente um contexto, com o schema de nome igual ao do módulo:

**Um contexto, um schema** · tests/Lacoos.ArchitectureTests/ModuleBoundaryTests.cs · linhas 109–118

```
[Theory]
[MemberData(nameof(DataModuleNames))]
public void Data_module_has_a_single_context_with_its_own_schema(string module)
{
    var contexts = Modules[module].GetTypes().Where(IsModuleContext).ToList();

    var context = Assert.Single(contexts);
    var schema = context.GetField("SchemaName")?.GetRawConstantValue();
    Assert.Equal(module.ToLowerInvariant(), schema);
}
```

E as migrations não podem apontar para outros schemas. Como o EF Core registra isso no código gerado, uma expressão regular basta:

**Nenhuma FK para outro schema** · tests/Lacoos.ArchitectureTests/ModuleBoundaryTests.cs · linhas 158–182

```
[Fact]
public void Migrations_do_not_create_foreign_keys_to_other_schemas()
{
    var root = Path.Combine(RepositoryRoot(), "src", "Modules");
    var offending = new List<string>();

    foreach (var (module, _) in Modules)
    {
        var folder = Path.Combine(root, module, $"Lacoos.Modules.{module}", "Persistence",
"Migrations");
        if (!Directory.Exists(folder))
        {
            continue;
        }

        foreach (var file in Directory.GetFiles(folder, "*Initial.cs"))
        {
            var principals = Regex.Matches(File.ReadAllText(file), "principalSchema: \"(?<schema>[a-
z_]+)\"")
                .Select(m => m.Groups["schema"].Value)
                .Where(schema => schema != module.ToLowerInvariant());
            offending.AddRange(principals.Select(schema => $"{module} -> {schema}"));
        }
    }

    Assert.Empty(offending);
}
```

Essa regra lê o arquivo `*Initial.cs` de cada módulo. Quando um módulo ganhar uma segunda migration, o filtro precisará ficar mais amplo. É o tipo de detalhe que vale anotar: **a regra também é código**, e tem manutenção.

## Regra 6: o host e o kernel

**Host sem acesso a interiores e kernel sem conhecer módulos** · tests/Lacoos.ArchitectureTests/ModuleBoundaryTests.cs · linhas 138–156

```
[Fact]
public void Host_only_uses_module_entry_points()
{
    var forbidden = Modules.Keys
        .SelectMany(module => InternalLayers.Select(layer => $"Lacoos.Modules.{module}.{layer}"))
        .ToArray();

    var result =
Types.InAssembly(typeof(Program).Assembly).ShouldNot().HaveDependencyOnAny(forbidden).GetResult();

    Assert.True(result.IsSuccessful, Failures(result));
}

[Fact]
public void Shared_kernel_does_not_know_the_modules()
{
    var result =
Types.InAssembly(typeof(IModule).Assembly).ShouldNot().HaveDependencyOn("Lacoos.Modules").GetResult()
;

    Assert.True(result.IsSuccessful, Failures(result));
}
```

Essa última regra protege a direção do grafo inteiro. Se o kernel pudesse conhecer um módulo, tudo poderia depender de tudo.

## As regras funcionam mesmo?

Um teste que nunca falha pode estar quebrado. Por isso, durante a construção do projeto, fizemos a prova: uma **violação temporária** foi injetada de propósito, com um módulo acessando o interior de outro. Quatro regras falharam, como esperado. Depois, a violação foi revertida.

Vale repetir esse exercício sempre que você criar uma regra: **veja o teste falhar** ao menos uma vez.

## O mapa completo das regras

| Regra | Casos | Protege |
|---|---|---|
| Só contratos (binário) | 9 | Referências entre módulos |
| Só contratos ( `.csproj` ) | 9 | Referências ainda sem uso |
| Sem interior alheio (tipos) | 9 | Uso de tipos de outros módulos |
| Só a porta de entrada é pública | 9 | Encapsulamento |
| Contratos sem dependências | 8 | Pureza dos contratos |
| Um contexto, um schema | 8 | Posse dos dados |
| Migrations próprias | 8 | Independência de migrations |
| Dashboard sem persistência | 1 | Módulo de composição |
| Host só usa portas de entrada | 1 | Host fino |

| Regra | Casos | Protege |
|---|---|---|
| Kernel não conhece módulos | 1 | Direção do grafo |
| Sem FK entre schemas | 1 | Referência por identificador |

> **O que as regras não pegam**
>
> As regras não detectam consultas SQL escritas à mão apontando para o schema de outro módulo, e não impedem ciclos entre contratos: os cinco pares do capítulo 7 passam por todas elas. Também não há limite para o tamanho do contrato. Uma boa próxima regra seria uma **lista explícita das dependências permitidas entre módulos**, que tornaria um novo ciclo uma decisão consciente, e não um acidente.

## 13.8 Resumo e exercícios

- Sem verificação automática, uma fronteira lógica erode com o tempo.
- Funções de aptidão transformam regras de arquitetura em falhas de build.
- Proteja em camadas: compilador, analisadores, testes e revisão.
- Ferramentas verificam regras, mas não dizem se as regras são boas.
- Sempre veja a regra falhar ao menos uma vez.

### Exercícios

1. Escreva uma regra de arquitetura com NetArchTest que proíba o uso de `DateTime.Now` nos módulos.
2. Crie uma lista das dependências permitidas entre os módulos do Laçoos e um teste que a verifique. O que acontece quando alguém adiciona uma nova?
3. Coloque `public` em uma classe interna de um módulo e rode os testes de arquitetura. Qual regra falha, e a mensagem ajuda a entender o motivo?

CAPÍTULO 14

# Entrega e operação

*A maior vantagem prática de um monolito modular aparece aqui: um artefato, um pipeline e um lugar para olhar quando algo der errado.*


14.1 O que muda na operação
14.2 Contêineres e imagens
14.3 Configuração: os doze fatores
14.4 Migrations em produção
14.5 Health checks
14.6 Observabilidade
14.7 Escalando um monolito
14.8 CI/CD
14.9 Colocando o Laçoos em contêineres
14.10 Resumo e exercícios


## 14.1 O que muda na operação

Em microsserviços, operar é uma disciplina à parte. Em um monolito modular, a operação continua simples, mas não é automática. Alguns cuidados valem a pena:

- Um artefato, gerado uma vez e promovido entre ambientes.
- Configuração fora do código.
- Migrations como uma etapa própria.
- Saúde da aplicação verificável.
- Observabilidade que preserve a noção de módulo.

## 14.2 Contêineres e imagens

Um contêiner empacota a aplicação e suas dependências. Três práticas são padrão de mercado:

- **Build em múltiplos estágios.** Uma imagem grande compila, e uma imagem pequena roda. O SDK não vai para produção.
- **Usuário sem privilégios.** Um processo invadido dentro do contêiner não deve ser `root`.
- **Versões fixas de imagens.** `postgres:latest` pode mudar sem aviso e quebrar a compatibilidade.

## 14.3 Configuração: os doze fatores

O manifesto **Twelve-Factor App**, publicado pela Heroku, reúne boas práticas de aplicações em nuvem. Uma das mais conhecidas é **guardar a configuração no ambiente**: tudo o que muda entre ambientes fica fora do código, em especial os segredos.

> **Segredos no repositório**
>
> Uma senha em um commit continua no histórico mesmo depois de removida. Se isso acontecer, o caminho seguro é **trocar a senha**. Mantenha um `.env.example` com valores falsos e deixe o `.env` real fora do Git.

## 14.4 Migrations em produção

Há duas formas de aplicar migrations:

| Abordagem | Vantagem | Risco |
|---|---|---|
| **Na inicialização da aplicação** | Simples | Várias réplicas migrando juntas, e a aplicação precisa de permissões altas |
| **Como etapa do pipeline** | Controle, log e falha clara | Um passo a mais |

Em produção, prefira a etapa separada.

### Compatibilidade entre versões

Durante uma implantação, a versão antiga e a nova podem rodar ao mesmo tempo, e ambas acessam o mesmo banco. Por isso, mudanças destrutivas se fazem em etapas, no padrão **expandir e contrair** (*expand and contract*):

1. **Expandir:** adicione a coluna nova, mantendo a antiga.
2. **Migrar:** a aplicação grava nas duas, e os dados são copiados.
3. **Contrair:** em uma entrega posterior, remova a antiga.

## 14.5 Health checks

Uma aplicação pode estar rodando e ainda assim inútil: sem banco, por exemplo. Orquestradores usam dois tipos de verificação:

- **Liveness:** o processo está vivo? Se não, reinicie.
- **Readiness:** ele consegue atender? Se não, não envie tráfego.

## 14.6 Observabilidade

**Observabilidade** é a capacidade de entender o que acontece por dentro olhando o que o sistema emite: logs, métricas e rastreamentos. O padrão aberto de mercado é o **OpenTelemetry**.

Em um monolito modular, o desafio é **não perder a noção de módulo**. Alguns recursos ajudam:

- Colocar o nome do módulo nos logs e nos spans.
- Medir latência e erros por módulo.

- Criar alertas com um dono por módulo.

Com isso, você responde "qual módulo está lento?" e tem base concreta para a decisão do capítulo 15.

## 14.7 Escalando um monolito

Um monolito escala melhor do que se costuma pensar:

- **Escala horizontal.** Várias réplicas atrás de um balanceador, desde que não guardem estado local.
- **Papéis por configuração.** O mesmo binário pode rodar em modos diferentes, um atendendo a API, outro processando tarefas em segundo plano. Isso separa cargas sem separar o código.
- **Banco.** Réplicas de leitura, cache e índices resolvem muita coisa.

Esse tipo de aplicação tem um ponto importante a observar: arquivos no disco local e sessões em memória impedem a escala horizontal. Decisões simples no início têm efeito no fim.

## 14.8 CI/CD

**Integração contínua** significa integrar pequenas mudanças com frequência, com testes automáticos. **Entrega contínua** significa que tudo o que passa nos testes está pronto para implantação. O pipeline típico é:

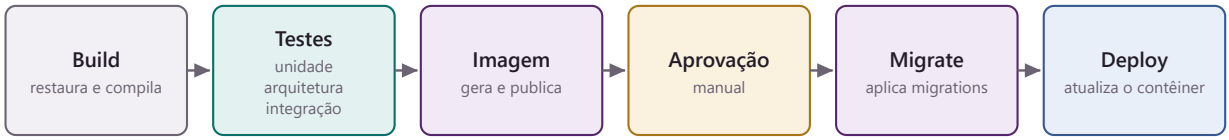



Figura 14.1. Um pipeline para o Laçoos. O mesmo artefato percorre todas as etapas depois de construído.

Regras práticas:

- **Construa uma vez.** A imagem testada é a imagem que sobe.
- **Falhe cedo.** Os testes de arquitetura levam segundos, então rodam primeiro.
- **Identifique cada versão.** Use o hash do commit como tag da imagem.
- **Tenha um caminho de volta.** Saiba como reverter antes de precisar.

### Monorepo e testes seletivos

Em um monolito grande, rodar tudo a cada mudança pode ficar lento. Como os módulos têm dependências explícitas, dá para rodar apenas os testes dos módulos afetados. Para um sistema com um minuto de testes, isso ainda não é necessário.

CONSTRUINDO O LAÇOOS · PASSO 14

# Colocando o Laçoos em contêineres

## O Dockerfile

**Build em múltiplos estágios** · Dockerfile

```
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /source

COPY Directory.Build.props Directory.Packages.props global.json ./
COPY src/ src/

RUN --mount=type=cache,id=lacoos-nuget,target=/root/.nuget/packages \
    dotnet publish src/Host/Lacoos.Api/Lacoos.Api.csproj -c Release -o /app -p:UseAppHost=false

FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS runtime
WORKDIR /app

# A pasta nasce com o dono certo: o volume nomeado herda essa permissão na primeira montagem.
RUN mkdir -p /app/data/uploads && chown -R $APP_UID:$APP_UID /app/data

COPY --from=build /app .

ENV ASPNETCORE_HTTP_PORTS=8080 \
    FileStorage__RootPath=/app/data/uploads

USER $APP_UID
EXPOSE 8080

ENTRYPOINT ["dotnet", "Lacoos.Api.dll"]
```

Cada decisão aqui tem um motivo:

- **Dois estágios.** A imagem final usa `aspnet:10.0`, sem o SDK.
- `--mount=type=cache` mantém os pacotes NuGet entre builds, deixando as próximas compilações mais rápidas.
- `USER $APP_UID` roda o processo sem `root`. A imagem oficial do .NET já traz esse usuário.
- **O `chown` antes do `USER`** resolve uma armadilha: um volume nomeado herda a permissão da pasta na primeira montagem. Sem isso, a aplicação não conseguiria gravar uploads.

## O Compose

O Compose define o ambiente completo para desenvolvimento:

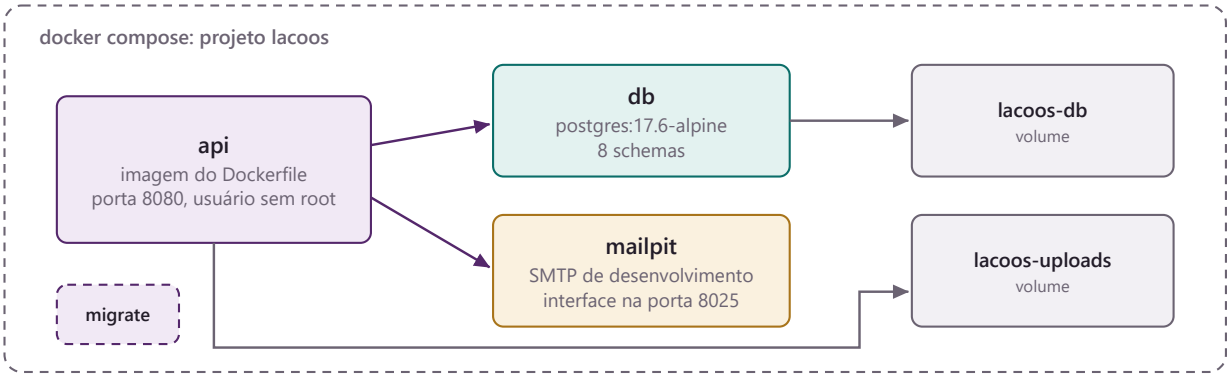



**Figura 14.2.** Os serviços do Compose. O serviço migrate só roda quando solicitado.

**O banco, com versão fixa e verificação de saúde** · docker-compose.yml · linhas 7–22

```
services:
  db:
    image: postgres:17.6-alpine
    environment:
      POSTGRES_DB: ${LACOOS_DB_NAME:?copie .env.example para .env}
      POSTGRES_USER: ${LACOOS_DB_USER:?copie .env.example para .env}
      POSTGRES_PASSWORD: ${LACOOS_DB_PASSWORD:?copie .env.example para .env}
    ports:
      - "${LACOOS_DB_PORT:-5432}:5432"
    volumes:
      - lacoos-db:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U $${POSTGRES_USER} -d $${POSTGRES_DB}"]
      interval: 5s
      timeout: 3s
      retries: 12
```

Três detalhes importam:

- A sintaxe `${VAR:?mensagem}` faz o Compose **falhar com uma mensagem clara** se o `.env` estiver faltando, em vez de iniciar com senha vazia.
- As variáveis têm o prefixo `LACOOS_DB_`. Isso nasceu de um problema real: uma variável global `POSTGRES_USER` de outro projeto, na máquina de desenvolvimento, sobrepunha o `.env`, porque variáveis do ambiente têm prioridade. O prefixo evita conflitos.
- A porta do host é configurável, porque outro PostgreSQL pode estar usando a 5432.

A API só inicia depois que o banco estiver saudável:

**Ordem de inicialização garantida pelo health check** · docker-compose.yml · linhas 30–36

```
api:
  build: .
  depends_on:
    db:
      condition: service_healthy
    mailpit:
      condition: service_started
```

## Configuração sem segredos

Nenhuma senha aparece no código ou no Compose. O .NET mapeia variáveis de ambiente para a configuração, e `__` vira `:`. Assim, `ConnectionStrings__Postgres` vira `ConnectionStrings:Postgres`.

Se a variável não existir, a aplicação falha ao iniciar com uma mensagem direta. Falhar cedo é melhor do que falhar na primeira requisição.

## Migrations como etapa explícita

Lembra do argumento `migrate` do capítulo 5? O Compose o expõe em um perfil separado:

**Migrations como um passo independente** · docker-compose.yml · linhas 54–63

```yaml
# Passo explícito para produção: docker compose --profile migrate run --rm migrate
migrate:
  build: .
  profiles: ["migrate"]
  command: ["migrate"]
  depends_on:
    db:
      condition: service_healthy
  environment:
    <<: *api-environment
```

Aplicando migrations de forma explícita

```
docker compose --profile migrate run --rm migrate
```

É a **mesma imagem** da API, que aplica as migrations e encerra. Em desenvolvimento, `Database:MigrateOnStartup` é `true` por conveniência. Em produção, a opção padrão é `false`, e o pipeline roda esse comando antes de atualizar a aplicação.

## Health check

O endpoint `/health` testa de verdade a conexão com o banco:

**Verificação de saúde** · src/Host/Lacoos.Api/Infrastructure/PostgresHealthCheck.cs

```
using Lacoos.SharedKernel.Persistence;
using Microsoft.Extensions.Diagnostics.HealthChecks;
using Npgsql;

namespace Lacoos.Api.Infrastructure;

internal sealed class PostgresHealthCheck(IConfiguration configuration) : IHealthCheck
{
    public async Task<HealthCheckResult> CheckHealthAsync(HealthCheckContext context,
CancellationToken ct = default)
    {
        try
        {
            await using var connection = new
NpgsqlConnection(configuration.GetConnectionString(ModulePersistence.ConnectionStringName));
            await connection.OpenAsync(ct);
            await using var command = new NpgsqlCommand("SELECT 1", connection);
            await command.ExecuteScalarAsync(ct);
```

```
            return HealthCheckResult.Healthy();
        }
        catch (Exception exception) when (exception is NpgsqlException or TimeoutException or
InvalidOperationException)
        {
            return HealthCheckResult.Unhealthy("Banco de dados indisponível.", exception);
        }
    }
}
```

## O pipeline planejado

O pipeline ainda não faz parte do repositório. Um exemplo do formato, no Azure Pipelines, ficaria assim:

**azure-pipelines.yml (esboço)** · Exemplo ilustrativo: não faz parte do repositório

```yaml
trigger:
  branches:
    include: [main]

stages:
  - stage: Test
    jobs:
      - job: Test
        pool: { vmImage: ubuntu-latest }
        steps:
          - task: UseDotNet@2
            inputs: { useGlobalJson: true }
          - script: dotnet test --solution LacoosApi.sln
            displayName: Testes de unidade, arquitetura e integração

  - stage: Image
    dependsOn: Test
    jobs:
      - job: Build
        pool: { vmImage: ubuntu-latest }
        steps:
          - script: docker build -t lacoos-api:$(Build.SourceVersion) .
            displayName: Gera a imagem
```

O agente de build do Azure Pipelines tem Docker, por isso os testes com Testcontainers funcionam lá. Observe como o monolito modular simplifica: **uma** imagem, **um** pipeline, para nove módulos.

> **Pendências para produção**
>
> O Laçoos foi validado em contêineres locais, mas não em produção. Faltam alguns pontos: uploads ficam em disco local, o que impede várias réplicas e exige um `IFileStorage` para armazenamento em nuvem; atrás de um proxy HTTPS é preciso tratar os *forwarded headers*, ou o limite por IP e os cookies seguros não funcionarão; e não há métricas nem rastreamento por módulo. A boa notícia é que cada item é uma mudança pequena e localizada, graças às portas do capítulo 10.

## 14.10 Resumo e exercícios

- Um monolito modular tem um artefato, um pipeline e operação simples.
- Use imagens em múltiplos estágios, usuário sem privilégios e versões fixas.
- Configuração e segredos ficam no ambiente, nunca no repositório.
- Em produção, aplique migrations como etapa separada e use expandir e contrair.
- Mantenha a noção de módulo nos logs e nas métricas.
- Um monolito escala horizontalmente, desde que não tenha estado local.

### Exercícios

1. Qual seria o plano para renomear uma coluna muito usada sem tempo de indisponibilidade, usando expandir e contrair?
2. Por que a imagem que passa nos testes precisa ser exatamente a que vai para produção?
3. Implemente um `IFileStorage` fictício para nuvem. Quais arquivos do repositório mudariam? Algum módulo seria afetado?

PARTE V

# Evolução e mercado

Como extrair um módulo quando isso fizer sentido, o que empresas reais aprenderam e uma visão completa de tudo o que foi aplicado no Laçoos.

CAPÍTULO 15

# Evoluindo: quando e como extrair um módulo

*Um monolito modular não é um degrau obrigatório para microsserviços. Pode ser o destino final. Mas, quando extrair fizer sentido, ele deixa o caminho bem mais curto.*


15.1 Motivos bons e ruins para extrair
15.2 Avaliando um candidato
15.3 O padrão Strangler Fig
15.4 Os passos da extração
15.5 O que muda na extração
15.6 O caminho inverso
15.7 Antipadrões
15.8 Simulando a extração de Payments
15.9 Resumo e exercícios


## 15.1 Motivos bons e ruins para extrair

Extrair um módulo é trocar uma complexidade por outra. Precisa de uma razão concreta.

| Bons motivos | Motivos ruins |
|---|---|
| Um módulo precisa escalar de forma muito diferente | "Todo mundo usa microsserviços" |
| Times se bloqueiam em entregas e o custo é mensurável | "O monolito ficou grande" |
| Uma parte exige isolamento regulatório ou de segurança | "Queremos currículo moderno" |
| Uma parte precisa de outra tecnologia | "O código está bagunçado" |
| Um módulo com falhas frequentes derruba o restante | "O diretor pediu" |

Se o código está bagunçado, extrair só distribui a bagunça. Antes de extrair, **meça**: latência, frequência de entrega, conflitos entre times e custo de infraestrutura.

## 15.2 Avaliando um candidato

Bons candidatos têm características que você consegue verificar:

- **Poucas dependências de entrada e saída.** Contratos pequenos.
- **Sem ciclos.** Um módulo em um ciclo arrasta os outros.
- **Dados bem isolados.** Sem FK, sem `JOIN` externo.
- **Baixa frequência de conversa.** Chamadas frequentes viram latência.
- **Consistência eventual aceitável.** O negócio tolera um atraso.

## 15.3 O padrão Strangler Fig

Martin Fowler nomeou o padrão **Strangler Fig** em 2004, inspirado nas figueiras que crescem ao redor de uma árvore até substituí-la. A ideia é migrar aos poucos, sem uma grande reescrita. O sistema novo assume funcionalidades uma a uma, enquanto o antigo continua funcionando.

Em um monolito modular, o processo é mais simples, porque a costura já existe: **o contrato**.

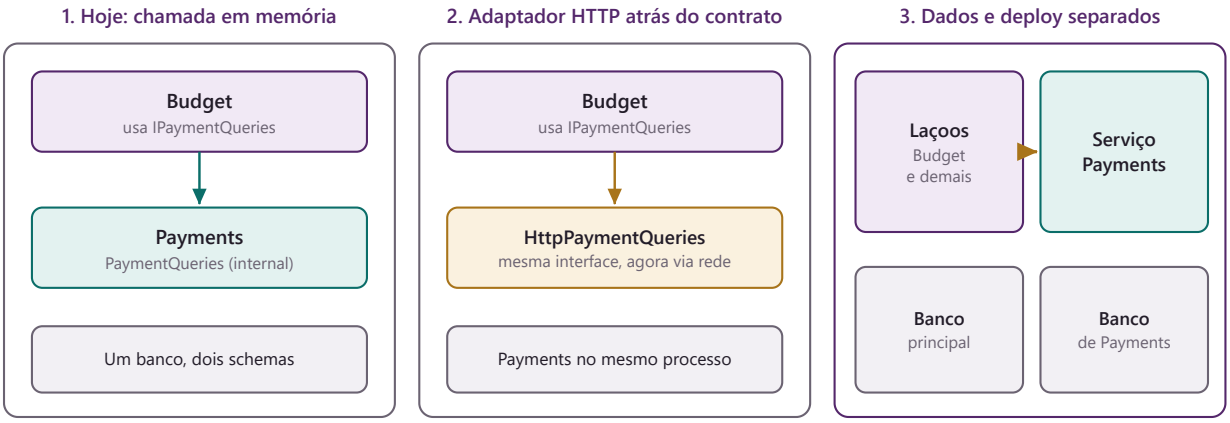



**Figura 15.1.** Etapas da extração. O contrato continua igual, e só a implementação por trás dele muda.

## 15.4 Os passos da extração

1. **Prepare.** Elimine os ciclos, verifique os dados e meça o que for preciso.
2. **Defina a API de rede.** Use o contrato do módulo como ponto de partida.
3. **Crie um adaptador.** Implemente o contrato com chamadas HTTP ou gRPC, ainda sem mover nada.
4. **Trate as falhas.** Defina timeouts, novas tentativas, disjuntores e comportamento quando o serviço estiver fora.
5. **Separe a implantação.** Rode o módulo em um processo próprio, com um pipeline próprio.
6. **Separe os dados.** Mova o schema para um banco próprio.
7. **Remova o código antigo.** Use um *feature flag* para alternar e, depois, apague.

## 15.5 O que muda na extração

A parte mais perigosa é o que **era implícito**. Em memória, muitas coisas funcionam sem você notar:

| Em memória | Pela rede |
|---|---|
| Contexto da requisição é compartilhado | Precisa ser enviado explicitamente |
| Autenticação já aconteceu | Precisa propagar identidade |
| Chamada não falha por rede | Timeout, nova tentativa, idempotência |
| Exceção atravessa a chamada | Vira um código de status |

| Em memória | Pela rede |
|---|---|
| Mesma versão de contrato | Compatibilidade e versionamento |

## 15.6 O caminho inverso

Fronteiras são hipóteses, e às vezes a evidência pede o caminho contrário. **Fundir módulos** é uma evolução legítima, e dentro de um monolito ela é barata: dois módulos que só mudam juntos e que conversam muito provavelmente são um só. O mesmo vale para serviços: a Segment voltou de mais de uma centena de serviços para um só, como veremos no próximo capítulo.

## 15.7 Antipadrões

Estes são os erros mais comuns ao longo da vida de um monolito modular:

| Antipadrão | Sintoma | Correção |
|---|---|---|
| **Módulo por entidade** | CRUDs rasos, regra espalhada | Modele capacidades |
| **Kernel lixeira** | `Common` crescendo sempre | Critério e devolução aos módulos |
| **"Só esse JOIN"** | SQL que cruza schemas | Composição ou modelo de leitura |
| **Contrato gordo** | Dezenas de métodos genéricos | Métodos orientados ao uso |
| **Entidade no contrato** | Tipos do EF Core expostos | Records próprios |
| **Fronteira sem teste** | Referências novas passam sem ninguém notar | Testes de arquitetura |
| **Conversa excessiva** | Um caso de uso faz dez chamadas a outro módulo | Reveja a fronteira |
| **Eventos por moda** | Fluxo impossível de seguir | Volte às chamadas diretas |
| **Módulo-deus** | Todos dependem dele | Divida ou mova as regras |
| **Extração prematura** | Serviço criado sem motivo mensurável | Espere a dor aparecer |

CONSTRUINDO O LAÇOOS · PASSO 15

### Simulando a extração de Payments

Vamos fazer um exercício honesto: e se precisássemos extrair o Payments? Essa etapa **não faz parte do repositório**. O objetivo é mostrar o que a arquitetura facilita e o que ela não resolve.

#### O que já está pronto

- O schema `payments` não tem nenhuma FK para fora.
- O interior é `internal`, e ninguém o acessa.
- O contrato `IPaymentQueries` lista tudo o que os outros precisam.
- O módulo tem suas próprias migrations.

## O adaptador

O primeiro passo seria uma classe que implementa o mesmo contrato usando HTTP. Os consumidores não mudam, e o `Register` escolheria a nova implementação.

**Adaptador HTTP atrás do mesmo contrato** · Exemplo ilustrativo: não faz parte do repositório

```
internal sealed class HttpPaymentQueries(HttpClient http, IWeddingContext wedding) : IPaymentQueries
{
    public async Task<bool> HasPaymentsForSupplierAsync(Guid supplierId, CancellationToken ct)
    {
        // O casamento precisa ir na URL: antes, vinha do escopo da requisição.
        var url = $"/internal/weddings/{wedding.WeddingId}/suppliers/{supplierId}/has-payments";
        return await http.GetFromJsonAsync<bool>(url, ct);
    }

    // Os demais métodos do contrato seguem a mesma forma.
}
```

## O que a arquitetura não resolve

É aqui que aparecem os problemas reais:

**1. O contexto implícito.** O contrato não tem um parâmetro `weddingId`, porque em memória o `WeddingContext` é compartilhado. Pela rede, esse contexto precisa viajar. Assim, a mesma facilidade que simplificou o código é um acoplamento escondido.

**2. O triângulo financeiro.** Payments depende de Budget e de Suppliers, e ambos dependem dele. Extraí-lo sem resolver isso criaria dependências de rede em círculo. Toda listagem de pagamentos passaria a fazer chamadas de volta ao monolito. Os ciclos que aceitamos no capítulo 7 cobram seu preço agora.

**3. As validações.** Hoje, `budget.ItemExistsAsync` é uma chamada garantida. Pela rede, o que fazer se o Budget estiver fora? Recusar o pagamento? Aceitar sem validar? É uma decisão de negócio, não técnica.

**4. As exclusões.** A verificação `HasPaymentsForSupplierAsync` antes de excluir um fornecedor fica mais sujeita a condições de corrida, porque a janela entre verificar e excluir aumenta.

**5. A composição do Dashboard.** Uma chamada em memória vira rede em todo acesso à tela inicial, que precisaria tolerar a falha desse serviço.

## A conclusão honesta

O contrato torna a mudança possível, mas não gratuita. A boa notícia é que **os problemas estão visíveis**: dá para listá-los em uma página, porque as dependências são explícitas. Em uma bola de lama, nem saberíamos por onde começar. Para o Laçoos, essa extração não se justifica hoje, e talvez nunca se justifique.

> **Para lembrar**
>
> O maior valor de um monolito modular para o futuro é **opcionalidade**: você pode extrair, mas não é obrigado. E se nunca precisar, nada foi desperdiçado, porque as fronteiras já se pagaram no dia a dia.

## 15.9 Resumo e exercícios

- Extraia por um motivo mensurável, nunca por moda.
- Bons candidatos têm contratos pequenos, nenhum ciclo e dados isolados.
- O padrão Strangler Fig migra aos poucos, e o contrato é a costura natural.
- O perigo está no que era implícito: contexto, identidade, falhas.
- Fundir módulos também é evolução.

### Exercícios

1. Qual módulo do Laçoos seria o melhor candidato à extração? Use os critérios do capítulo e o mapa da Figura 4.1.
2. Reescreva a assinatura de `HasPaymentsForSupplierAsync` para funcionar pela rede. O que muda?
3. Se o Budget estiver fora do ar, um pagamento deveria ser aceito? Discuta do ponto de vista do negócio.

CAPÍTULO 16

# O que o mercado aprendeu

*Casos reais são úteis quando lemos o contexto junto com a conclusão. O que funcionou em uma empresa nem sempre vale para a sua.*

16.1 Shopify: o monolito modular em grande escala
16.2 Segment: microsserviços de ida e volta
16.3 Prime Video: um serviço, não uma empresa
16.4 Google: separar escrita de implantação
16.5 Ferramentas por ecossistema
16.6 O que todos os casos têm em comum
16.7 Onde o Laçoos se encaixa
16.8 Resumo e exercícios

## 16.1 Shopify: o monolito modular em grande escala

A Shopify é o caso mais citado. Em 2019, Kirsten Westeinde publicou *Deconstructing the Monolith*, descrevendo um dos maiores códigos Ruby on Rails existentes, com mais de uma década de vida e mais de mil desenvolvedores.

O que eles fizeram:

- **Rejeitaram microsserviços** como solução geral, citando o custo de pipelines múltiplos, infraestrutura, latência e refatorações coordenadas.
- **Começaram por ouvir.** Em 2017, fizeram uma pesquisa com os desenvolvedores para entender as dores.
- **Reorganizaram o código por domínio.** Saíram de `models`, `views` e `controllers` para áreas como pedidos, envio, estoque e cobrança. Cerca de 6 mil classes foram rotuladas, e a mudança foi feita de forma automatizada.
- **Mediram as violações.** Uma ferramenta interna, o Wedge, acompanhava o progresso.
- **Colheram resultados.** O isolamento prévio permitiu trocar o antigo motor de impostos por um sistema novo, algo que, segundo eles, teria sido quase impossível antes.

Duas lições do texto valem para qualquer projeto: "a melhor hora de refatorar é o mais tarde possível", porque você aprende sobre o domínio enquanto constrói; e mudar de arquitetura deve acontecer quando as dores aparecem, não antes.

O retrospecto de 2024 sobre o Packwerk, visto no capítulo 13, é a continuação honesta dessa história: a modularização levou anos, isolar um único pacote levou muitos meses, e as ferramentas ajudaram menos do que o esperado em alguns pontos.

## 16.2 Segment: microsserviços de ida e volta

Em 2018, Alexandra Noonan publicou *Goodbye Microservices: From 100s of Problem Children to 1 Superstar*. A Segment entrega dados de clientes a centenas de destinos. Eles criaram um serviço e uma fila para cada destino, para isolar falhas.

A abordagem resolveu um problema e criou outros. Com mais de uma centena de serviços, o custo de manter bibliotecas, testes e implantações explodiu, e a produtividade caiu. A equipe voltou a um único serviço e um único repositório.

A lição não é "microsserviços são ruins". É que uma divisão **pelo critério errado**, um serviço por destino, multiplicou o trabalho sem trazer autonomia, porque uma equipe pequena mantinha tudo.

## 16.3 Prime Video: um serviço, não uma empresa

Em 2023, a equipe de análise de qualidade do Prime Video contou que moveu um serviço de monitoramento de áudio e vídeo de uma arquitetura distribuída para um único processo, reduzindo o custo de infraestrutura em mais de 90%.

A notícia correu como "Amazon abandona microsserviços". Isso é um exagero. O caso trata de **um** serviço, com um perfil específico: os dados de vídeo passavam entre componentes de forma muito intensa, e cada transferência tinha custo. O Prime Video como um todo continua sendo um sistema distribuído.

A lição útil é mais modesta e mais valiosa: **a fronteira deve seguir os dados**. Quando componentes trocam volumes grandes, colocá-los no mesmo processo pode ser muito melhor.

## 16.4 Google: separar escrita de implantação

Em 2023, pesquisadores do Google publicaram *Towards Modern Development of Cloud Applications*, no HotOS. O argumento: a forma de escrever o código e a forma de implantar deveriam ser decisões separadas. Eles propõem escrever a aplicação como um **monolito modular** e deixar um runtime decidir como distribuir os componentes. O artigo relata reduções de até 15 vezes na latência e 9 vezes no custo no protótipo, comparado à abordagem tradicional.

É interessante porque valida, em um ambiente com enorme experiência em sistemas distribuídos, a separação entre modularidade lógica e física do capítulo 2. O protótipo deu origem ao framework Service Weaver, para Go. Vale ler as críticas também: alguns autores apontam que o modelo se aplica melhor a um conjunto mais limitado de aplicações.

## 16.5 Ferramentas por ecossistema

| Ecossistema | Ferramentas e referências |
|---|---|
| **.NET** | NetArchTest, ArchUnitNET, Wolverine, projeto de referência `modular-monolith-with-ddd` |
| **Java** | Spring Modulith, ArchUnit, sistema de módulos da linguagem |
| **Ruby** | Packwerk, Rails Engines |
| **Go** | Pacotes `internal`, Service Weaver |
| **PHP** | Deptrac |
| **Node.js/TS** | Nx, dependency-cruiser, NestJS modules |

### Spring Modulith

O **Spring Modulith** é o exemplo mais maduro de suporte de framework. Ele interpreta os pacotes de uma aplicação Spring Boot como módulos, separa o que é API do que é interno e verifica a estrutura em um teste. O **Event Publication Registry** grava as publicações de eventos na transação original, em um mecanismo semelhante ao Outbox. Na consulta feita em novembro de 2026, o anúncio mais recente era o da versão 2.2 RC1, e a linha estável era a 2.1.

### .NET

Não existe um equivalente oficial ao Spring Modulith em .NET. O ecossistema combina peças: projetos separados, `internal`, testes de arquitetura e uma biblioteca de mensagens, se houver necessidade.

O repositório `modular-monolith-with-ddd`, de Kamil Grzybek, é uma referência importante. É uma aplicação completa, com módulos, CQRS, eventos, Outbox e testes de arquitetura, além de um registro de decisões. É bem mais elaborado do que o Laçoos, e vale estudá-lo depois deste livro, entendendo que nem todo sistema precisa daquele nível de estrutura.

> **Licenças mudam**
>
> Em 2025, MediatR e AutoMapper passaram a ter licença comercial nas versões novas, e o MassTransit anunciou o mesmo para a versão 9. As versões antigas seguem com suas licenças originais, mas tendem a deixar de evoluir. Antes de adotar uma dependência central, verifique a licença atual. Também é um bom argumento para manter essas dependências atrás de abstrações suas.

## 16.6 O que todos os casos têm em comum

Olhando os casos em conjunto, os padrões se repetem:

- **O contexto decide.** Nenhum caso prova uma arquitetura universal.
- **Boas fronteiras vêm antes da tecnologia.** Todas as histórias de sucesso cuidaram disso.
- **Mudanças são incrementais.** Quem evoluiu aos poucos teve melhores resultados.
- **Medir ajuda.** Violações, custos e tempo de entrega embasaram as decisões.
- **Mudar de ideia é normal.** A Segment voltou atrás, e a Shopify revisou suas ferramentas.

> **Viés de sobrevivência**
>
> Artigos de engenharia contam histórias que deram certo, ou que ao menos renderam uma boa lição, e vêm de empresas com um contexto muito distante da maioria. Trate cada caso como um dado, e não como uma receita. O texto de Fowler, *MonolithFirst*, reconhece abertamente que as evidências sobre esse tema são limitadas.

## 16.7 Onde o Laçoos se encaixa

Comparado ao mercado, o Laçoos é um monolito modular **enxuto**:

| Prática de mercado | Laçoos |
|---|---|
| Módulos por capacidade de negócio | Sim |
| Interior oculto pelo compilador | Sim |
| Schema e migrations por módulo | Sim |
| Testes de arquitetura | Sim |
| Chamadas síncronas por contrato | Sim |
| Eventos com Outbox | Não: passos ordenados |
| CQRS com modelos de leitura | Não: composição |
| Modelo de domínio rico | Parcial: regras puras |
| Observabilidade por módulo | Não |

Essas ausências são **deliberadas**. Cada uma tem um gatilho conhecido para ser reavaliada, como veremos no último capítulo.

## 16.8 Resumo e exercícios

- A Shopify mostrou que dá para modularizar em grande escala, com esforço e por muitos anos.
- A Segment mostrou o custo de uma divisão pelo critério errado.
- O caso do Prime Video trata de um serviço específico, não de uma mudança de empresa.
- O Google propõe separar a escrita do código da decisão de implantação.
- Trate casos de mercado como evidência, e não como receita.

### Exercícios

1. Leia o texto original da Shopify e identifique uma decisão que não se aplicaria ao seu contexto.
2. Por que a divisão da Segment por destino não trouxe autonomia de times?
3. Compare o `modular-monolith-with-ddd` com o Laçoos. Que estrutura dele você adotaria hoje, e o que deixaria de fora?

CAPÍTULO 17

# O Laçoos de ponta a ponta

*Agora que todas as peças foram apresentadas, este capítulo mostra o sistema inteiro: o que foi construído, como cada conceito apareceu e o que ainda falta.*


17.1 O sistema em números
17.2 Uma requisição do início ao fim
17.3 Mapa: do conceito ao código
17.4 As decisões que definiram o projeto
17.5 O que está imperfeito
17.6 Gatilhos para evoluir
17.7 Levando para o seu projeto
17.8 Palavras finais
17.9 Exercícios finais


## 17.1 O sistema em números

Estes números vêm do repositório, do banco em execução e da última execução dos testes.

| Medida | Valor |
|---|---|
| Módulos de negócio | 9 |
| Projetos na solução | 22 |
| Schemas no PostgreSQL | 8 |
| Tabelas de negócio | 24 |
| Chaves estrangeiras | 17, nenhuma entre schemas |
| Caminhos de API no OpenAPI | 58 |
| Operações HTTP | 101 |
| Casos de teste | 236: 53 de unidade, 64 de arquitetura, 119 de integração |
| Linhas de C# (sem migrations) | Cerca de 9.900 |

Olhando o tamanho de cada módulo, percebe-se quão pequenos são os contratos em relação ao interior:

| Módulo | Linhas no interior | Linhas no contrato |
|---|---|---|
| Payments | 1.026 | 37 |
| Gifts | 837 | 10 |
| Weddings | 774 | 13 |
| Suppliers | 712 | 10 |
| Identity | 689 | 13 |
| Budget | 536 | 16 |
| Guests | 521 | 12 |
| Checklist | 479 | 19 |

| Módulo | Linhas no interior | Linhas no contrato |
|---|---|---|
| Dashboard | 103 | Nenhum |

O kernel tem 754 linhas, o host tem 309, e os testes, 3.021.

Essa segunda tabela é a ideia de módulo profundo do capítulo 3, em forma de número. O `Payments` esconde mais de mil linhas atrás de um contrato de 37. Quem o consome precisa conhecer menos de 4% do código.

## 17.2 Uma requisição do início ao fim

Vamos acompanhar um `POST` que cria um pagamento e ver onde cada camada entra.

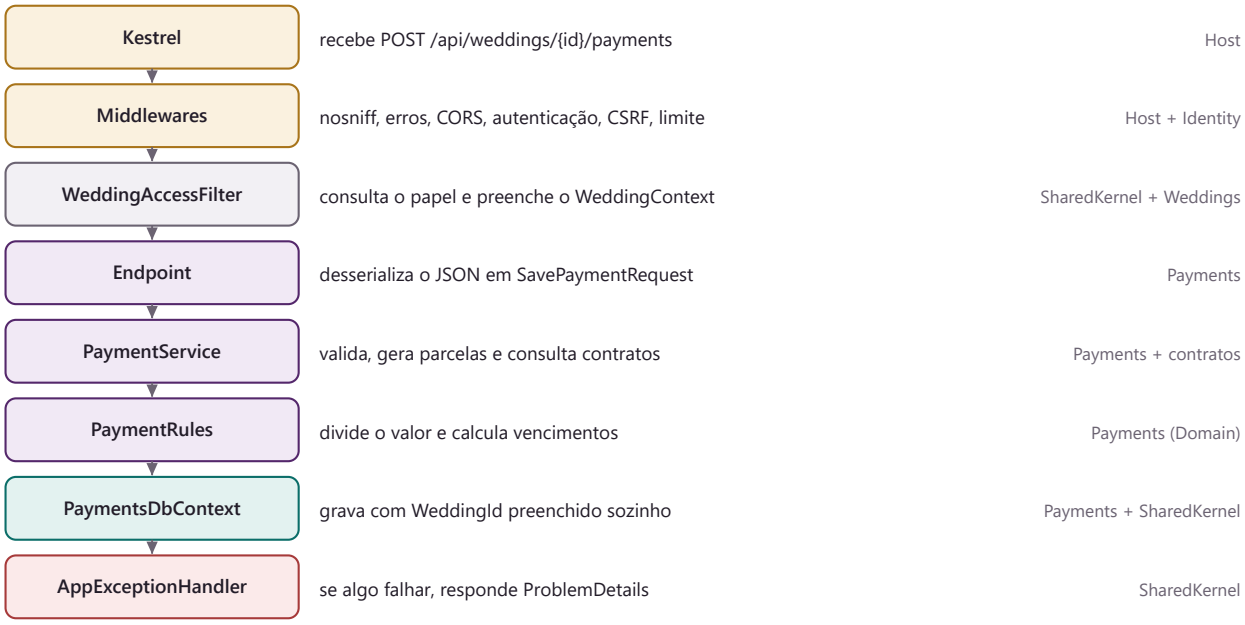



**Figura 17.1.** O caminho completo de uma requisição, com a parte do sistema responsável por cada etapa.

Observe a coluna da direita. Em nenhum momento o código de Payments trata autenticação, papéis, formato de erro ou isolamento. O módulo só precisa pensar em pagamentos. Essa é a recompensa da organização: **cada parte tem um assunto só**.

## 17.3 Mapa: do conceito ao código

Esta tabela é o índice do projeto. Para cada conceito, ela aponta onde ele está aplicado.

| Conceito | Capítulo | Onde ver no repositório |
|---|---|---|
| Módulo por capacidade | 3, 4 | `src/Modules/` |
| Interior oculto | 3 | Classes `internal` em cada módulo |
| Versões centralizadas | 3 | `Directory.Packages.props` |
| Composition root | 5 | `src/Host/Lacoos.Api/Program.cs` |

| Conceito | Capítulo | Onde ver no repositório |
|---|---|---|
| Porta de entrada | 5 | `SharedKernel/Modules/IModule.cs` |
| Regras puras | 6 | `Payments/Domain/PaymentRules.cs` |
| Histórico imutável | 6 | `Payments/Domain/Payment.cs` |
| Contrato orientado ao uso | 7 | `Budget.Contracts/BudgetContracts.cs` |
| Consulta em lote | 7 | `IBudgetQueries.GetItemNamesAsync` |
| Schema por módulo | 8 | `SharedKernel/Persistence/ModuleDbContext.cs` |
| Referência por identificador | 8 | `Budget/Domain/BudgetItem.cs` |
| Migrations por módulo | 8 | `*/Persistence/Migrations/` |
| Passos ordenados | 9 | `Guests/Application/GuestService.cs` |
| Operação idempotente | 9 | `Gifts/Application/GiftGuestLinks.cs` |
| Trava pessimista | 9 | `SharedKernel/Persistence/RowLock.cs` |
| Multi-tenancy automático | 10 | `SharedKernel/Tenancy/` |
| Inversão de dependência | 10 | `IWeddingMembershipResolver` |
| Tratamento central de erros | 10 | `SharedKernel/Web/AppExceptionHandler.cs` |
| Portas de infraestrutura | 10 | `IFileStorage`, `IEmailSender`, `IClock` |
| Composição de API | 11 | `Dashboard/DashboardModule.cs` |
| Testes com banco real | 12 | `IntegrationTests/Support/ApiFixture.cs` |
| Teste por varredura | 12 | `IntegrationTests/AccessControlTests.cs` |
| Isolamento testado | 12 | `IntegrationTests/TenantIsolationTests.cs` |
| Funções de aptidão | 13 | `ArchitectureTests/ModuleBoundaryTests.cs` |
| Imagem em estágios | 14 | `Dockerfile` |
| Migration como etapa | 14 | Perfil `migrate` do `docker-compose.yml` |

## 17.4 As decisões que definiram o projeto

| Decisão | Alternativa rejeitada | Motivo |
|---|---|---|
| Monolito modular | Microsserviços | Time pequeno, operação simples |
| Schema por módulo | Banco por módulo | Mesmo isolamento lógico, custo muito menor |
| Sem FK entre módulos | FK entre schemas | Autonomia e extração futura |
| Projeto de contrato separado | Tipos públicos no módulo | Compilador protege, sem ciclos de build |
| Chamadas síncronas | Eventos | Poucos fluxos, depuração simples |
| Passos ordenados | Transação compartilhada | Mantém a fronteira também em execução |
| Filtro global de casamento | `WHERE` manual | Segurança por padrão |
| Composição no Dashboard | Modelo de leitura | Volume pequeno, dados sempre atuais |

| Decisão | Alternativa rejeitada | Motivo |
|---|---|---|
| `DbContext` usado direto | Repositórios | Menos código, sem ganho real aqui |
| Sessão em banco e cookie | JWT | Logout imediato e menos exposição ao JavaScript |
| Lista explícita de módulos | Reflexão | Simples e visível |

## 17.5 O que está imperfeito

Um projeto honesto termina com uma lista do que precisa melhorar. Reunimos aqui as limitações conhecidas.

### Arquitetura

- **Ciclos entre contratos.** Cinco pares dependem um do outro, com o triângulo financeiro como principal ponto de atenção.
- **Sem transação entre módulos.** Operações que cruzam módulos acontecem em passos ordenados.
- **Corrida em exclusões.** Sem FK, há uma janela pequena para referências órfãs.
- **Regras ainda sem cobertura.** SQL manual e novos ciclos não são detectados.
- **Modelo próximo do anêmico.** Parte das regras depende dos serviços.

### Funcionalidade

- Parcelas em `próximas` não incluem as atrasadas, que aparecem separadas.
- Parcelas de valor zero contam como pagas.
- A busca não ignora acentos.
- O status do item de orçamento (`planned`, `contracted`, `cancelled`) foi uma suposição, porque o texto original da especificação estava incompleto. Ele precisa ser conferido com o front-end.

### Validação

- O sistema foi testado de ponta a ponta em contêineres locais, mas **não em produção**. HTTPS, SMTP real e a integração com o Angular ainda não foram validados.
- Não há testes de carga.
- O fluxo de clique no Swagger UI não foi testado em um navegador real, embora o script de CSRF tenha sido verificado isoladamente.

## 17.6 Gatilhos para evoluir

Cada ausência tem um motivo para ser revista. Em vez de adotar padrões por precaução, defina os gatilhos:

| Se isso acontecer... | Considere... |
|---|---|
| Mais de um consumidor precisa reagir à mesma ação | Eventos em memória |

| Se isso acontecer... | Considere... |
|---|---|
| Uma falha entre passos gera inconsistência percebida pelo usuário | Outbox |
| O Dashboard ficar lento com dados reais | Modelo de leitura |
| Mudanças financeiras sempre tocam três módulos | Fundir Budget, Payments e Suppliers |
| Regras de pagamento se repetirem nos serviços | Modelo de domínio mais rico |
| Um módulo exigir escala ou time próprio | Extração com Strangler Fig |
| Mais pessoas passarem a mexer em contratos | `CODEOWNERS` e lista de dependências |
| Implantar em produção com várias réplicas | Armazenamento externo de arquivos |
| Aparecer um incidente difícil de diagnosticar | OpenTelemetry por módulo |

> **Para lembrar**
>
> Uma boa arquitetura sabe o que **deixou de fazer** e sob quais condições mudaria de ideia.

## 17.7 Levando para o seu projeto

Se você quer aplicar o que viu, siga uma ordem que funciona tanto em um sistema novo quanto em um existente.

### Em um sistema novo

1. Descreva o domínio e encontre os contextos.
2. Crie a estrutura, com o host, o kernel e um módulo.
3. Adicione os testes de arquitetura **no primeiro dia**, enquanto são fáceis.
4. Construa um fluxo completo, do endpoint ao banco.
5. Repita com o próximo módulo, revendo as fronteiras conforme aprende.

### Em um sistema existente

1. Mapeie as dependências atuais e encontre as áreas mais independentes.
2. Mova o código para pastas ou projetos por domínio, sem mudar comportamento.
3. Adicione regras de arquitetura com uma lista de exceções.
4. Separe os dados aos poucos, começando por um schema.
5. Reduza as exceções como uma catraca: o número só pode cair.

## 17.8 Palavras finais

Monolito modular não é uma moda nova nem um meio-termo sem graça. É uma aplicação cuidadosa de ideias antigas: esconder decisões, depender de abstrações e dar um dono a cada dado. Parnas descreveu isso em 1972.

O que o torna atual é a percepção de que muita dor em sistemas não vem da falta de serviços, e sim da falta de fronteiras. Se você guardar uma coisa deste livro, que seja esta: **escolha suas fronteiras com cuidado, proteja-as com automação e esteja disposto a mudá-las quando aprender algo novo**.

## 17.9 Exercícios finais

1. Escolha um item da lista de limitações e implemente a melhoria. Quais arquivos mudam? O impacto ficou contido em um módulo?
2. Adicione ao Laçoos um módulo novo e pequeno, como `Notes`, para anotações do casamento. Siga todos os passos: projetos, `IModule`, schema, migration, endpoints, contrato, se necessário, e testes de arquitetura.
3. Escreva os ADRs de três decisões da tabela deste capítulo, incluindo as consequências negativas.

# Apêndice A: Checklist de revisão

Use esta lista para avaliar um monolito modular, ou para revisar um módulo novo.

### Fronteiras

- ☐ Cada módulo tem uma responsabilidade que cabe em uma frase.
- ☐ Os módulos representam capacidades de negócio, não tabelas.
- ☐ Sei dizer qual é o dono de cada informação importante.
- ☐ Palavras com significados diferentes estão em contextos diferentes.
- ☐ O que ficou fora do escopo está registrado.

### Encapsulamento

- ☐ O compilador impede o acesso ao interior de um módulo.
- ☐ Só a porta de entrada e os contratos são públicos.
- ☐ Os contratos não expõem entidades, `IQueryable` nem tipos de infraestrutura.
- ☐ As dependências de cada módulo podem ser vistas em um arquivo.

### Dados

- ☐ Toda tabela tem um único módulo dono.
- ☐ Nenhum módulo lê ou grava tabelas de outro.
- ☐ Cada módulo tem seu schema e suas migrations.
- ☐ Referências entre módulos são apenas identificadores.
- ☐ Existe índice para as colunas de referência.

### Comunicação e consistência

- ☐ Contratos são pequenos e orientados ao uso.
- ☐ Consultas entre módulos funcionam em lote.
- ☐ Cada operação entre módulos tem falhas mapeadas.
- ☐ A ordem dos passos deixa o sistema em um estado aceitável em caso de falha.
- ☐ Operações repetíveis são idempotentes.
- ☐ Regras de limite estão protegidas contra concorrência.
- ☐ Os ciclos entre módulos são conhecidos e justificados.

### Compartilhado

- ☐ O kernel não tem regras de negócio.
- ☐ Não existe uma pasta `Common` ou `Utils` crescendo sem controle.
- ☐ Autenticação, erros e isolamento são aplicados por padrão.

- ☐ Falhas de contexto resultam em nenhum dado, e não em todos os dados.

### Testes e verificação

- ☐ Testes de arquitetura rodam no pipeline.
- ☐ Toda regra de arquitetura foi vista falhando ao menos uma vez.
- ☐ Regras puras têm testes de unidade.
- ☐ Testes de integração usam o banco real.
- ☐ Existe teste de isolamento entre clientes.
- ☐ Rotas novas são cobertas automaticamente por testes de segurança.

### Operação

- ☐ Um artefato é construído uma vez e promovido.
- ☐ Nenhum segredo está no repositório.
- ☐ Migrations rodam como etapa explícita em produção.
- ☐ Existe verificação de saúde que testa dependências reais.
- ☐ A aplicação não guarda estado local que impeça a escala horizontal.
- ☐ Logs e métricas identificam o módulo.

### Decisões

- ☐ Decisões importantes estão em ADRs, com as consequências negativas.
- ☐ Cada padrão ausente tem um gatilho para ser reavaliado.

# Apêndice B: Glossário

**ACID.** Atomicidade, consistência, isolamento e durabilidade: garantias de uma transação de banco.

**ADR (Architecture Decision Record).** Documento curto que registra uma decisão, seu contexto e suas consequências.

**Acoplamento.** Grau em que uma parte depende de outra.

**Agregado.** No DDD, conjunto de objetos tratado como uma unidade de consistência.

**API Composition.** Montar uma resposta chamando vários módulos e juntando em memória.

**At-least-once.** Garantia de entrega em que a mensagem chega pelo menos uma vez, podendo repetir.

**Bounded context (contexto delimitado).** Fronteira em que um modelo e sua linguagem têm um significado único.

**Coesão.** Grau em que os elementos de uma parte tratam do mesmo assunto.

**Composition root.** Ponto único em que as dependências da aplicação são ligadas.

**Consistência eventual.** Propriedade em que dados separados ficam iguais depois de algum tempo, não de imediato.

**Contrato.** API pública de um módulo: interfaces e tipos de que os outros podem depender.

**CQRS.** Separação entre o modelo de escrita e o modelo de leitura.

**CQS.** Princípio em que um método ou consulta, ou altera, mas não os dois.

**CSRF.** Ataque que faz o navegador da vítima enviar uma requisição autenticada sem que ela queira.

**DDD (Domain-Driven Design).** Abordagem de design centrada no domínio de negócio.

**Dual write (gravação dupla).** Gravar em dois sistemas sem uma transação comum, o que pode deixá-los diferentes.

**Evento de domínio.** Fato interno a um módulo.

**Evento de integração.** Fato publicado para outros módulos, que faz parte do contrato.

**Fitness function (função de aptidão).** Verificação automática de uma característica da arquitetura.

**Host.** Projeto executável que carrega e inicia os módulos.

**Idempotência.** Propriedade de uma operação cujo resultado é o mesmo se executada uma ou várias vezes.

**Inbox.** Registro de mensagens já processadas, para ignorar repetidas.

**Information hiding (ocultamento de informação).** Esconder dentro de um módulo as decisões que podem mudar.

**Linguagem ubíqua.** Vocabulário comum entre especialistas e desenvolvedores, presente no código.

**Microsserviços.** Sistema formado por serviços independentes que se comunicam pela rede.

**Modelo anêmico.** Entidades só com dados, com a lógica em serviços.

**Modelo de leitura (read model).** Estrutura de dados própria para uma consulta.

**Módulo profundo.** Módulo com interface pequena e muita funcionalidade.

**Monolito.** Sistema implantado como uma unidade.

**Monolito distribuído.** Vários serviços que precisam ser implantados juntos.

**Monolito modular.** Sistema implantado como uma unidade, com fronteiras internas fortes.

**Multi-tenancy.** Atender vários clientes isolados com o mesmo sistema.

**Outbox.** Tabela que guarda eventos na mesma transação do dado, para entrega posterior.

**Porta (port).** Interface que define o que o código precisa do mundo externo.

**ProblemDetails.** Formato padronizado de erro em APIs HTTP (RFC 9457).

**Saga.** Processo em vários passos, com ações de compensação em vez de uma transação única.

**Schema.** Espaço de nomes dentro de um banco de dados.

**Shared kernel.** Pequena parte do código compartilhada por vários módulos.

**Strangler Fig.** Migrar um sistema aos poucos, substituindo partes enquanto ele funciona.

**Testcontainers.** Biblioteca que sobe contêineres descartáveis durante os testes.

**Transaction Script.** Organizar a lógica como um procedimento por caso de uso.

**Vertical Slice.** Organizar o código por funcionalidade, e não por camada.

# Apêndice C: Referências

## Livros


Evans, Eric. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003.

Ford, Neal; Parsons, Rebecca; Kua, Patrick. *Building Evolutionary Architectures*. O'Reilly, 2017.

Fowler, Martin. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.

Martin, Robert C. *Clean Architecture*. Prentice Hall, 2017.

Newman, Sam. *Monolith to Microservices*. O'Reilly, 2019.

Newman, Sam. *Building Microservices*, 2ª edição. O'Reilly, 2021.

Ousterhout, John. *A Philosophy of Software Design*. Yaknyam Press, 2018.

Richardson, Chris. *Microservices Patterns*. Manning, 2018.

Seemann, Mark. *Dependency Injection in .NET*. Manning, 2011.

Skelton, Matthew; Pais, Manuel. *Team Topologies*. IT Revolution, 2019.

Vernon, Vaughn. *Implementing Domain-Driven Design*. Addison-Wesley, 2013.

Yourdon, Edward; Constantine, Larry. *Structured Design*. Prentice Hall, 1979.


## Artigos clássicos


Conway, Melvin. *How Do Committees Invent?* Datamation, 1968.

Foote, Brian; Yoder, Joseph. *Big Ball of Mud*. PLoP, 1997.

Garcia-Molina, Hector; Salem, Kenneth. *Sagas*. ACM SIGMOD, 1987.

Helland, Pat. *Life beyond Distributed Transactions*. CIDR, 2007.

Parnas, David. *On the Criteria To Be Used in Decomposing Systems into Modules*. Communications of the ACM, 1972.


## Textos e casos de mercado


Brown, Simon. *Modular Monoliths*. GOTO Berlin, 2018. https://gotopia.tech/sessions/515/modular-monoliths

Cockburn, Alistair. *Hexagonal Architecture*, 2005.

Fowler, Martin. *MonolithFirst*, 2015. https://martinfowler.com/bliki/MonolithFirst.html

Fowler, Martin. *StranglerFigApplication*, 2004. https://martinfowler.com/bliki/StranglerFigApplication.html

Ghemawat, Sanjay et al. *Towards Modern Development of Cloud Applications*. HotOS, 2023.

McGibbon, Gannon; Salzberg, Chris. *A Packwerk Retrospective*. Shopify Engineering, 2024. https://shopify.engineering/a-packwerk-retrospective

Noonan, Alexandra. *Goodbye Microservices: From 100s of Problem Children to 1 Superstar*. Segment, 2018.

Nygard, Michael. *Documenting Architecture Decisions*, 2011.

Prime Video Tech. *Scaling up the Prime Video audio/video monitoring service and reducing costs by 90%*, 2023.

Richardson, Chris. *Pattern: Transactional outbox*. https://microservices.io/patterns/data/transactional-outbox

Westeinde, Kirsten. *Deconstructing the Monolith*. Shopify Engineering, 2019. https://shopify.engineering/deconstructing-monolith-designing-software-maximizes-developer-productivity

Wiggins, Adam. *The Twelve-Factor App*. https://12factor.net

## Ferramentas e projetos

ArchUnitNET. https://www.nuget.org/packages/TngTech.ArchUnitNET

Grzybek, Kamil. *modular-monolith-with-ddd*. https://github.com/kgrzybek/modular-monolith-with-ddd

NetArchTest. https://www.nuget.org/packages/NetArchTest.Rules

Packwerk. https://github.com/Shopify/packwerk

Spring Modulith. https://docs.spring.io/spring-modulith/reference/

Testcontainers. https://testcontainers.com

Wolverine. https://wolverinefx.net

## Licenças mencionadas

Bogard, Jimmy. *AutoMapper and MediatR Commercial Editions Launch Today*, 2025. https://www.jimmybogard.com/automapper-and-mediatr-commercial-editions-launch-today/

MassTransit. *Announcing MassTransit v9*. https://masstransit.io/introduction/v9-announcement

*Informações de mercado, versões de ferramentas e licenças foram consultadas em novembro de 2026 e podem mudar.*